

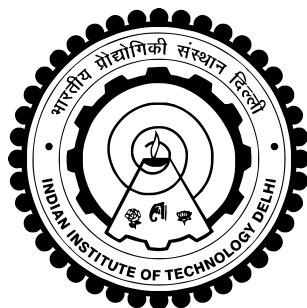
Distributed Least Squares Solutions for Linear Equations

by

Raswanth J & Sankarshan Sawhney
(2021EE30179) & (2021EE30175)

under the guidance of

Professor Priyank Srivastava



Department of Electrical Engineering
Indian Institute of Technology Delhi
May 2025

Abstract

In modern large-scale systems such as sensor networks, distributed robotics, and power grids, centralized computation is often infeasible due to bandwidth limitations, privacy constraints, and communication bottlenecks. In such systems, it becomes imperative to design distributed algorithms that enable agents to collaboratively solve global problems using only local information exchange. A fundamental and frequently encountered task in these applications is the solution of a set of linear equations, which may arise from modeling, estimation, or control problems.

In most practical scenarios, system dynamics or observations can be linearized, transforming the underlying problem into one of solving linear equations. However, due to measurement noise or model imperfections, these systems often become inconsistent. In such cases, finding a least squares solution becomes the most viable approach. Importantly, we aim to achieve this without requiring any agent to broadcast its data globally—preserving data privacy and reducing communication overhead.

This work addresses the challenge of solving linear equations in a fully distributed manner. We design a novel algorithm that computes the least squares solution even in the presence of inconsistencies, and we demonstrate its adaptability to varying network topologies. We begin with the case of undirected and connected graphs, extend the method to directed but weight-balanced graphs, and ultimately generalize it to arbitrary strongly connected and weight-unbalanced directed graphs.

Our proposed approach integrates consensus dynamics, gradient-based optimization, and auxiliary variables to ensure convergence to the least squares solution. We verify performance through extensive simulations. These simulations assess the behavior of the algorithm under static and time-varying graphs, highlighting the scalability and robustness of the proposed solution.

This report not only contributes a new distributed solver but also offers a deeper understanding of the trade-offs involved in designing linear solvers over networks. The findings hold significant promise for application in decentralized estimation, distributed control, federated learning, and privacy-aware multi-agent systems.

Contents

1	Introduction	2
1.1	Problem Formulation	3
1.2	Literature Review	4
2	Solution in Undirected Networks	6
3	Observations for Simulations in Undirected network	9
4	Algorithm for Directed Weight-balanced Networks	15
4.1	Centralized Algorithm	15
4.2	Distributed Algorithm	16
5	Observations for Simulations in Directed Weight-balanced network	17
5.1	Observation for a 4-Node graph	17
5.2	Observation for a 6-Node graph	20
5.3	Inference	25
6	Algorithm for Directed Weight-Unbalanced Networks	26
6.1	Algorithm for globally known Graph network	26
6.2	Algorithm for unknown Graph network	26
7	Observations for Simulations in Directed Weight-Unbalanced network	28
7.1	Observation for a 4-Node graph	28
7.2	Observation for a different 4-Node graph	31
7.3	Observation for a 7-Node graph	35
8	Results from Multi Node systems	40
8.1	Multi-file Implementation in a single computer	40
8.2	Raspberry Pi Implementation in LAN (Using Ethernet)	47
8.3	Raspberry Pi Implementation in WiFi (Using Internet)	54
9	Observations of Plots for Time varying A, b and Graph structure	58

List of Figures

1.1	Example of a 4 Node graph	4
3.1	Ring graph	9
3.2	Convergence of x in Node 1 in Ring graph	10
3.3	Convergence of x in Node 2 in Ring graph	10
3.4	Convergence of x in Node 3 in Ring graph	11
3.5	Convergence of x in Node 4 in Ring graph	11
3.6	Dense graph	12
3.7	Convergence of x in Node 1 in dense graph	12
3.8	Convergence of x in Node 2 in dense graph	13
3.9	Convergence of x in Node 3 in dense graph	13
3.10	Convergence of x in Node 4 in dense graph	14
5.1	4 Node graph used for implementation	17
5.2	Convergence of x in Node 1 in the above graph network	18
5.3	Convergence of x in Node 2 in the above graph network	19
5.4	Convergence of x in Node 3 in the above graph network	19
5.5	Convergence of x in Node 4 in the above graph network	20
5.6	6-Node graph used for implementation	20
5.7	Convergence of x in Node 1 in the above graph network	22
5.8	Convergence of x in Node 2 in the above graph network	23
5.9	Convergence of x in Node 3 in the above graph network	23
5.10	Convergence of x in Node 4 in the above graph network	24
5.11	Convergence of x in Node 5 in the above graph network	24
5.12	Convergence of x in Node 6 in the above graph network	25
7.1	4 Node graph used	28
7.2	Convergence of x in Node 1 in the above graph network	29
7.3	Convergence of x in Node 2 in the above graph network	30
7.4	Convergence of x in Node 3 in the above graph network	30
7.5	Convergence of x in Node 4 in the above graph network	31
7.6	4 Node graph used for upcoming implementation	31
7.7	Convergence of x in Node 1 in the above graph network	33
7.8	Convergence of x in Node 2 in the above graph network	33
7.9	Convergence of x in Node 3 in the above graph network	34
7.10	Convergence of x in Node 4 in the above graph network	34
7.11	7-Node graph used	35
7.12	Convergence of x in Node 1 in the above graph network	36
7.13	Convergence of x in Node 2 in the above graph network	37
7.14	Convergence of x in Node 3 in the above graph network	37

7.15	Convergence of x in Node 4 in the above graph network	38
7.16	Convergence of x in Node 5 in the above graph network	38
7.17	Convergence of x in Node 6 in the above graph network	39
7.18	Convergence of x in Node 7 in the above graph network	39
8.1	Normal 4 Node graph used	40
8.2	Convergence of x in Node 1	41
8.3	Convergence of x in Node 1	41
8.4	Convergence of x in Node 1	42
8.5	Normal 6-Node graph used	42
8.6	Convergence of x in Node 1	44
8.7	Convergence of x in Node 2	44
8.8	Convergence of x in Node 3	45
8.9	Convergence of x in Node 1	46
8.10	Convergence of x in Node 2	46
8.11	Convergence of x in Node 3	47
8.12	Normal 4 Node graph used	47
8.13	Picture of the setup with Raspberry Pi	48
8.14	Picture with LAN connections	48
8.15	Picture of the whole setup	49
8.16	Convergence of x in Node 1	50
8.17	Convergence of x in Node 1	51
8.18	Different 4 Node graph used	51
8.19	Convergence of x in Node 1	52
8.20	Convergence of x in Node 1	53
8.21	Normal 4 Node graph used	54
8.22	Picture of the setup with Raspberry Pi	54
8.23	Picture of the whole setup	55
8.24	Convergence of x in Node 1	56
8.25	Convergence of x in Node 2	56
9.1	Convergence of x in Node 3	59
9.2	Change every 100000 iteration	60
9.3	Change every 50000 iteration	60
9.4	Change every 25000 iteration	61
9.5	Change every 10000 iteration	61
9.6	Change every 100000 iteration for high η	62
9.7	Change every 300000 iteration for low η	63
9.8	Change every 50000 iteration for low η	63
9.9	Plot for changes without y	64
9.10	Plot for changes without y	65
9.11	Old 4 Node graph used	65
9.12	New 4 Node graph used	66
9.13	Plot for changes without y	66
9.14	Plot for changes without y	67
9.15	Plot for changes without y	67
9.16	Plot for normal convergence without any change in the 7 Node system . .	68
9.17	Plot for change in graph network in a 7 Node system	69

Statement of Contribution

This project was jointly carried out by Raswanth J (2021EE30179) and Sankarshan Sawhney (2021EE30175) as part of the fulfillment for the course ELD450 (BTP Part-2) in Electrical Department under Control and Automation.

Both authors contributed equally to the literature review, algorithm design, theoretical analysis, simulations, and documentation. Specific responsibilities were shared as follows:

- **Raswanth J**

- Designed and executed simulations for algorithm validation.
- Helped in development of the mathematical formulation of the distributed algorithms.
- Tuned parameters and analyzed numerical performance.
- Involved in practical implementation with Raspberry Pi in LAN and Wi-Fi network.
- Organized the structure of the report and handled graphical representations.

- **Sankarshan Sawhney**

- Helped in design and executed simulations for algorithm validation
- Developed the mathematical formulation of the distributed algorithms.
- Tuned parameters and analyzed numerical performance.
- Involved in practical implementation with Raspberry Pi in LAN and Wi-Fi network.
- Organized the structure of the report and handled graphical representations.

All discussions, reviews, and final decisions were made jointly, and the outcomes presented in this report reflect equal and collaborative effort from both contributors.

Chapter 1

Introduction

The efficient solution of linear equations forms the backbone of numerous scientific and engineering applications, ranging from modeling physical systems to optimizing complex networks. Traditional techniques typically rely on centralized computation, which can be both resource-intensive and impractical when applied to large-scale or geographically distributed systems. In contrast, distributed algorithms offer a scalable and robust alternative by enabling multiple computational nodes to collaboratively solve global problems using only local data and communication. In literature there has been many efficient approaches to solve linear equations, this can be seen in [1] and [2].

This project investigates the design, implementation, and performance of distributed algorithms for solving linear equations in networked environments. The central idea is to decompose a complex computational task into smaller sub-tasks that can be processed in parallel across various nodes in a system. Such an approach inherently enhances scalability and fault tolerance, making it especially suitable for cloud computing platforms, multi-core architectures, and decentralized networks.

The report lays a strong emphasis on the mathematical foundations and theoretical underpinnings of distributed linear solvers. It explores the algorithmic frameworks that govern these solvers, with particular attention to consensus-based methods. A consensus algorithm, in this context, ensures that all nodes in a distributed network agree on a common solution through iterative, local interactions. This is vital for maintaining consistency and reliability in distributed systems, especially in applications where communication may be limited or asynchronous.

The primary objective of this work is to develop a distributed consensus algorithm capable of computing the least squares solution to a system of linear equations. The algorithm accommodates inconsistent systems and general graph topologies—including undirected, directed weight-balanced, and directed weight-unbalanced graphs—by leveraging concepts from graph theory, optimization, and linear algebra.

By benchmarking the proposed algorithm against centralized approaches and validating it through simulations, this study demonstrates the practical relevance of distributed computation in modern technological landscapes. Applications of such algorithms are far-reaching, extending to fields such as data analysis, sensor fusion, control systems, robotics, and federated learning—any domain where efficiency, privacy, and resilience are paramount.

There have been many advancements in the methods of solving linear equations. There are many methods which involve the distributed algorithms to solve linear equations in different ways. But there are some assumptions which are made to proceed to solve the

linear equations which include consistency of linear equations as mentioned in [3]. The work [4] also contains a similar approach to solve linear equations in a distributed fashion. There also have been cases where there are constraints on different matrices which are used to solve linear equations. This includes cases where each node only produces a row of a matrix or column. In this project we will try to solve the linear equations for a generalized case and inconsistent cases.

We consider linear algebraic equations where the coefficient matrices and constant vector for the overall problem are given, respectively, by the summation of the individual agent's coefficient matrices and constant vectors. Our starting point here is to try to change the objective function in such a way that we will be able to reach least square solutions through the formulations. We will also be considering different algorithms to solve linear equations and try to distribute them.

1.1 Problem Formulation

Consider a network of n agents working together to solve a linear system of equations in a distributed way. The system is represented as:

$$\left(\sum_{i=1}^n A_i \right) x = \left(\sum_{i=1}^n b_i \right)$$

where each agent i only has access to its local data A_i and b_i and communicates with its neighboring out-degree agents to retrieve their values/data and also shares its own values/data to its in-degree neighbors. The goal for all agents is to collaboratively find the global solution x using only local computations and interactions. Each matrix A_i and b_i for all nodes have the same dimensions.

The approach starts by associating each agent with its own estimate of the solution, x_i and reformulates the problem as:

$$\left(\sum_{i=1}^n A_i x_i \right) = \left(\sum_{i=1}^n b_i \right)$$

$$x_i = x_j \quad \forall i, j.$$

The above equation is a very general case of solving linear equations as here in our case every single node contributes to the overall $Ax = b$ case. Whereas in most of the cases, we get either a row or a column of the A and b from every node and we combine those data to form the overall $Ax = b$. In the work [5] and [6], every single node contributes a row to the final $Ax = b$. Whereas on the other hand in the work [7] every agent had a row of matrix A and part of vector b .

We then write a convex optimization to solve this problem with our object function being $f(x)$ as:

$$f(\mathbf{x}) = \left(\sum_{i=1}^n (A_i x_i - b_i) \right)^\top \left(\sum_{i=1}^n (A_i x_i - b_i) \right)$$

We then bring in some standard notations to represent the optimization and then go on with it. The size of the matrix A_i is taken to be $p \times q$ where p is the number of

equations available in a matrix and q is the number of variables that we solve for in a node. The size of x_i is taken to be $q \times 1$ where q are the number of variables that we solve for in a node. We use $\mathbf{L} = L \otimes \mathbf{I}_q$ and

$$f(\mathbf{x}) = (\mathbf{Ax} - \mathbf{b})^\top \mathbb{1} \mathbb{1}^\top (\mathbf{Ax} - \mathbf{b}),$$

where $\mathbb{1} = \mathbf{1} \otimes \mathbf{I}$, ($\mathbf{1}$ is a constant vector of 1's throughout), $\mathbf{A} \in \mathbb{R}^{np \times nq}$ denotes the block-diagonal matrix obtained after putting the matrices $\{A_i\}_{i=1}^n$ along the principal diagonal, and

$$\mathbf{b} = [b_1, \dots, b_n] \in \mathbb{R}^{np}.$$

Similarly $\mathbf{x}$ is same as $\mathbf{b}$ coming out to be

$$\mathbf{x} = [x_1, \dots, x_n] \in \mathbb{R}^{nq}.$$

L is the Laplacian matrix of the graph structure we have. Laplacian L is defined as, $L = D - A$ where D is the degree matrix and A is the adjacency matrix of our graph network. There is again a conflict on what the degree should select for our degree matrix, whether is it in-degree or out-degree. In our case we focus primarily on in-degree. So For us the Laplacian turns out to be $L = D_{in} - A$. For example for the below graph structure our Laplacian turns out to be,

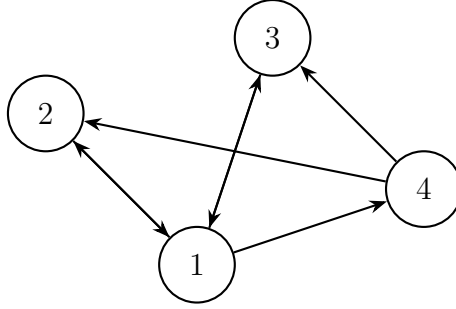


Figure 1.1: Example of a 4 Node graph

$$A = \begin{bmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 0 \end{bmatrix} \quad D_{in} = \begin{bmatrix} 2 & 0 & 0 & 0 \\ 0 & 2 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \quad L = \begin{bmatrix} 2 & -1 & -1 & -1 \\ -1 & 2 & 0 & 0 \\ -1 & 0 & 2 & 0 \\ 0 & -1 & -1 & 1 \end{bmatrix}$$

1.2 Literature Review

Existing Distributed Approaches

For consistent set of equations

The objective function along with the constraint of all x_i 's to be equal is reformulated already in the paper [3] as:

$$\min_{\mathbf{x}} f(\mathbf{x}) \quad \text{s.t.} \quad \mathbf{L}^\top \mathbf{x} = 0 \quad (1.1)$$

The problem (1.1) can be solved over an undirected graph by reformulating it using the techniques in [8] and employing the saddle-point dynamics, see [9], [10] This function is then re written as a minimization problem of the following expression:

$$\min_{\mathbf{x}} \quad \frac{1}{2}\alpha\mathbf{x}^\top(\mathbf{L} + \mathbf{L}^\top)\mathbf{x} + \beta f(\mathbf{x})$$

Minimization of the above function leads the above expression to go to zero. The second term goes to zero because in this case the problem statement is consistent. There always exists a solution where there is a consensus among all nodes due to consistency of the set of the problem statement. Hence we can say that the minimum occurs at zero as both the terms become 0 at the consensus solution. Hence minimization of the above expression removes constraints and gives us a optimization problem which is then solved using gradient descent and its dynamics are found and written below. Since the expression contains terms $\mathbb{1}^\top(\mathbf{Ax} - \mathbf{b})$ which involves sum of all the $\sum_{i=1}^n (A_i x_i - b_i)$ values throughout the system it is distributed by introducing a new variable $\mathbf{y}$ which measures the estimate of $\frac{1}{n}\mathbb{1}^\top(\mathbf{Ax} - \mathbf{b})$ using dynamic average consensus. The equation $L^\top$ also poses us a trouble in distribution, this is then handled by by writing $L + L^\top$ as αL . We get the dynamics of each variable to be,

$$\begin{aligned}\dot{\mathbf{x}} &= -\alpha\mathbf{Lx} - n\beta\mathbf{A}^\top\mathbf{y} \\ \dot{\mathbf{y}} &= -\alpha\mathbf{ALx} - n\beta\mathbf{AA}^\top\mathbf{y} - \gamma\mathbf{Ly}\end{aligned}$$

In the above equations the parameters α , β and γ are step parameters that are used for normalizing the equations. The value $n\beta\mathbf{AA}^\top\mathbf{y}$ might get larger in some cases and hence we need some step parameters to normalize and reduce its effect to prevent divergence. α and γ also has the same action. These parameters are also responsible to speed up our algorithm in some cases. Fine tuning of the parameters will give us good and faster convergence.

For inconsistent set of equations

Bringing back the same expression that was used earlier, we want the first term to go to zero because we want a consensus solution. But we can not be certain that the minimum of the expression occurs when the first term tends to 0 as there can be possibilities where the expression has a lower value at finite values of first and second term. If the first term does not become 0, it will violate the constraint and hence this function cannot be used.

In order to tackle the above problem one method of forcing the first term to be zero is to make the first term's coefficient α infinite. This gives us an solution which is close to the actual solution but we can never find the exact solution through this way.

Disadvantages of the above method:

1. We never get an exact solution for the above approach.
2. Our solution depends on the value of alpha that we take and hence is variable.
3. It might be a problem when the coefficients are matrices are huge.

Chapter 2

Solution in Undirected Networks

In this chapter we will be discussing about constrained optimization technique used to solve linear equations in a distributed way for Undirected networks (bidirectional graphs).

Approach using Dynamic average consensus

Centralized Approach

Least square formula for solving a linear equations having coefficient matrix A and constant vector b is given by:

$$x = (AA^\top)^{-1}A^\top b$$

The above formula can be directly used when we have values for all A matrices and b vector. But the above method can only be used for non-singular $AA^\top$ matrices

Distributed Approach using dynamic average consensus

Here we Apply dynamic average consensus over all the matrices A and b throughout the system so that we have the average value of all A_i and b_i available at every node. We then apply the least square formula to find the solution. The dynamic average consensus equations leads us to the results given by:

$$A'_j = \frac{1}{n} \sum_{i=1}^n A_i \quad \forall j$$
$$b'_j = \frac{1}{n} \sum_{i=1}^n b_i \quad \forall j$$

The works [11] and [12] contains more details about the average consensus used. The algorithm used for average consensus is given by:

$$\dot{x} = -Lx + \dot{z}.$$

where x is the state that is being averaged over all nodes and z is the change in input at the node that is related to x . This approach is similar to the approach in the work [13]

Disadvantages of the above algorithm:

1. Time taken for convergence is very high due to dynamic average consensus of every single element in matrix A_i .
2. The issue of scalability arises when the number of nodes increases which will slow the transfer and reduce efficiency
3. The above formulations will work only for cases where $AA^\top$ is non singular and hence will not be able to solve all kinds of inconsistent equations.

Using Constrained Optimization

In this section we will be using constrained optimization in order to include the constraint of all x_i being equal into our equation and then we do optimization using Lagrangian to find the final solution. This method involves the equations which have been used in section and also notations used earlier. The optimization that we are willing to perform is given by,

$$f(\mathbf{x}) = (\mathbf{Ax} - \mathbf{b})^\top \mathbb{1} \mathbb{1}^\top (\mathbf{Ax} - \mathbf{b})$$

under the constraint of,

$$\mathbf{L}^\top \mathbf{x} = 0.$$

This can be combined altogether by the use of Lagrangian function with Lagrangian multipliers. [14] tells us the usage of Lagrangian along with the necessary matrix conditions that are supposed to be employed. The Lagrangian function is written as,

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = (\mathbf{Ax} - \mathbf{b})^\top \mathbb{1} \mathbb{1}^\top (\mathbf{Ax} - \mathbf{b}) + \boldsymbol{\lambda}^\top \mathbf{L}^\top \mathbf{x}$$

$\boldsymbol{\lambda}$ is a vector with dimensions $\lambda \in \mathbb{R}^{nq \times 1}$.

[15] tells us about the saddle point dynamics of the Lagrangian equation. The saddle points of $\mathbf{x}$ and $\boldsymbol{\lambda}$ are written as,

$$\dot{\mathbf{x}} = -\nabla_{\mathbf{x}} \mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) \quad \text{and} \quad \dot{\boldsymbol{\lambda}} = \nabla_{\boldsymbol{\lambda}} \mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}),$$

Since the dynamics of $\mathbf{x}$ contains the term $\mathbb{1}^\top (\mathbf{Ax} - \mathbf{b})$ which has the sum of $\sum_{i=1}^n (A_i x_i - b_i)$ over all the matrix used for every $\mathbf{x}$ we bring in another variable $\mathbf{y}$ which is responsible to estimate $\frac{1}{n} \mathbb{1}^\top (\mathbf{Ax} - \mathbf{b})$ throughout the system at any time using dynamic average consensus. The equation for dynamic average consensus is given by

$$\dot{\mathbf{x}} = -\mathbf{L}\mathbf{x} + \dot{\mathbf{z}}.$$

Hence the dynamics of $\mathbf{x}$ is written as,

$$\dot{\mathbf{x}} = -\alpha \mathbf{L}\boldsymbol{\lambda} - n\beta \mathbf{A}^\top \mathbf{y}.$$

The dynamics of $\mathbf{y}$ is hence written as,

$$\dot{\mathbf{y}} = -\alpha \mathbf{A}\mathbf{L}\boldsymbol{\lambda} - n\beta \mathbf{A}\mathbf{A}^\top \mathbf{y} - \gamma \bar{\mathbf{L}}\mathbf{y}.$$

In the above equation $\bar{\mathbf{L}}$ is written similar to $\mathbf{L}$ as $\bar{\mathbf{L}} = L \otimes \mathbf{I}_p$. $\mathbf{L}$ has the expression $\mathbf{L} = L \otimes \mathbf{I}_q$. We differentiate both here because of the problem with dimension between $\mathbf{x}$ and $\mathbf{y}$. The dynamics of $\boldsymbol{\lambda}$ is written as,

$$\dot{\lambda} = \mathbf{L}^\top \mathbf{x}.$$

where α, β, γ are step size parameters.

By observation we can see that every term in dynamics of $\mathbf{x}$ and $\mathbf{y}$ are distributed, whereas the term $\mathbf{L}^\top \mathbf{x}$ in λ contains $\mathbf{L}^\top$ which creates problem in distributing it because $\mathbf{L}^\top$ require in-degree neighbor's information to perform data in a directed graph. Hence, assuming our graph network to be undirected makes $\mathbf{L}^\top = \mathbf{L}$ which then poses no problem while distributing the equation. Hence we assume $\mathbf{L}^\top = \mathbf{L}$ from now on in all equations related to undirected graph. Hence we then can use the dynamics which we got to solve linear equations distributively in an undirected graph.

Chapter 3

Observations for Simulations in Undirected network

This chapter mainly focusses on the results that have been observed after implementing the above algorithm in the code above. Different graph networks have been used for different cases and the observations have been included in them.

Observations for a ring Graph

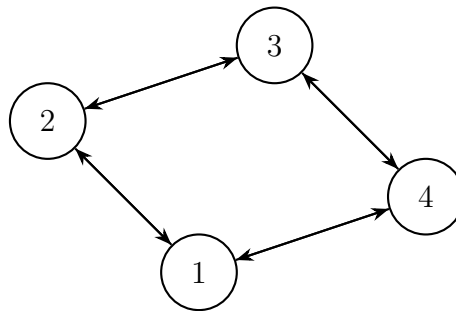


Figure 3.1: Ring graph

The above graph is an example that has been used in the simulation. The values of matrix A and b that has been used is given in the code part of the last chapter. For clarity the matrix value's sum is observed as:

$$\text{Sum}(A_i) = \begin{bmatrix} 7 & 4 & 8 & 5 \\ 7 & 3 & 7 & 8 \\ 5 & 4 & 8 & 8 \\ 3 & 6 & 5 & 2 \\ 8 & 6 & 2 & 5 \end{bmatrix} \quad \text{Sum}(b_i) = \begin{bmatrix} 113 \\ 96 \\ 109 \\ 86 \\ 67 \end{bmatrix}$$

The solution of the above sum and the convergence of the node values are equal as expected.

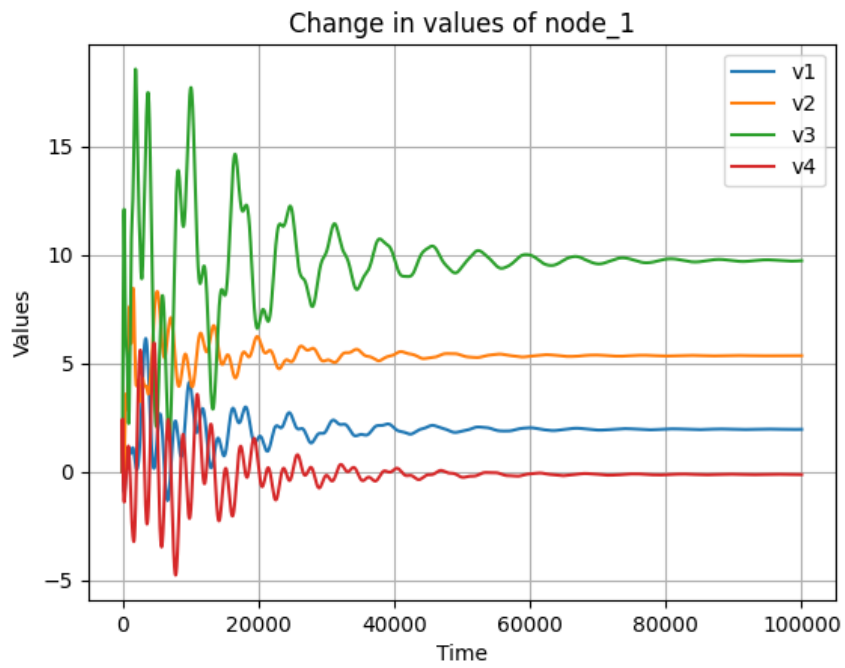


Figure 3.2: Convergence of x in Node 1 in Ring graph

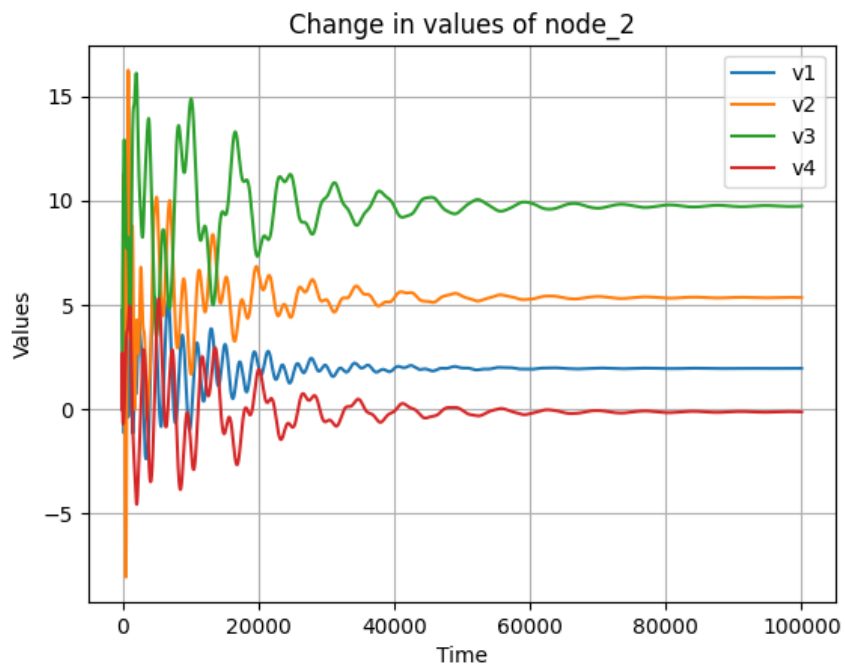


Figure 3.3: Convergence of x in Node 2 in Ring graph

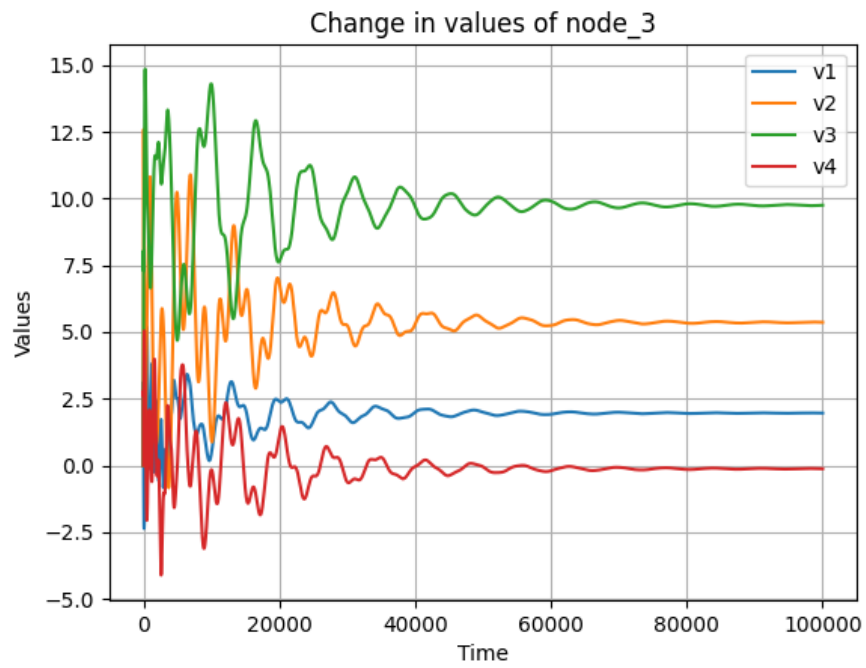


Figure 3.4: Convergence of x in Node 3 in Ring graph

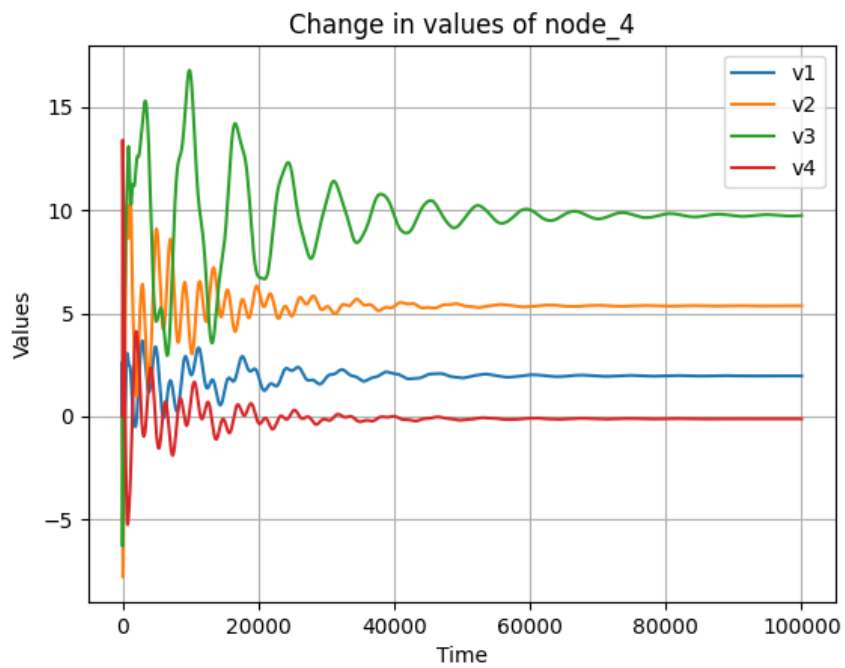


Figure 3.5: Convergence of x in Node 4 in Ring graph

Observations for a dense (Fully-connected) Graph

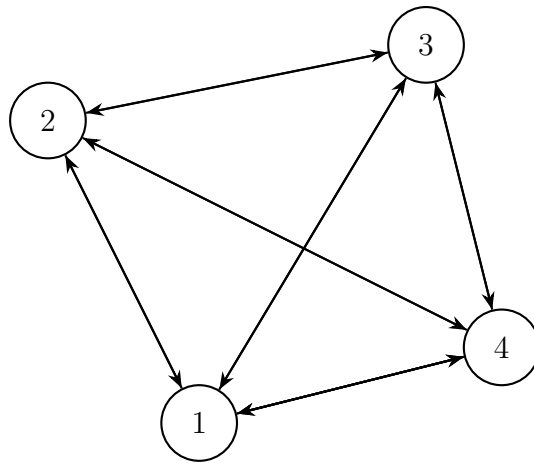


Figure 3.6: Dense graph

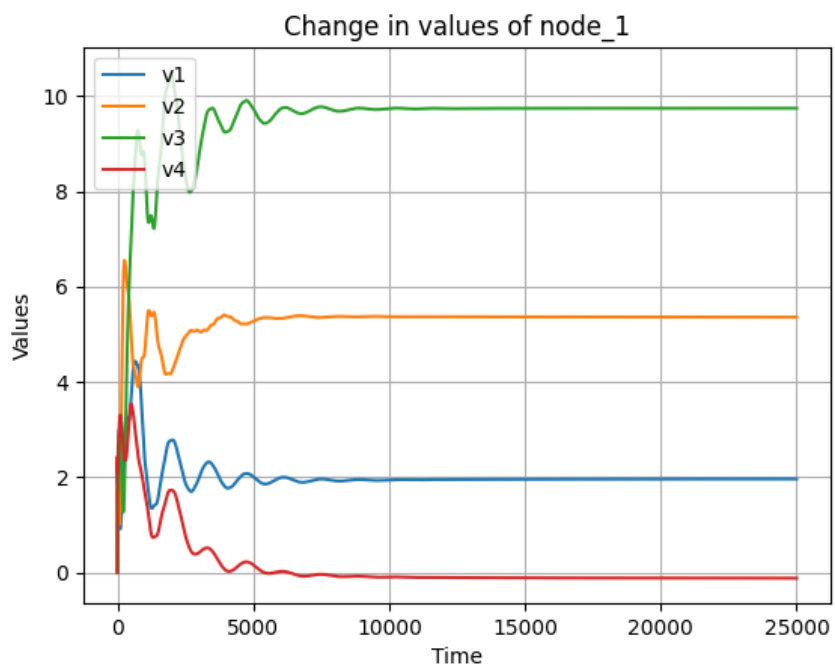


Figure 3.7: Convergence of x in Node 1 in dense graph

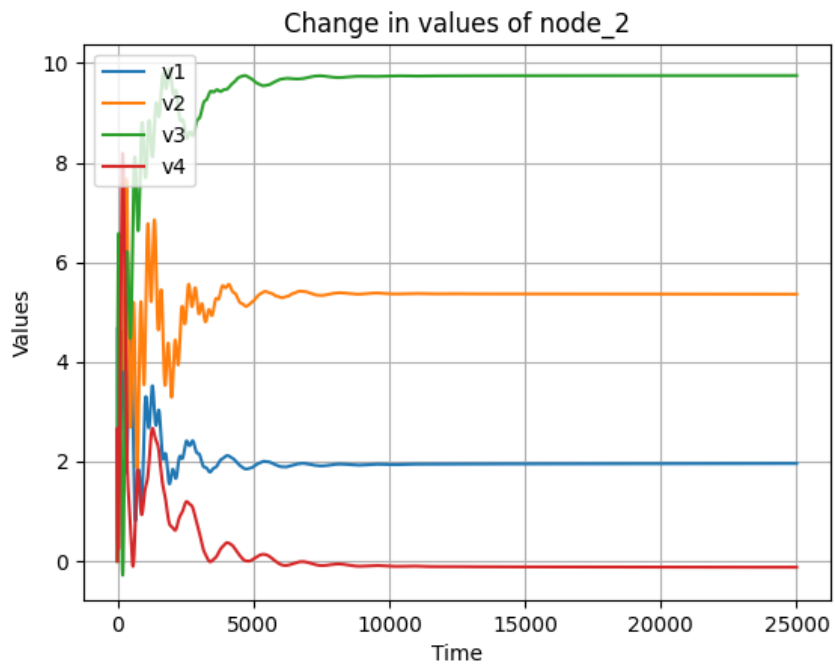


Figure 3.8: Convergence of x in Node 2 in dense graph

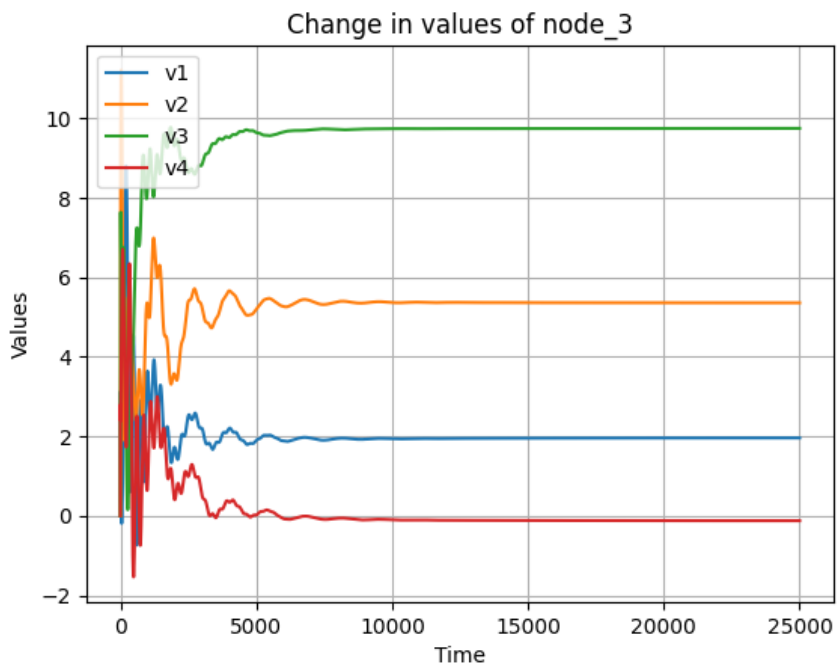


Figure 3.9: Convergence of x in Node 3 in dense graph

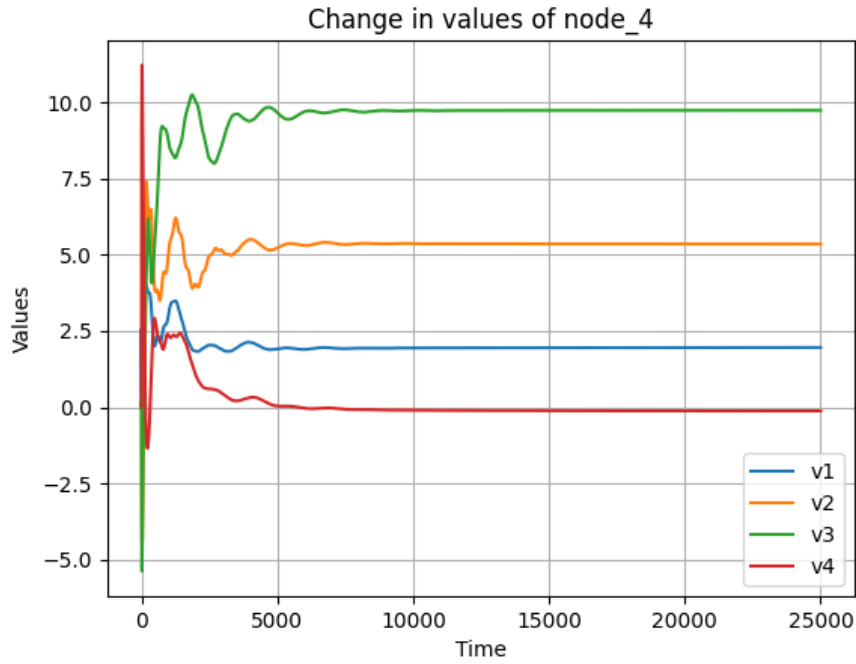


Figure 3.10: Convergence of x in Node 4 in dense graph

Inferences from the simulations

We observe that our values converge very fast in the case of Dense network due to high communication and data transfer between every nodes. On the other hand, the convergence is slow in the case ring graph which is less dense case. We also observe slow and consistent oscillations in ring network due to poor connectivity.

But this algorithm does not work with directed graphs. The next chapter discusses the algorithm for weight balanced directed networks.

Chapter 4

Algorithm for Directed Weight-balanced Networks

In the fore coming chapter we will be introducing the centralized algorithm first for solving inconsistent set of equations in weight-balanced networks and then we proceed on to distribute the centralized algorithm to get the desired results. A weight-balanced network is a type of directed graph where each node has indegree equal to the outdegree, meaning each node sends and receives the same total amount of "weight" or "influence".

4.1 Centralized Algorithm

We first formulate the centralized algorithm using a new Lagrangian rather than the old one as shown below, this Lagrangian has been taken from [16]

$$\mathcal{L}(\mathbf{x}, \boldsymbol{\lambda}) = f(\mathbf{x}) + \mathbf{x}^T \mathbf{L} \boldsymbol{\lambda} + \frac{1}{2} \mathbf{x}^T \mathbf{L} \mathbf{x} \quad (4.1)$$

where $f(\mathbf{x})$ is our old objective function used in the previous chapters given as,

$$f(\mathbf{x}) = (\mathbf{A}\mathbf{x} - \mathbf{b})^T \mathbb{1} \mathbb{1}^T (\mathbf{A}\mathbf{x} - \mathbf{b}) \quad (4.2)$$

which is a measure of the squared error loss for a given $\mathbf{x}$ over our given network. The equation (4.1) is our constrained Lagrangian also called as augmented Lagrangian not the same as our normal Lagrangian. $\boldsymbol{\lambda}$ is again another column vector similar to $\mathbf{b}$. We then use gradient descent to find the optimizers of (4.1). But we observe that the gradient descent of our Lagrangian comes out to be, $\dot{\mathbf{x}} = -2\mathbf{A}^T \mathbb{1} \mathbb{1}^T (\mathbf{A}\mathbf{x} - \mathbf{b}) - \frac{1}{2}(\mathbf{L}^T + \mathbf{L})\mathbf{x} - \mathbf{L}\boldsymbol{\lambda}$, which contains the term $\mathbf{L}^T$ which is not distributed because it requires information from the in-neighbors in our graph. We found that this equation can be replaced as the equation shown below from the [16],

$$\dot{\mathbf{x}} = -2\mathbf{A}^T \mathbb{1} \mathbb{1}^T (\mathbf{A}\mathbf{x} - \mathbf{b}) - \alpha \mathbf{L} \mathbf{x} - \mathbf{L} \boldsymbol{\lambda} \quad (4.3a)$$

$$\dot{\boldsymbol{\lambda}} = \mathbf{L} \mathbf{x} \quad (4.3b)$$

The proof for the above equation can be directly observed in [16]. From (4.3) we see that all terms of the equation can be distributed except for the term $-2\mathbf{A}^T \mathbb{1} \mathbb{1}^T (\mathbf{A}\mathbf{x} - \mathbf{b})$ which requires summation over the whole network because of the term $\mathbb{1}^T (\mathbf{A}\mathbf{x} - \mathbf{b})$. For this purpose we hence introduce a new variable $\mathbf{y}$ to distribute the term.

4.2 Distributed Algorithm

As we discussed in the previous section the term $-2\mathbf{A}^T \mathbb{1} \mathbb{1}^T (\mathbf{Ax} - \mathbf{b})$ is not distributed. We hence introduce a variable $\mathbf{y}$ which is a dynamic average consensus of measure of error in the whole network, in other words it estimates the error $\frac{1}{n} \mathbb{1}^T (\mathbf{Ax} - \mathbf{b})$. We hereby introduce the new algorithm which is based on (4.3). We then define the new saddle point dynamics to be,

$$\dot{\mathbf{x}} = -2n\beta \mathbf{A}^T \mathbf{y} - \alpha \mathbf{Lx} - \mathbf{L}\lambda \quad (4.4a)$$

$$\dot{\mathbf{y}} = -\bar{\mathbf{L}}\mathbf{y} - 2\eta\beta \mathbf{AA}^T \mathbf{y} - \alpha \mathbf{ALx} - \mathbf{AL}\lambda \quad (4.4b)$$

$$\dot{\lambda} = \mathbf{Lx} \quad (4.4c)$$

The design parameter β is used with condition $\beta > 0$. Here in the above equation the dimensions of the variable $\mathbf{y}$ is $y_i \in \mathbb{R}^p$.

The above algorithm is only applicable for weight-balanced networks as the above algorithm requires equal data exchange in order to maintain equilibrium. The algorithm that can be used for weight-unbalanced networks will be introduced in the later chapters.

Important note

Another important step in the above lies in the initiation of the variables for consensus. From [3] we can see that the variable x has no condition on initiation. The variable λ is a Lagrangian multiplier, hence it can also have any initiation. On the other hand variable $\mathbf{y}$ is responsible to track the sum from start, hence it should have a proper initiation of $\mathbf{y} = \mathbf{Ax} - \mathbf{b}$. In most cases we will have an initiation of $\mathbf{x} = 0$, hence we can initialize $\mathbf{y} = -\mathbf{b}$ for all cases with $\mathbf{x} = 0$.

For the variables α and β , it is very important to note that these are step parameters which are responsible to normalise the terms in our expression. Hence, it is important for these variables to be tuned accordingly. The values that we have used for β lies between $0.05 - 0.005$. Whereas α has values between $5 - 100$. These values can be increased even further to observe better convergence, but too high values can lead to divergence and hence will disrupt the equilibrium of the solutions.

Chapter 5

Observations for Simulations in Directed Weight-balanced network

This chapter includes all observations for the Directed Weight-balanced networks where there is no two way transfer of data.

5.1 Observation for a 4-Node graph

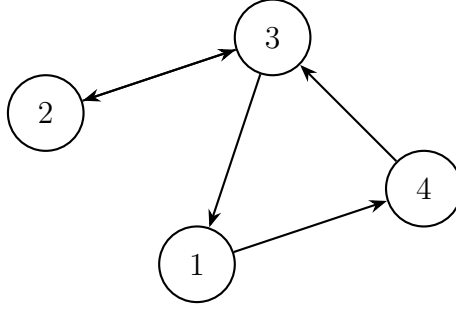


Figure 5.1: 4 Node graph used for implementation

The above graph is an example that has been used in the simulation. We observe that the indegree and outdegree of all the nodes in the graph are equal. The values of matrix A and b that has been used is given below:

$$A_1 = \begin{bmatrix} 5 & 0 & 2 & 3 \\ 5 & 2 & 2 & 6 \\ 3 & 4 & 5 & 8 \\ 0 & 5 & 4 & 2 \\ 8 & 2 & 2 & 0 \end{bmatrix} \quad A_2 = \begin{bmatrix} 6 & 1 & 6 & 4 \\ 4 & 0 & 4 & 2 \\ 1 & 0 & 3 & 6 \\ 0 & 0 & 0 & 1 \\ 4 & 1 & 2 & 3 \end{bmatrix} \quad A_3 = \begin{bmatrix} 2 & 1 & 0 & 3 \\ 6 & 2 & 3 & 5 \\ 1 & 4 & 6 & 0 \\ 2 & 3 & 2 & 2 \\ 0 & 3 & 1 & 0 \end{bmatrix} \quad A_4 = \begin{bmatrix} -6 & 2 & 0 & -5 \\ -8 & -1 & -2 & -5 \\ 0 & -4 & -6 & -6 \\ 1 & -2 & -1 & -3 \\ -4 & 0 & -3 & 2 \end{bmatrix}$$
$$b_1 = \begin{bmatrix} 61 \\ 51 \\ 1 \\ 55 \\ 1 \end{bmatrix} \quad b_2 = \begin{bmatrix} 112 \\ 11 \\ 2 \\ 80 \\ 1 \end{bmatrix} \quad b_3 = \begin{bmatrix} 57 \\ 38 \\ 100 \\ 58 \\ 53 \end{bmatrix} \quad b_4 = \begin{bmatrix} -117 \\ -4 \\ 6 \\ -107 \\ 12 \end{bmatrix}$$

$$\text{Sum}(A_i) = \begin{bmatrix} 7 & 4 & 8 & 5 \\ 7 & 3 & 7 & 8 \\ 5 & 4 & 8 & 8 \\ 3 & 6 & 5 & 2 \\ 8 & 6 & 2 & 5 \end{bmatrix} \quad \text{Sum}(b_i) = \begin{bmatrix} 113 \\ 96 \\ 109 \\ 86 \\ 67 \end{bmatrix}$$

The solution of the above sum is expected to be:

$$\text{Solution vector} = [1.96203376 \quad 5.35884552 \quad 9.74942138 \quad -0.12373404]$$

Actual solutions in each node

Node 1 has values: $[1.96188549 \quad 5.35894742 \quad 9.74937256 \quad -0.12358671]$

Node 2 has values: $[1.9618821 \quad 5.35893897 \quad 9.74936284 \quad -0.12358939]$

Node 3 has values: $[1.96188831 \quad 5.35894242 \quad 9.74936806 \quad -0.12358817]$

Node 4 has values: $[1.96189361 \quad 5.35894683 \quad 9.74937324 \quad -0.12358554]$

Evolution of Solution in each Node

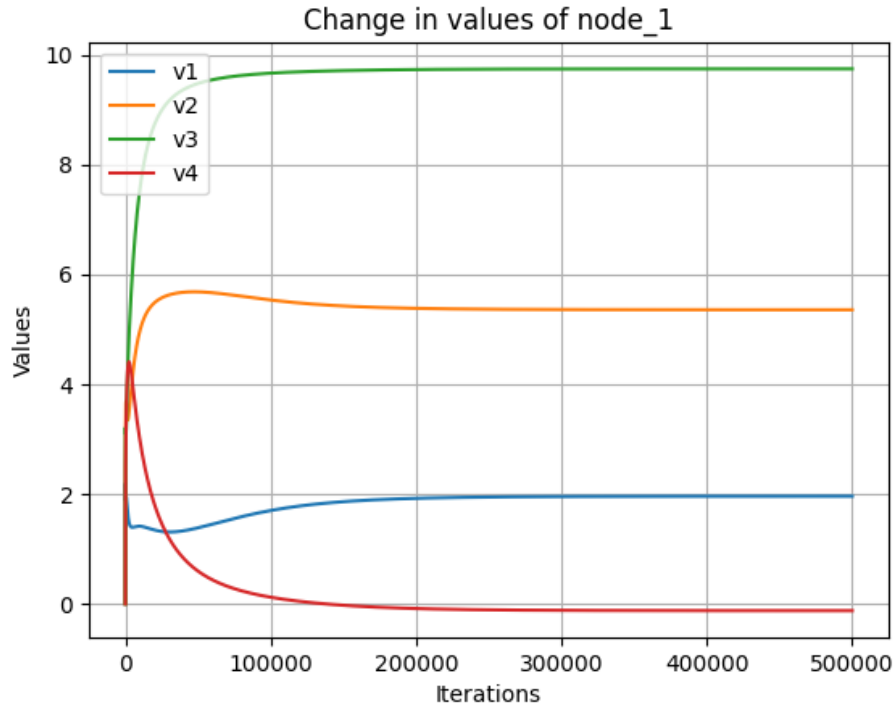


Figure 5.2: Convergence of x in Node 1 in the above graph network

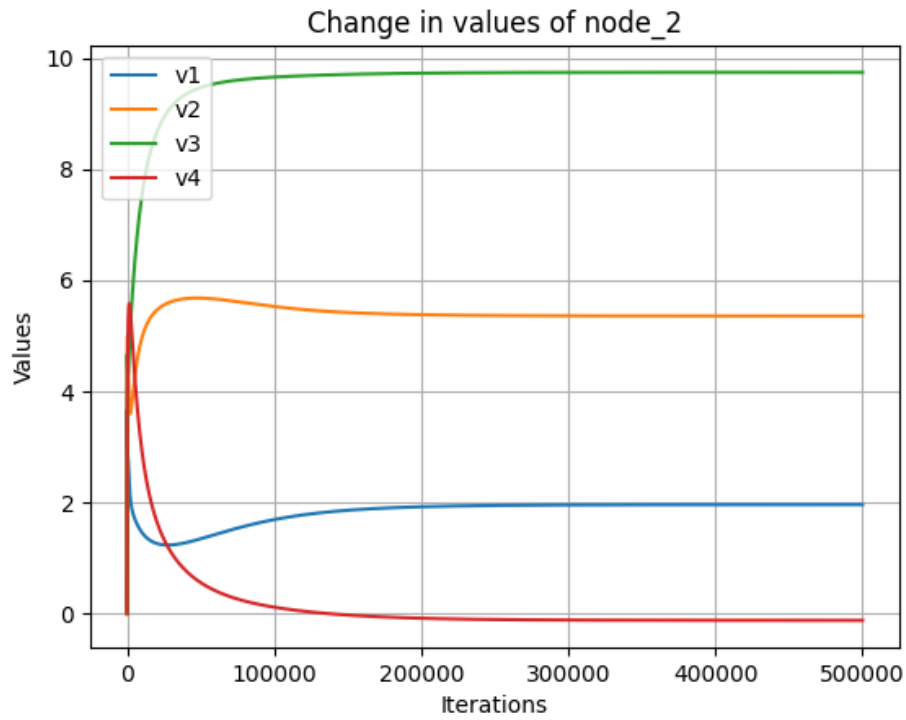


Figure 5.3: Convergence of x in Node 2 in the above graph network

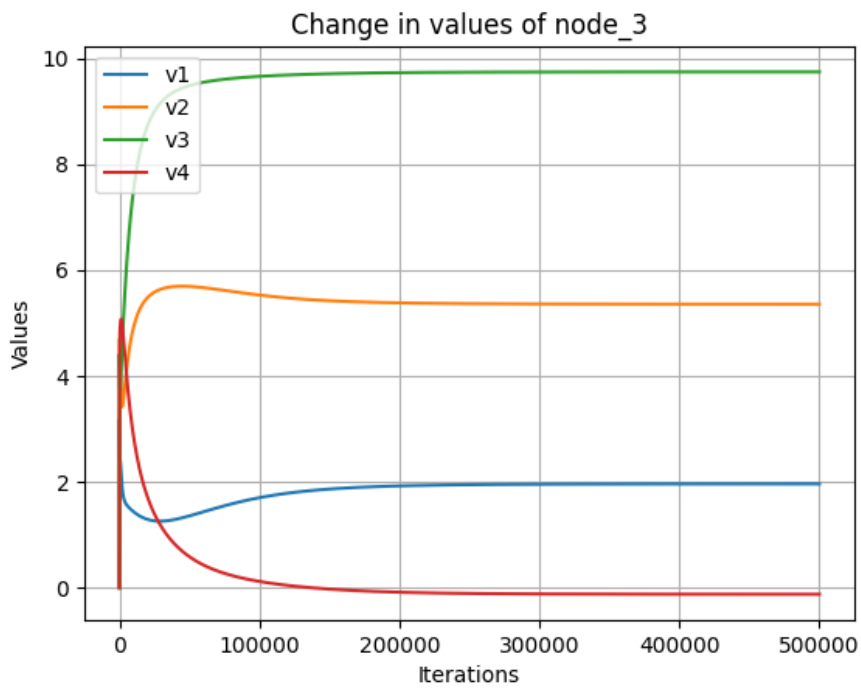


Figure 5.4: Convergence of x in Node 3 in the above graph network

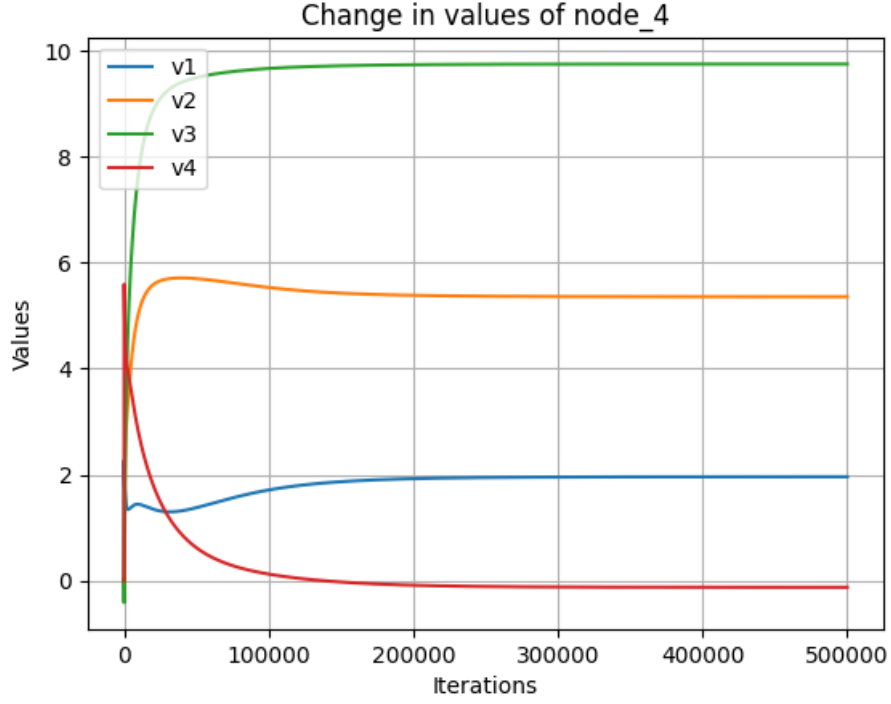


Figure 5.5: Convergence of x in Node 4 in the above graph network

5.2 Observation for a 6-Node graph

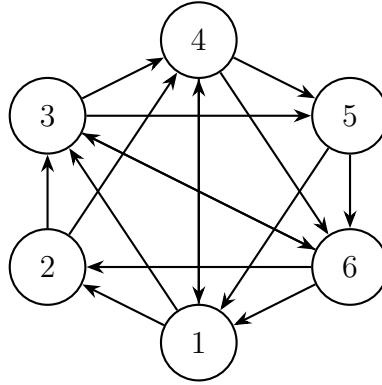


Figure 5.6: 6-Node graph used for implementation

The above graph is an example that has been used in the simulation. We observe that the indegree and outdegree of all the nodes in the graph are equal. The values of matrix A and b that has been used is given below:

$$A_1 = \begin{bmatrix} 0 & -5 & 2 & 3 & 1 \\ 1 & -4 & 3 & -5 & 4 \\ -2 & -1 & 1 & -1 & 3 \\ -4 & -3 & 0 & 2 & 4 \\ -2 & -2 & 5 & -1 & 5 \\ -4 & 1 & -1 & -4 & -4 \\ 1 & -2 & 5 & -4 & 3 \end{bmatrix} \quad A_2 = \begin{bmatrix} 5 & 4 & -3 & -5 & -3 \\ 1 & -1 & -1 & -4 & 1 \\ -1 & -4 & 4 & -1 & 2 \\ -5 & 5 & -4 & 3 & -1 \\ 5 & 0 & -1 & -3 & -5 \\ 3 & 0 & 4 & -2 & -4 \\ -3 & -4 & 5 & 3 & -2 \end{bmatrix}$$

$$A_3 = \begin{bmatrix} 1 & 4 & 1 & 0 & -1 \\ -4 & -2 & 5 & -2 & 5 \\ 3 & 4 & 5 & 3 & -5 \\ 0 & 2 & -2 & 3 & 1 \\ -5 & -1 & -3 & 5 & -1 \\ -5 & -5 & 5 & 3 & -5 \\ -1 & 0 & 4 & 0 & 0 \end{bmatrix} \quad A_4 = \begin{bmatrix} -1 & 4 & 0 & 5 & -5 \\ 4 & -4 & -3 & -1 & -2 \\ -5 & -4 & 0 & -4 & -1 \\ 1 & 4 & 5 & 1 & 4 \\ 3 & -5 & -3 & -1 & 5 \\ -4 & -4 & 1 & -1 & 0 \\ -3 & -2 & 3 & 3 & -5 \end{bmatrix}$$

$$A_5 = \begin{bmatrix} 5 & 4 & 4 & 4 & -5 \\ -1 & -3 & 0 & 3 & 5 \\ 3 & -4 & -3 & 5 & 1 \\ 4 & -4 & -2 & -2 & 3 \\ 3 & 0 & 5 & 1 & 4 \\ 4 & -5 & 4 & 1 & 0 \\ -4 & -5 & 2 & 3 & -4 \end{bmatrix} \quad A_6 = \begin{bmatrix} -9 & -4 & 1 & -1 & 17 \\ 6 & 19 & 0 & 15 & -12 \\ 4 & 13 & 0 & 0 & 1 \\ 8 & 1 & 12 & -6 & -8 \\ 1 & 9 & 4 & 1 & -5 \\ 7 & 14 & -12 & 4 & 14 \\ 13 & 20 & -16 & 2 & 11 \end{bmatrix}$$

$$b_1 = \begin{bmatrix} 1 \\ 5 \\ 8 \\ 2 \\ -2 \\ 10 \\ 12 \end{bmatrix} \quad b_2 = \begin{bmatrix} -2 \\ -5 \\ -5 \\ 5 \\ 6 \\ 10 \\ 8 \end{bmatrix} \quad b_3 = \begin{bmatrix} 0 \\ 10 \\ 8 \\ 12 \\ 1 \\ -1 \\ -4 \end{bmatrix} \quad b_4 = \begin{bmatrix} 13 \\ 7 \\ 9 \\ 11 \\ 6 \\ 6 \\ -5 \end{bmatrix} \quad b_5 = \begin{bmatrix} 6 \\ -2 \\ 11 \\ 7 \\ -3 \\ 0 \\ -4 \end{bmatrix} \quad b_6 = \begin{bmatrix} -8 \\ -8 \\ -25 \\ -23 \\ -18 \\ -10 \\ 1 \end{bmatrix}$$

$$\text{Sum}(A_i) = \begin{bmatrix} 1 & 7 & 5 & 6 & 4 \\ 7 & 5 & 4 & 6 & 1 \\ 2 & 4 & 7 & 2 & 1 \\ 4 & 5 & 9 & 1 & 3 \\ 5 & 1 & 7 & 2 & 3 \\ 1 & 1 & 1 & 1 & 1 \\ 3 & 7 & 3 & 7 & 3 \end{bmatrix} \quad \text{Sum}(b_i) = \begin{bmatrix} 10 \\ 7 \\ 6 \\ 14 \\ -10 \\ 15 \\ 8 \end{bmatrix}$$

The solution of the above sum are expected to be:

$$\text{Solution vector} = [1.63167378 \quad 6.25549702 \quad -2.24321408 \quad -4.75325415 \quad 0.8661762]$$

Actual Solution

Node 1 has values: [1.6313958 6.25513795 -2.24289855 -4.75278324 0.86580541]
Node 2 has values: [1.63139586 6.25513785 -2.24289846 -4.75278335 0.8658055]
Node 3 has values: [1.63139582 6.25513799 -2.24289847 -4.75278326 0.8658054]
Node 4 has values: [1.63139582 6.25513783 -2.24289855 -4.75278337 0.86580542]
Node 5 has values: [1.63139587 6.25513797 -2.24289853 -4.75278318 0.86580537]
Node 6 has values: [1.63139573 6.25513801 -2.24289855 -4.75278321 0.86580541]

Evolution of Solutions in each Node

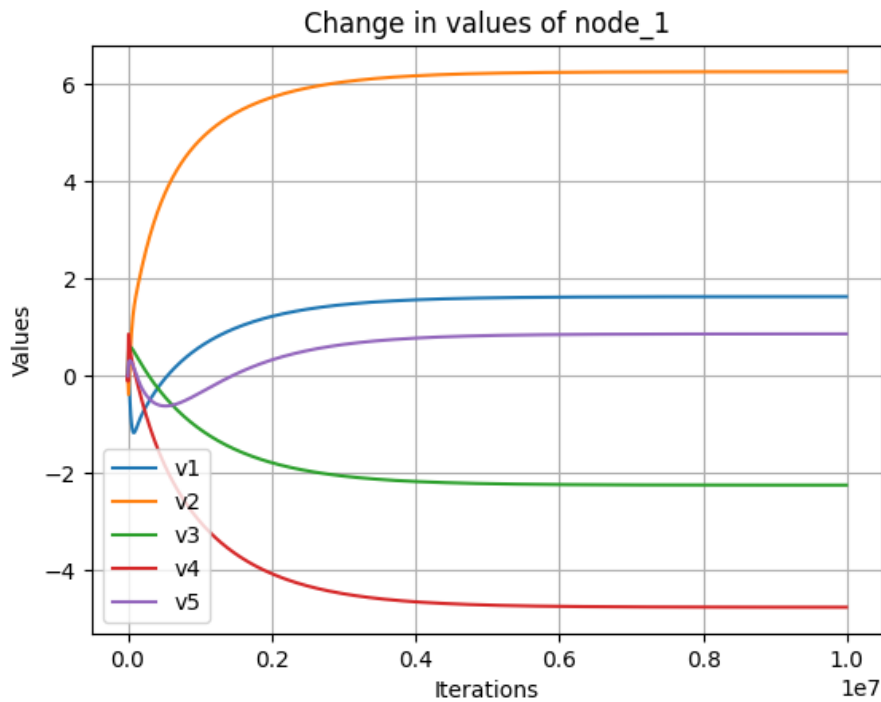


Figure 5.7: Convergence of x in Node 1 in the above graph network

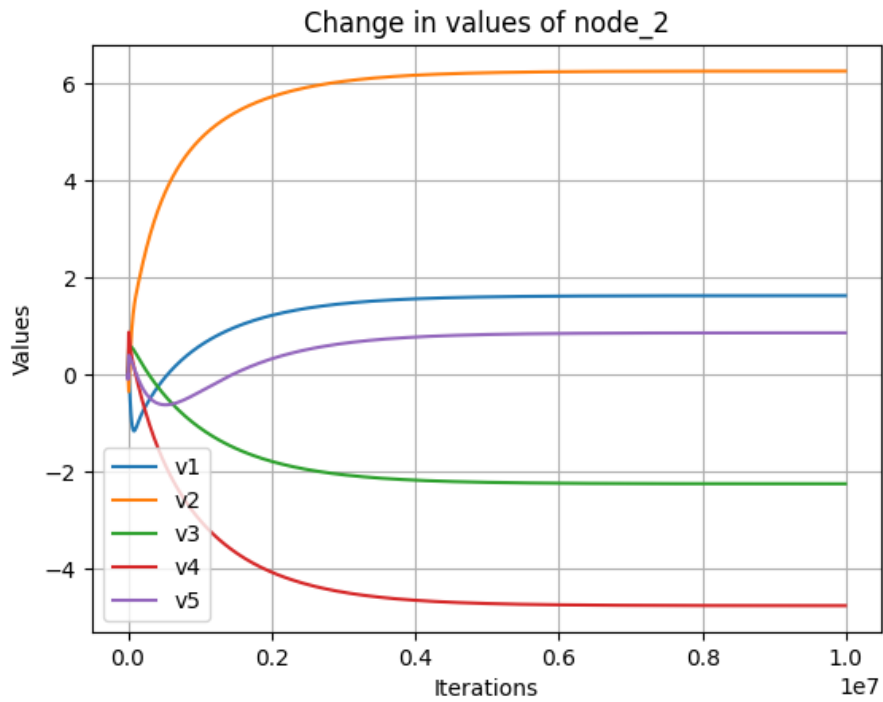


Figure 5.8: Convergence of x in Node 2 in the above graph network

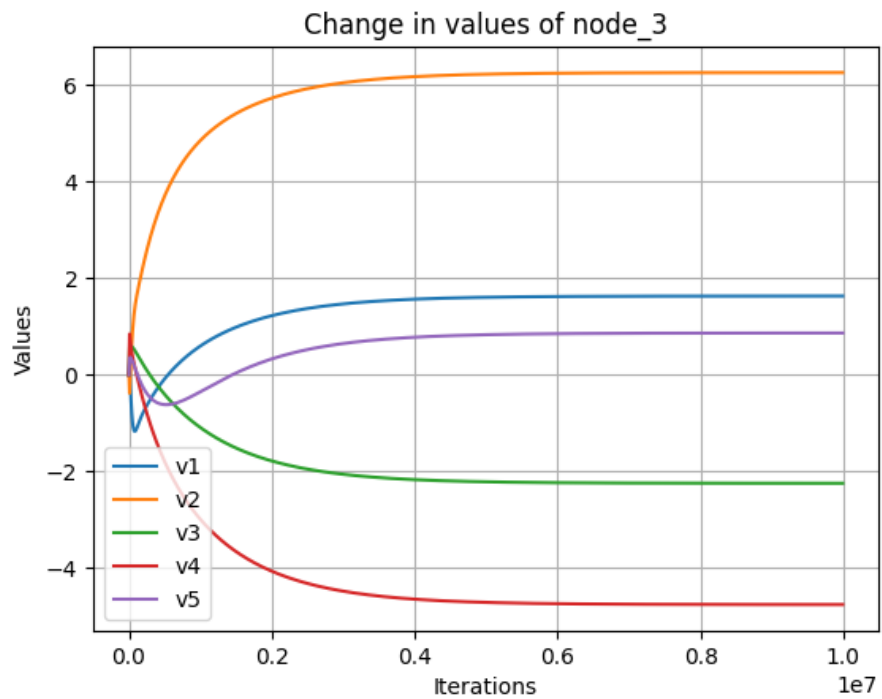


Figure 5.9: Convergence of x in Node 3 in the above graph network

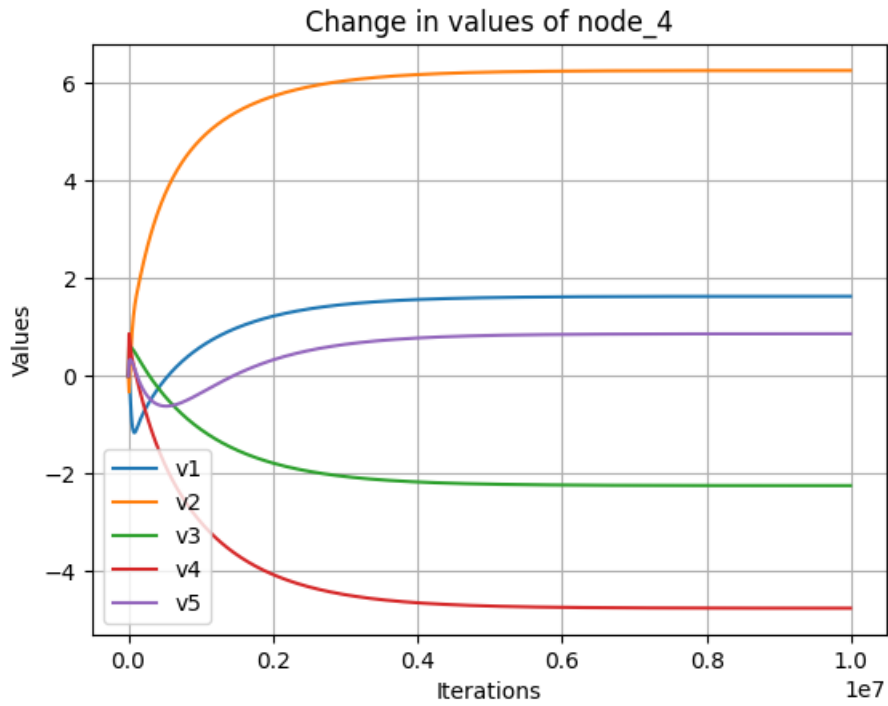


Figure 5.10: Convergence of x in Node 4 in the above graph network

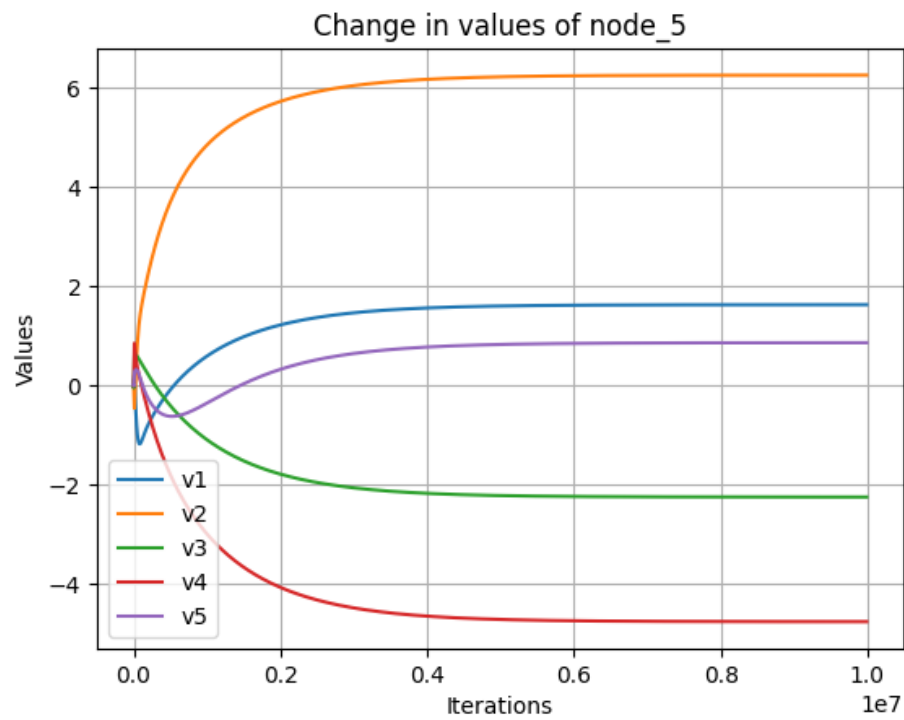


Figure 5.11: Convergence of x in Node 5 in the above graph network

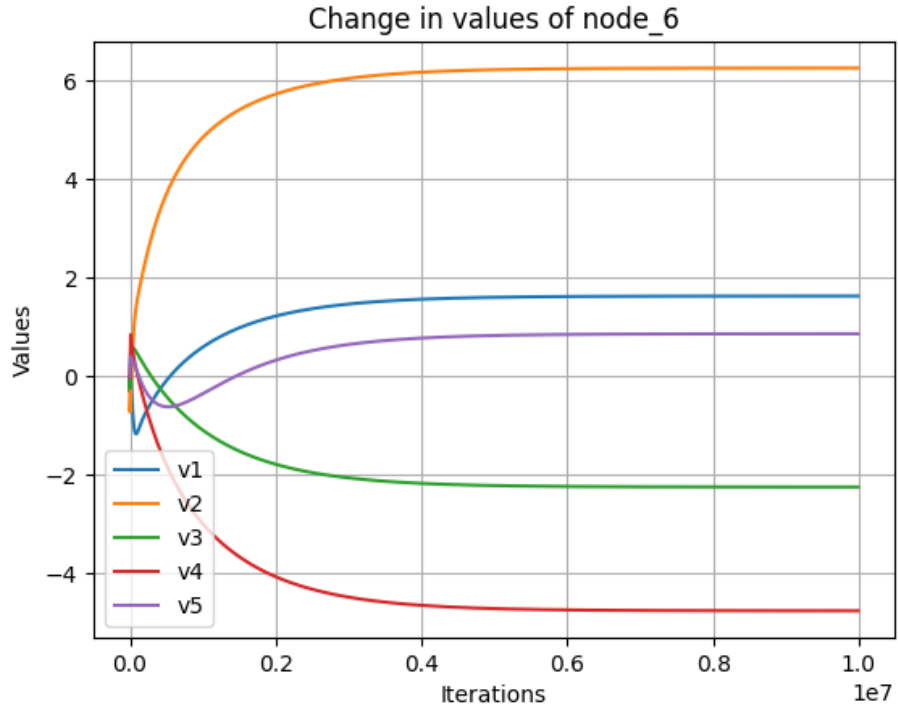


Figure 5.12: Convergence of x in Node 6 in the above graph network

5.3 Inference

From the above observations we can clearly infer that the time taken for convergence in directed graphs are very high due to limited data transfer. Hence it takes huge amount of time for our equations to converge to least square solutions.

Chapter 6

Algorithm for Directed Weight-Unbalanced Networks

In the upcoming chapter we will be utilizing the result which we got from unbalanced case with modifications of the Laplacian matrix to make it work for unbalanced case. (4.3) does not work in unbalanced case because $\mathbf{L}\mathbf{1} \neq \mathbf{0}$, so the term $\mathbf{L}\mathbf{x} \neq \mathbf{0}$ is not the condition for consensus anymore.

6.1 Algorithm for globally known Graph network

To overcome the above mentioned problem we introduce $\bar{\mathbf{V}}$ and $\bar{\mathbf{W}}$ which will be multiplied with the respective Laplacian matrices. This is the similar change that have been made in [3] in order to convert directed-weight balanced graphs to directed weight-unbalanced. This change will account for the imbalance in our graph network.

$$\dot{\mathbf{x}} = -2n\beta\mathbf{A}^T\mathbf{y} - \alpha\mathbf{L}\bar{\mathbf{V}}\mathbf{x} - \mathbf{L}\bar{\mathbf{V}}\lambda \quad (6.1a)$$

$$\dot{\mathbf{y}} = -\bar{\mathbf{L}}\bar{\mathbf{W}}\mathbf{y} - 2\eta\beta\mathbf{A}\mathbf{A}^T\mathbf{y} - \alpha\mathbf{A}\mathbf{L}\bar{\mathbf{V}}\mathbf{x} - \mathbf{A}\mathbf{L}\bar{\mathbf{V}}\lambda \quad (6.1b)$$

$$\dot{\lambda} = \mathbf{L}\bar{\mathbf{V}}\mathbf{x} \quad (6.1c)$$

where $\bar{\mathbf{V}} = \text{diag}(\bar{\mathbf{v}})$, $\bar{\mathbf{W}} = \text{diag}(\bar{\mathbf{w}})$, $\bar{\mathbf{v}} = \mathbf{1}_q \otimes \bar{v}$, $\bar{\mathbf{w}} = \mathbf{1}_p \otimes \bar{v}$, and $\bar{v}$ is a positive right eigen vector with eigen value of 0 of $\mathbf{L}$. The variables have the dimensions as, $\bar{v} \in \mathbb{R}^n$, $\bar{\mathbf{v}} \in \mathbb{R}^{nq}$, $\bar{\mathbf{w}} \in \mathbb{R}^{np}$, $\bar{\mathbf{V}} \in \mathbb{R}^{nq \times nq}$ and $\bar{\mathbf{W}} \in \mathbb{R}^{np \times np}$. The term $\bar{\mathbf{V}}\mathbf{L}\mathbf{x} = \mathbf{0}$ denotes consensus. This can be interpreted from the work [3]. The reason for introduction of two variables $\bar{\mathbf{V}}$ and $\bar{\mathbf{W}}$ is to match the dimensions of the variables. Here $\bar{\mathbf{V}}$ matches the dimensions of $\mathbf{x}$ and $\bar{\mathbf{W}}$ matches the dimensions of $\mathbf{y}$. Hence $\bar{\mathbf{W}}$ has no significance without the variable $\mathbf{y}$.

6.2 Algorithm for unknown Graph network

The above mentioned algorithm only works for the cases where the whole graph structure is known to every agent. This knowledge of the whole graph structure allows every node to find the eigen vector of the Laplacian locally and run the previous dynamics. But in scenarios where the whole graph network is not known, we will have to estimate this eigen value as a separate variable the same way we do for the other variables. So now we

introduce two new variables $\mathbf{v}$ and $\mathbf{w}$. This method is again adopted from [3]. The overall dynamics after addition of the new variables can be written as,

$$\dot{\mathbf{x}} = -2n\beta\mathbf{A}^T\mathbf{y} - \alpha\mathbf{L}\mathbf{V}\mathbf{x} - \mathbf{L}\mathbf{V}\lambda \quad (6.2a)$$

$$\dot{\mathbf{y}} = -\bar{\mathbf{L}}\mathbf{W}\mathbf{y} - 2\eta\beta\mathbf{A}\mathbf{A}^T\mathbf{y} - \alpha\mathbf{A}\mathbf{L}\mathbf{V}\mathbf{x} - \mathbf{A}\mathbf{L}\mathbf{V}\lambda \quad (6.2b)$$

$$\dot{\lambda} = -\mathbf{L}\mathbf{V}\mathbf{x} \quad (6.2c)$$

$$\dot{\mathbf{v}} = -\mathbf{L}\mathbf{v} \quad (6.2d)$$

$$\dot{\mathbf{w}} = -\mathbf{L}\mathbf{w} \quad (6.2e)$$

where $\mathbf{V} = \text{diag}(\mathbf{v})$ and $\mathbf{W} = \text{diag}(\mathbf{w})$. The two new variables $\mathbf{v}$ and $\mathbf{w}$ introduced will now converge to our necessary right eigen vector with 0 eigen value. The values of these vectors are then put in a diagonal matrix to form the vector $\mathbf{V}$ and $\mathbf{W}$ which is then replaced with $\bar{\mathbf{V}}$ and $\bar{\mathbf{W}}$ of the old dynamics. This will give us the new dynamics. Here as we discussed above the variable $\mathbf{v}$ associates with the dimensions of $\mathbf{x}$ and λ , whereas $\mathbf{w}$ associates more with the variable $\mathbf{y}$ due to the dimensions. Hence the variable $\mathbf{w}$ has no significance without $\mathbf{y}$.

Important Note

Coming back to discussion of initiation values of all the variables, we have adopted this algorithm purely from the previous implementation of directed balanced networks. Hence, we can conclude that the variable $\mathbf{x}$ and λ can have any initiation value. On the other hand $\mathbf{y} = \mathbf{A}\mathbf{x} - \mathbf{b}$ for non zero $\mathbf{x}$ vector and $\mathbf{y} = -\mathbf{b}$ for a zero $\mathbf{x}$ vector. Now coming to the new variables $\mathbf{v}$ and $\mathbf{w}$, from the (6.2) we can see that any zero vector initiation of these variables $\mathbf{v}$ and $\mathbf{w}$ will give no change to the variables and making them stay at 0 forever. Hence, initiation values of $\mathbf{v}$ and $\mathbf{w}$ should be mandatorily a non-zero vector and the value should be chosen such a way that it does not lead to divergence of our equations. It also has been observed that higher initiation values of the variables $\mathbf{v}$ and $\mathbf{w}$ will lead to faster convergence and better performance, on the other hand too huge variables leads to divergence.

The comments on the variable α and β remains the same as before. Too high values lead to divergence, whereas too low values will give us slow convergence. Hence, it is important for the user to tune the values such that we get proper and faster convergence.

Chapter 7

Observations for Simulations in Directed Weight-Unbalanced network

This chapter includes all observations for the Directed Weight-Unbalanced networks where there is neither two way transfer between nodes and the in-degree and out-degree of the nodes can be different. Basically the Directed Weight-Unbalanced case is the most general case of a connected graph. Hence, this algorithm that is used here is applicable to any fully connected systems.

7.1 Observation for a 4-Node graph

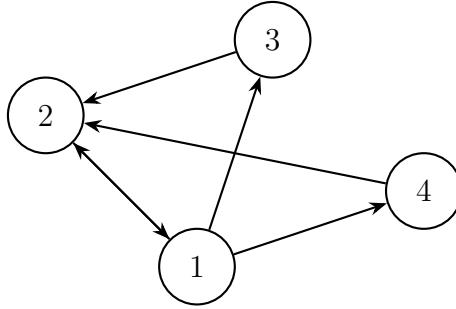


Figure 7.1: 4 Node graph used

The above graph is an example that has been used in the simulation. We observe that the indegree and outdegree of all the nodes in the graph are equal. The values of matrix A and b that has been used is given below:

$$A_1 = \begin{bmatrix} 5 & 0 & 2 & 3 \\ 5 & 2 & 2 & 6 \\ 3 & 4 & 5 & 8 \\ 0 & 5 & 4 & 2 \\ 8 & 2 & 2 & 0 \end{bmatrix} \quad A_2 = \begin{bmatrix} 6 & 1 & 6 & 4 \\ 4 & 0 & 4 & 2 \\ 1 & 0 & 3 & 6 \\ 0 & 0 & 0 & 1 \\ 4 & 1 & 2 & 3 \end{bmatrix} \quad A_3 = \begin{bmatrix} 2 & 1 & 0 & 3 \\ 6 & 2 & 3 & 5 \\ 1 & 4 & 6 & 0 \\ 2 & 3 & 2 & 2 \\ 0 & 3 & 1 & 0 \end{bmatrix} \quad A_4 = \begin{bmatrix} -6 & 2 & 0 & -5 \\ -8 & -1 & -2 & -5 \\ 0 & -4 & -6 & -6 \\ 1 & -2 & -1 & -3 \\ -4 & 0 & -3 & 2 \end{bmatrix}$$

$$b_1 = \begin{bmatrix} 61 \\ 51 \\ 1 \\ 55 \\ 1 \end{bmatrix} \quad b_2 = \begin{bmatrix} 112 \\ 11 \\ 2 \\ 80 \\ 1 \end{bmatrix} \quad b_3 = \begin{bmatrix} 57 \\ 38 \\ 100 \\ 58 \\ 53 \end{bmatrix} \quad b_4 = \begin{bmatrix} -117 \\ -4 \\ 6 \\ -107 \\ 12 \end{bmatrix}$$

$$\text{Sum}(A_i) = \begin{bmatrix} 7 & 4 & 8 & 5 \\ 7 & 3 & 7 & 8 \\ 5 & 4 & 8 & 8 \\ 3 & 6 & 5 & 2 \\ 8 & 6 & 2 & 5 \end{bmatrix} \quad \text{Sum}(b_i) = \begin{bmatrix} 113 \\ 96 \\ 109 \\ 86 \\ 67 \end{bmatrix}$$

The solution of the above sum is expected to be:

$$\text{Solution vector} = [1.96203376 \quad 5.35884552 \quad 9.74942138 \quad -0.12373404]$$

Actual solutions in each node

Node 1 has values: $[1.96169436 \quad 5.35907473 \quad 9.74930708 \quad -0.12343668]$

Node 2 has values: $[1.961694 \quad 5.35907487 \quad 9.74930703 \quad -0.1234365]$

Node 3 has values: $[1.96169378 \quad 5.35907732 \quad 9.7493112 \quad -0.12343815]$

Node 4 has values: $[1.96169563 \quad 5.35907146 \quad 9.74930265 \quad -0.12343576]$

Evolution of Solution in each Node

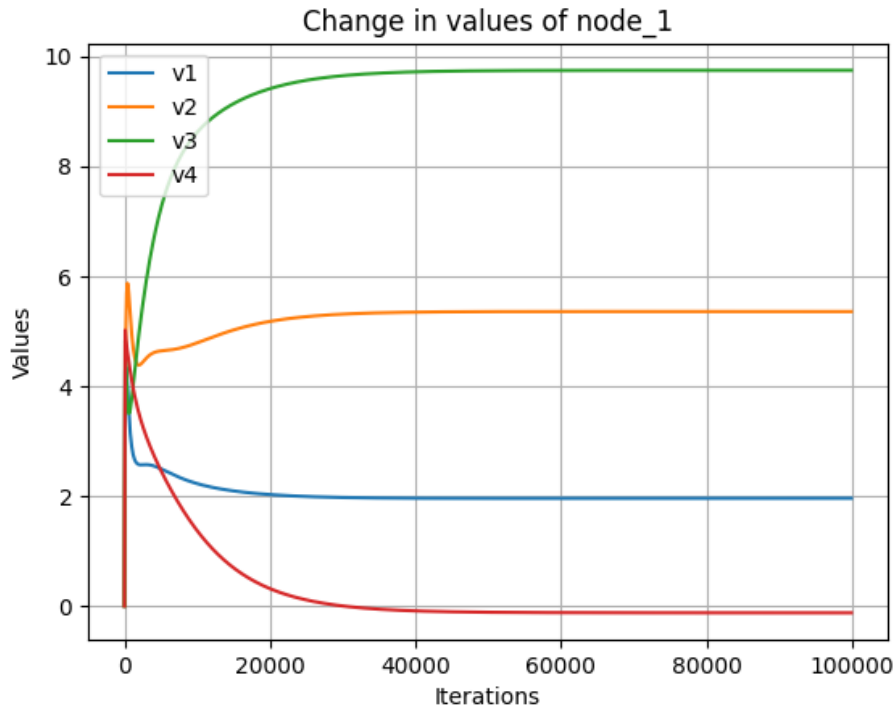


Figure 7.2: Convergence of x in Node 1 in the above graph network

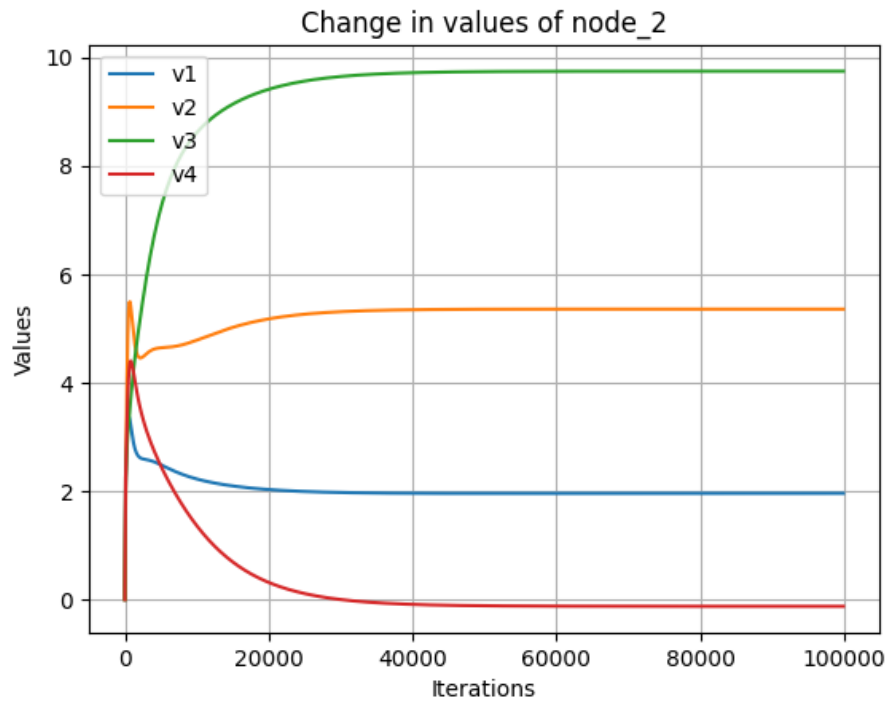


Figure 7.3: Convergence of x in Node 2 in the above graph network

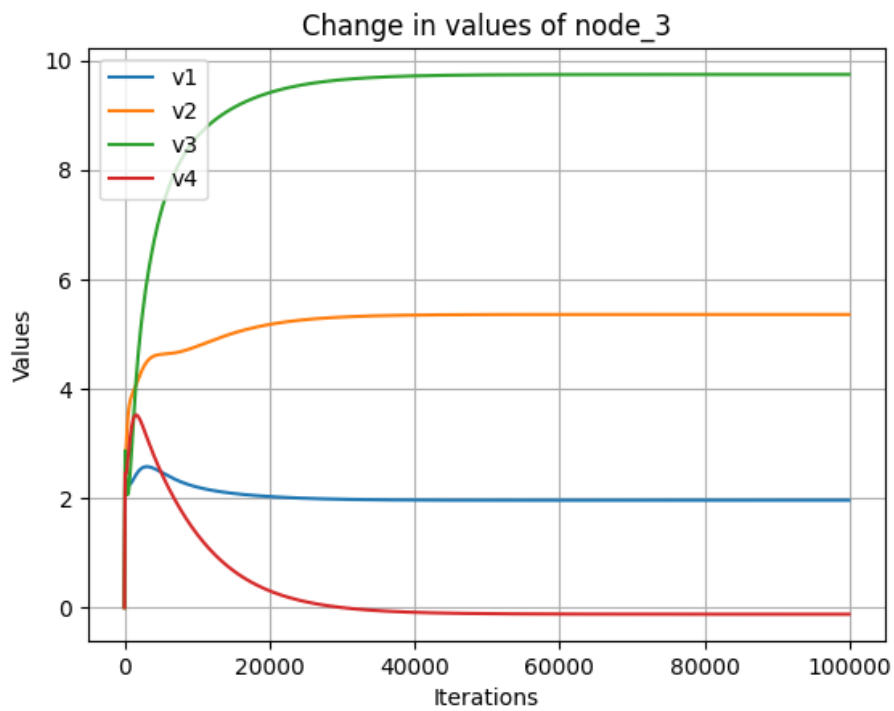


Figure 7.4: Convergence of x in Node 3 in the above graph network

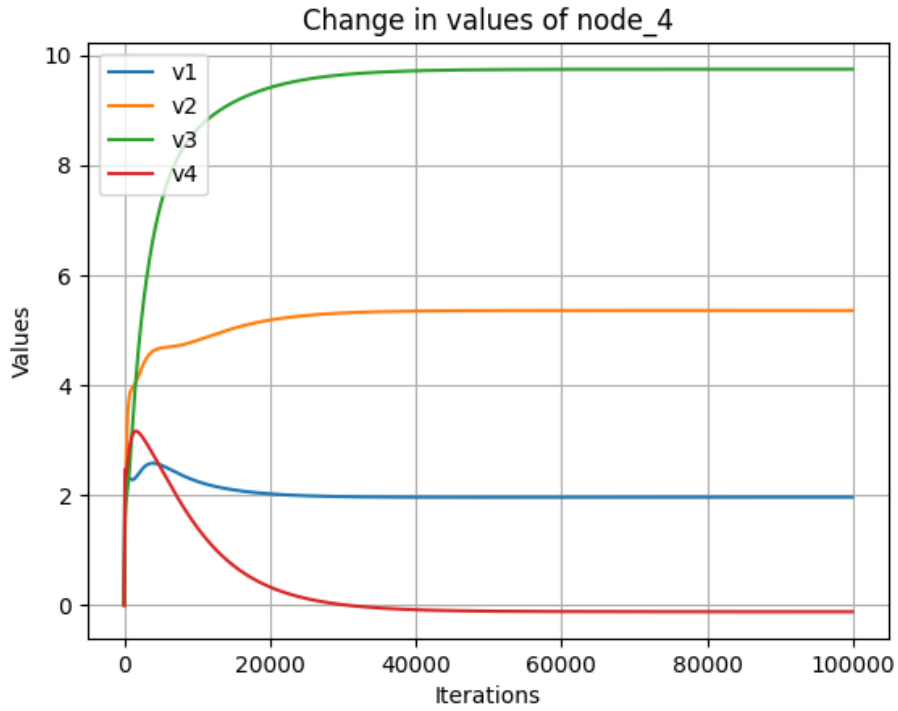


Figure 7.5: Convergence of x in Node 4 in the above graph network

7.2 Observation for a different 4-Node graph

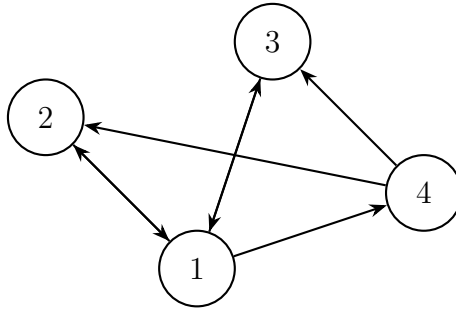


Figure 7.6: 4 Node graph used for upcoming implementation

The above graph is an example that has been used in the simulation. The values of matrix A and b that has been used is given below. In this case we gave single row of a set of five equations to 3 nodes and the other node had 2 rows. We then observed the evolution of solution.

$$A_1 = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad A_2 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 2 & -1 & 3 & -1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad A_3 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 2 & -1 & 2 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad A_4 = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 3 & -1 & 1 & 2 \\ 2 & 1 & 2 & -3 \end{bmatrix}$$

$$b_1 = \begin{bmatrix} 10 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad b_2 = \begin{bmatrix} 0 \\ 12 \\ 0 \\ 0 \\ 0 \end{bmatrix} \quad b_3 = \begin{bmatrix} 0 \\ 0 \\ 5 \\ 0 \\ 0 \end{bmatrix} \quad b_4 = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 15 \\ 7 \end{bmatrix}$$

$$\text{Sum}(A_i) = \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & -1 & 3 & -1 \\ 1 & 2 & -1 & 2 \\ 3 & -1 & 1 & 2 \\ 2 & 1 & 2 & -3 \end{bmatrix} \quad \text{Sum}(b_i) = \begin{bmatrix} 10 \\ 12 \\ 5 \\ 15 \\ 7 \end{bmatrix}$$

The solution of the above sum is expected to be:

$$\text{Solution vector} = [2.62343159 \quad 1.0521542 \quad 3.53575208 \quad 2.19002268]$$

Actual solutions in each node

Node 1 has values: [2.62347298 1.05212885 3.53571908 2.19000136]

Node 2 has values: [2.62347143 1.05215565 3.53569189 2.19002865]

Node 3 has values: [2.62347268 1.05212468 3.53572298 2.18999711]

Node 4 has values: [2.62347434 1.05212904 3.53571995 2.19000152]

Evolution of Solution in each Node

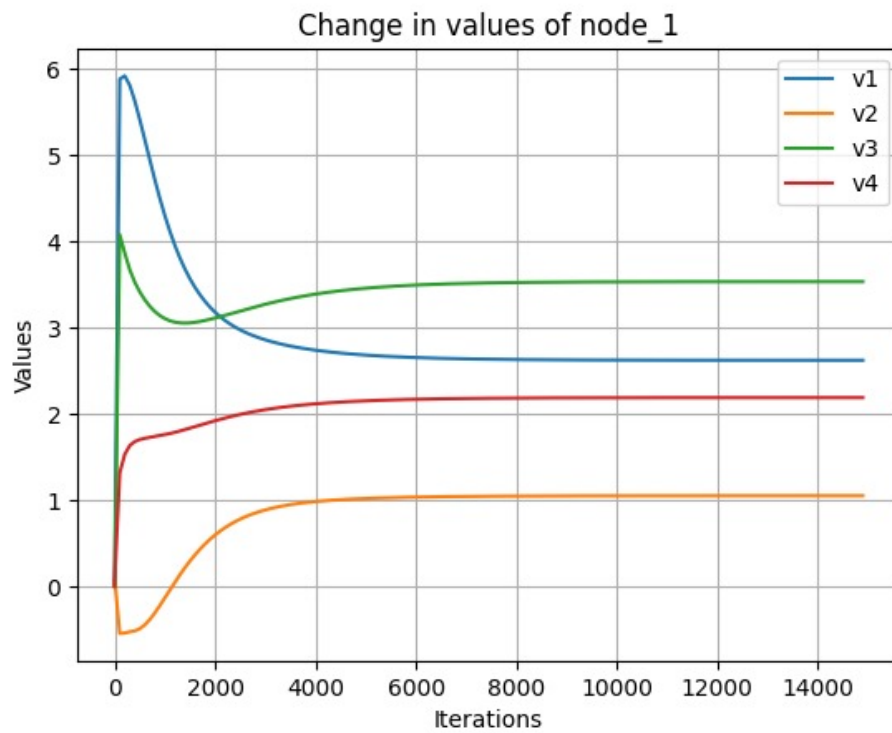


Figure 7.7: Convergence of x in Node 1 in the above graph network

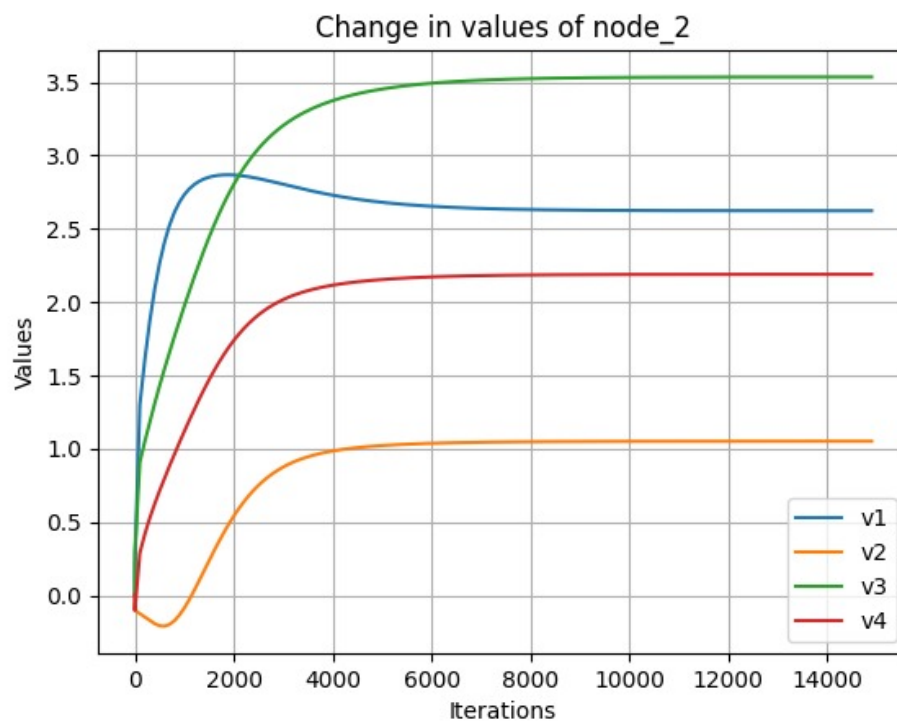


Figure 7.8: Convergence of x in Node 2 in the above graph network

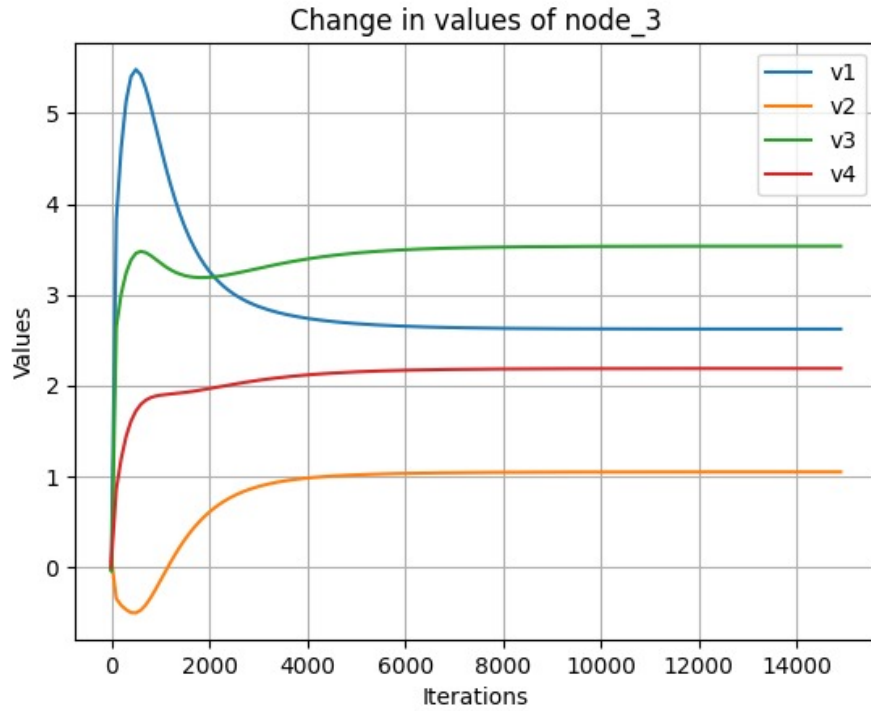


Figure 7.9: Convergence of x in Node 3 in the above graph network

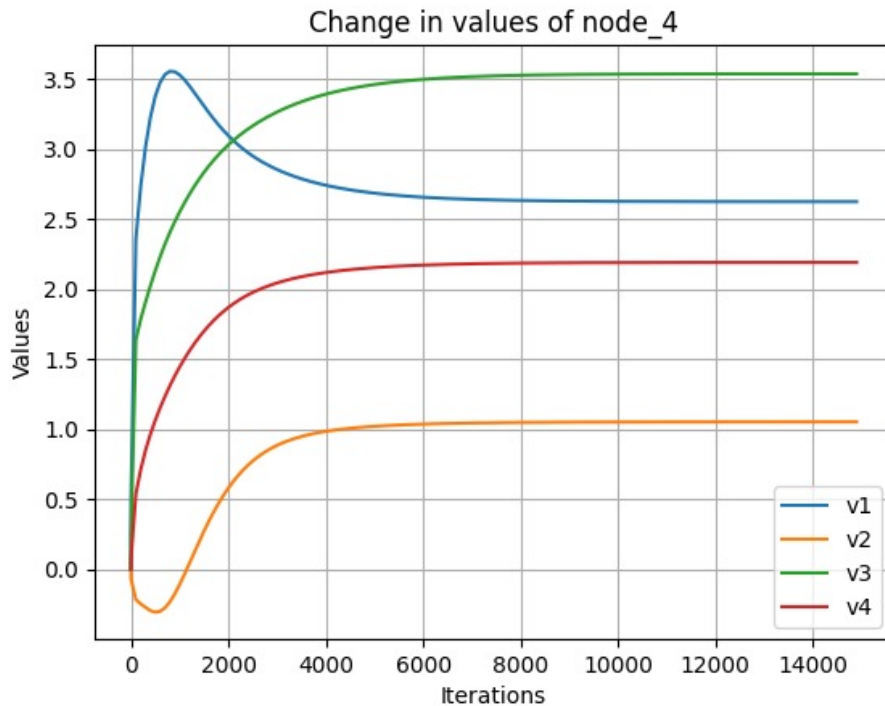


Figure 7.10: Convergence of x in Node 4 in the above graph network

The above example has been chosen specifically in such a way that the algorithm converges faster. Since the convergence occurs faster we can observe the initial evolution of values in different nodes clearly. So far all the graphs we observe for all nodes will be the

same because of high iteration number. Our nodes come in consensus much faster than they converge to required solution. Hence, this example has been added specifically to show the way in which different node values evolve. This example also illustrates us the generalization of the algorithm which we found. It is a general case of algorithm which includes the solutions for problems in sensor fusion where each row of the matrix A and vector b is provided by a node. This example clearly illustrates that we can have a cases where each nodes produces few rows of the matrix A and b, which then can be combined to find the final precise solution.

7.3 Observation for a 7-Node graph

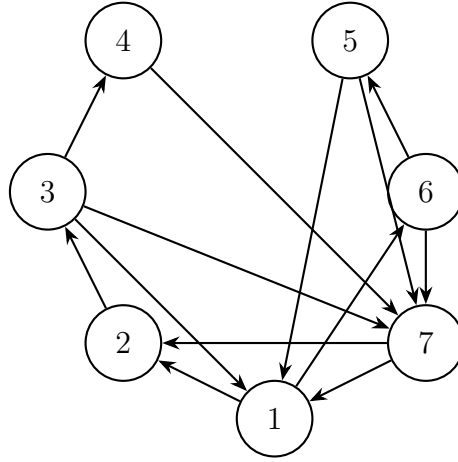


Figure 7.11: 7-Node graph used

The above graph is an example that has been used in the simulation. The A matrix has a dimension of $A \in \mathbb{R}^{9 \times 11}$ and there are 7 Nodes. So for simplicity only the sum of all A matrices and b vectors has been written here. The algorithm works for any kind of split of the sum matrix to 7 matrices and sum vector to any 7 vector.

$$\text{Sum}(A_i) = \begin{bmatrix} 7 & 4 & 8 & 5 & 3 & 1 & 2 & 1 & 6 \\ 7 & 3 & 7 & 8 & 1 & 4 & 4 & 2 & 8 \\ 5 & 4 & 8 & 8 & 3 & 7 & 9 & 11 & 1 \\ 3 & 6 & 5 & 2 & 2 & 2 & 3 & 3 & 4 \\ 8 & 6 & 2 & 5 & 1 & 4 & 9 & 3 & 5 \\ 1 & 9 & 6 & 8 & 8 & 9 & 7 & 1 & 4 \\ 5 & 4 & 4 & 4 & 7 & 8 & 3 & 2 & 1 \\ 2 & 3 & 5 & 2 & 6 & 9 & 1 & 3 & 5 \\ 11 & 2 & 3 & 1 & 2 & 1 & 5 & 11 & 10 \\ 1 & 0 & 4 & 8 & 2 & 5 & 1 & 0 & 0 \\ 2 & 4 & 6 & 7 & 12 & 13 & 1 & 10 & 8 \end{bmatrix} \quad \text{Sum}(b_i) = \begin{bmatrix} 113 \\ 96 \\ 109 \\ 86 \\ 67 \\ 154 \\ 71 \\ 186 \\ 100 \\ 36 \\ 47 \end{bmatrix}$$

The solution of the above sum are expected to be:

$$\text{Solution vector} = \begin{bmatrix} -6.48130661 & -19.6370442 & 28.61634296 & -23.70508843 & 4.9507962 \\ 8.2730601 & 28.12637275 & -13.38024296 & 11.14381635 & \end{bmatrix}$$

Actual Solution

Node 1 has values:	$\begin{bmatrix} -6.46569119 & -19.58963927 & 28.59725839 & -23.68991023 & 4.9171309 \\ 8.28885529 & 28.0786232 & -13.36392626 & 11.13018842 & \end{bmatrix}$
Node 2 has values:	$\begin{bmatrix} -6.46569101 & -19.58963936 & 28.59725856 & -23.68991036 & 4.91713091 \\ 8.28885533 & 28.07862341 & -13.36392655 & 11.1301882 & \end{bmatrix}$
Node 3 has values:	$\begin{bmatrix} -6.46569107 & -19.58963947 & 28.59725847 & -23.68991032 & 4.91713098 \\ 8.28885525 & 28.0786234 & -13.36392645 & 11.13018817 & \end{bmatrix}$
Node 4 has values:	$\begin{bmatrix} -6.46569109 & -19.58964022 & 28.5972586 & -23.68991043 & 4.9171309 \\ 8.28885524 & 28.07862381 & -13.36392672 & 11.13018874 & \end{bmatrix}$
Node 5 has values:	$\begin{bmatrix} -6.46569129 & -19.58963927 & 28.59725831 & -23.68991018 & 4.91713089 \\ 8.28885524 & 28.078623 & -13.36392613 & 11.13018841 & \end{bmatrix}$
Node 6 has values:	$\begin{bmatrix} -6.46569128 & -19.58963927 & 28.59725829 & -23.68991015 & 4.9171309 \\ 8.28885523 & 28.0786234 & -13.36392609 & 11.13018845 & \end{bmatrix}$
Node 7 has values:	$\begin{bmatrix} -6.46569108 & -19.58963937 & 28.59725842 & -23.68991009 & 4.91713094 \\ 8.28885514 & 28.07862355 & -13.36392626 & 11.13018837 & \end{bmatrix}$

Evolution of Solutions in each Node

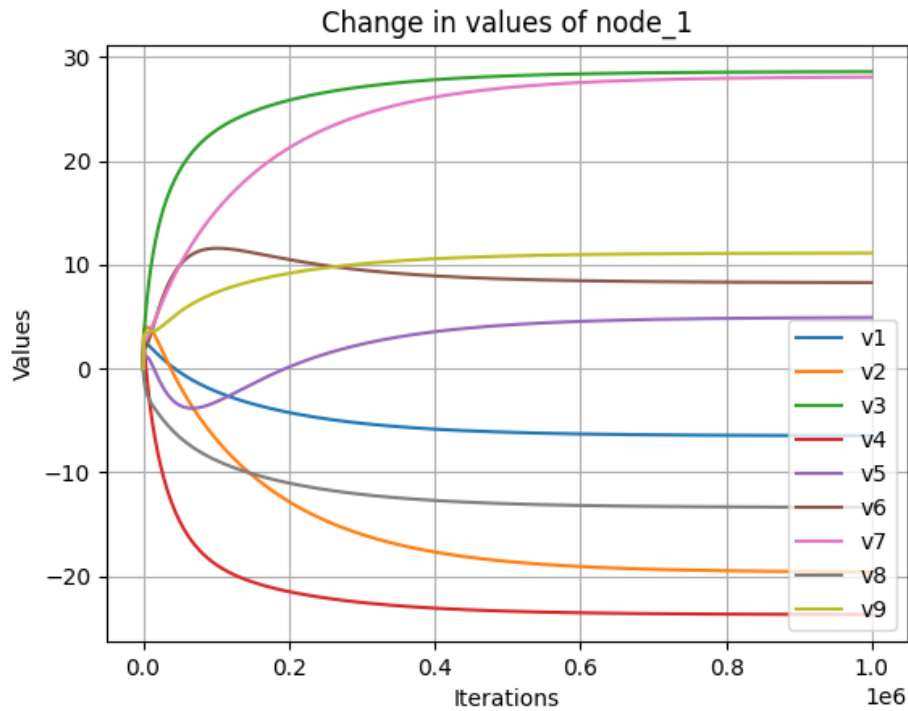


Figure 7.12: Convergence of x in Node 1 in the above graph network

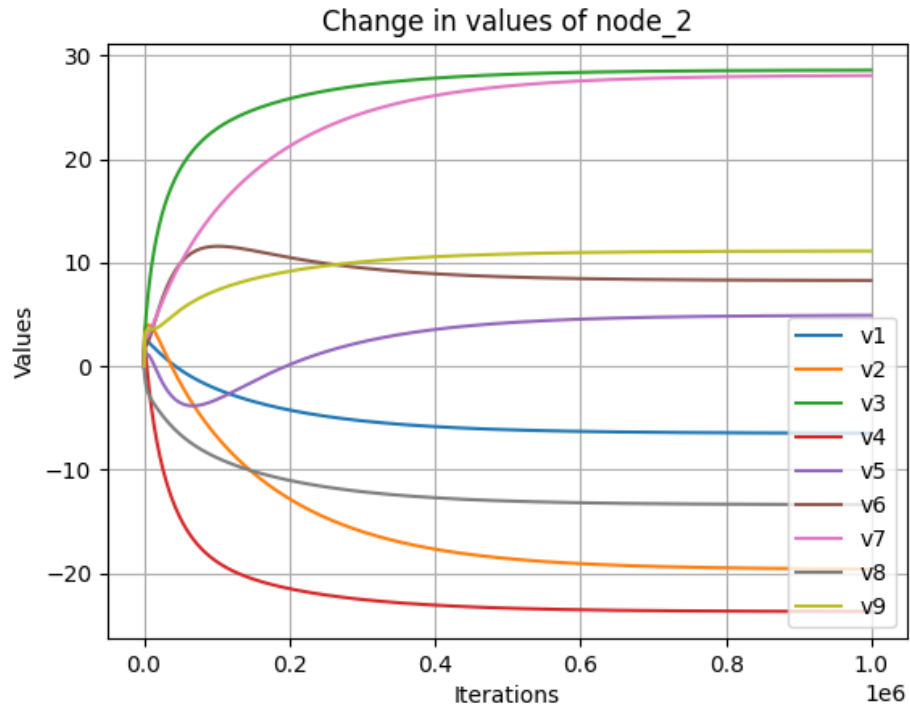


Figure 7.13: Convergence of x in Node 2 in the above graph network

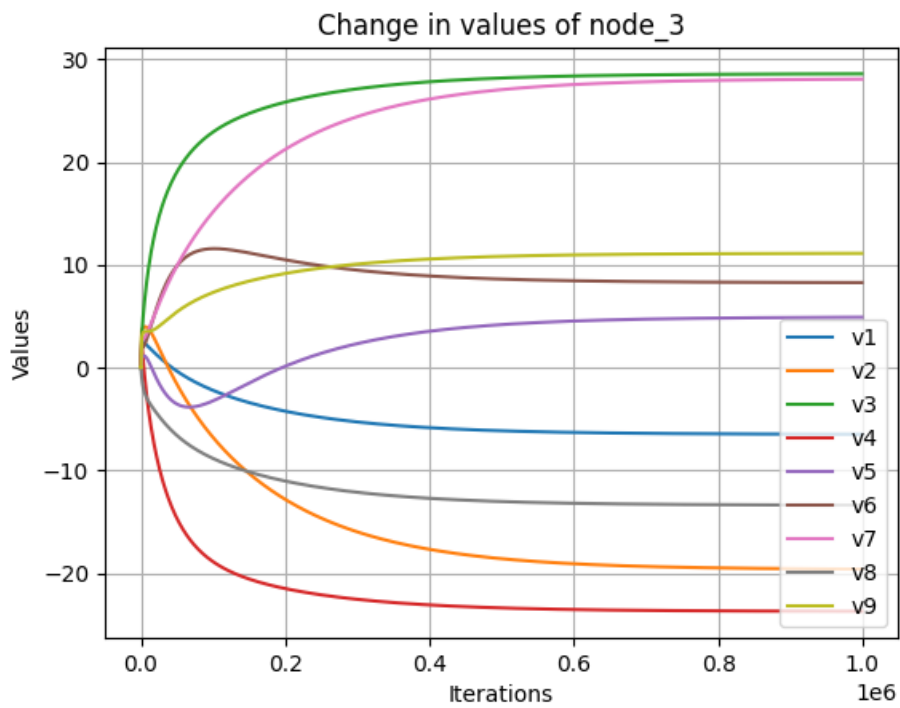


Figure 7.14: Convergence of x in Node 3 in the above graph network

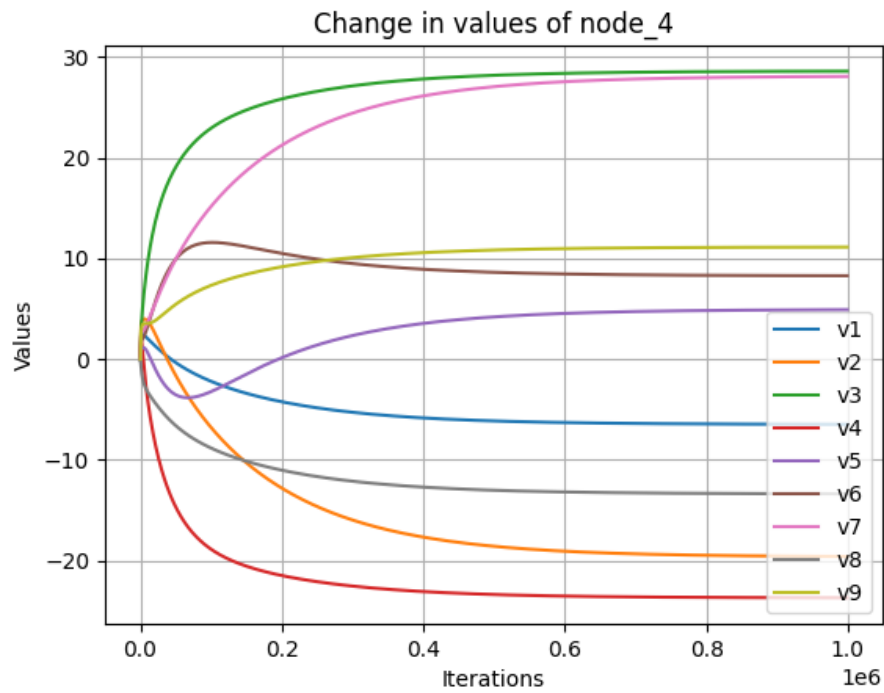


Figure 7.15: Convergence of x in Node 4 in the above graph network

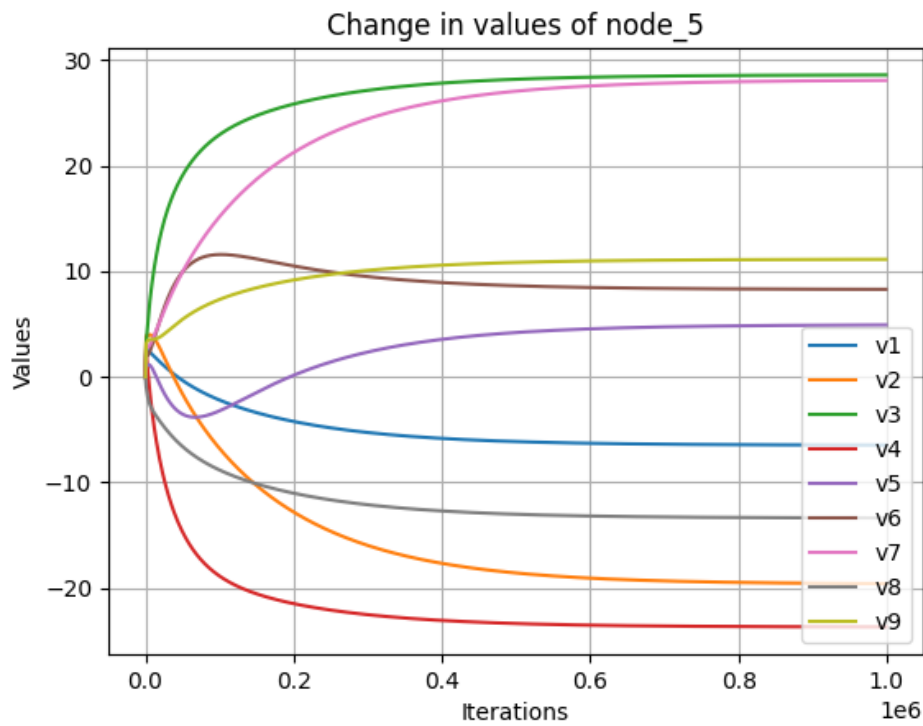


Figure 7.16: Convergence of x in Node 5 in the above graph network

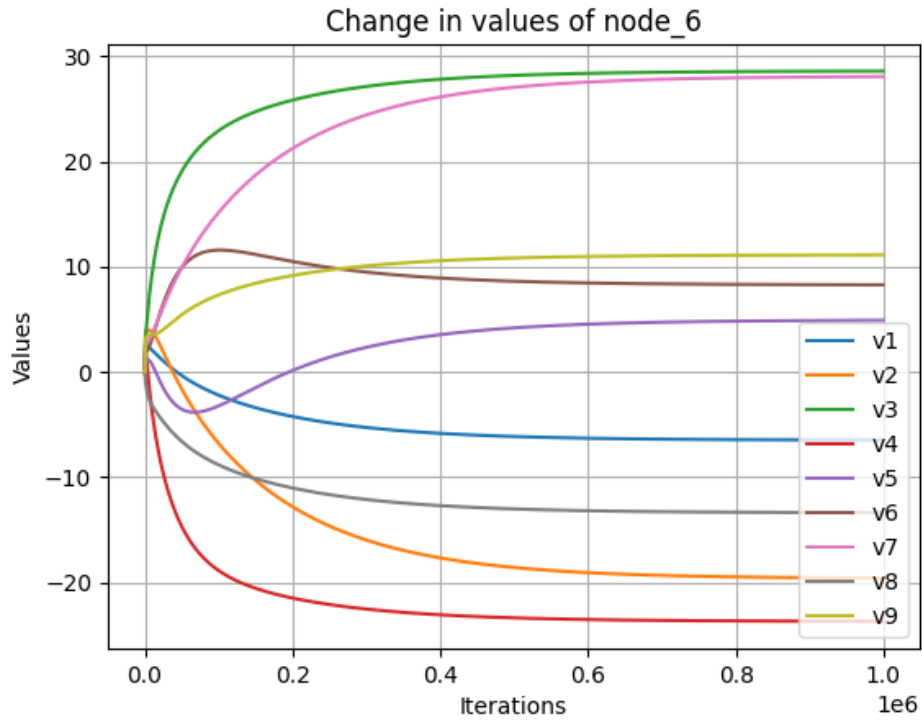


Figure 7.17: Convergence of x in Node 6 in the above graph network

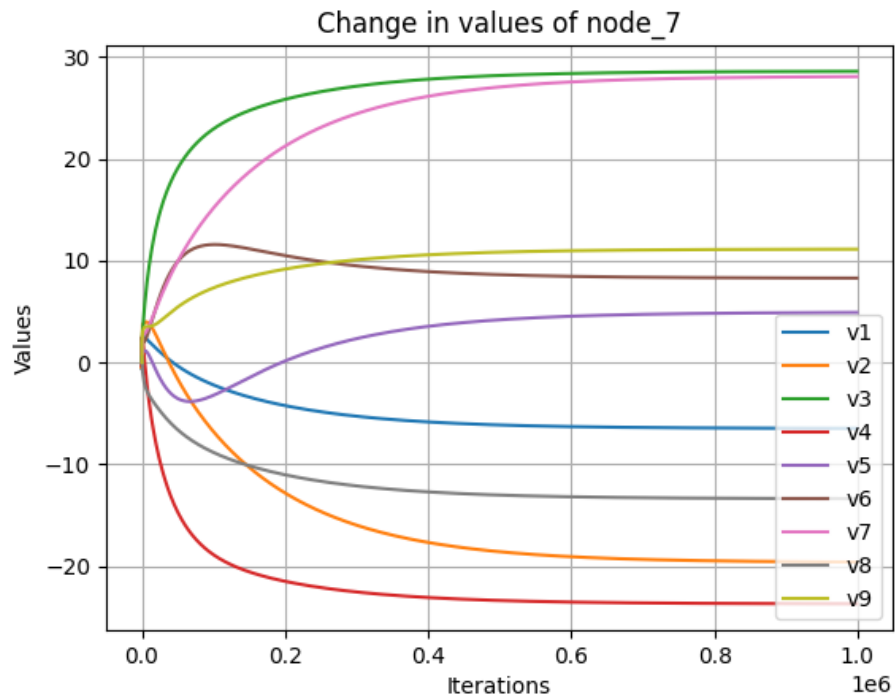


Figure 7.18: Convergence of x in Node 7 in the above graph network

We observe similar plots in each of the 7 nodes because the our node values are in consensus much faster (in less than 10000 iterations). But it takes more iterations for our values to converge to the right solution instead.

Chapter 8

Results from Multi Node systems

8.1 Multi-file Implementation in a single computer

In this section we show some results for multi-file implementation of certain cases. In single file implementation there was no issue of synchronizing every node as all things happened procedural, which was responsible for all iterations to be in sync with each other. On the other hand in multi-file implementation we have to manually synchronize every node with each other. So we have a sleep time functionality where every node sleeps for a certain period after implementing the iteration. The sleep time is manually set by us and is generally in a few milliseconds. So by this mechanism we synchronize every node. But the problem with this functionality is that we can not lower it more than a certain value as the python sleep function has some error. This error tends to manipulate the values when we set low sleep times. Some results are shown for in the upcoming chapter. We only consider directed weight-unbalanced networks to get the most generalized results.

Observations for 4-Node system

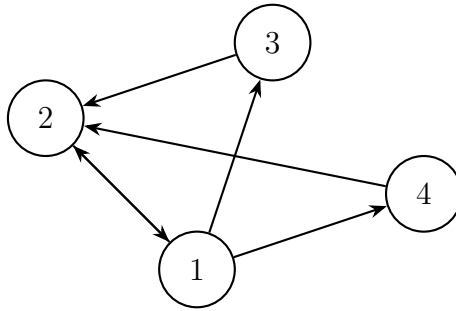


Figure 8.1: Normal 4 Node graph used

$$\text{Sum}(A_i) = \begin{bmatrix} 7 & 4 & 8 & 5 \\ 7 & 3 & 7 & 8 \\ 5 & 4 & 8 & 8 \\ 3 & 6 & 5 & 2 \\ 8 & 6 & 2 & 5 \end{bmatrix} \quad \text{Sum}(b_i) = \begin{bmatrix} 113 \\ 96 \\ 109 \\ 86 \\ 67 \end{bmatrix}$$

The solution of the above sum is expected to be:

Solution vector = $[1.96203376 \quad 5.35884552 \quad 9.74942138 \quad -0.12373404]$

We will only be showing observation from one node for ease in the upcoming sections.

Expected result

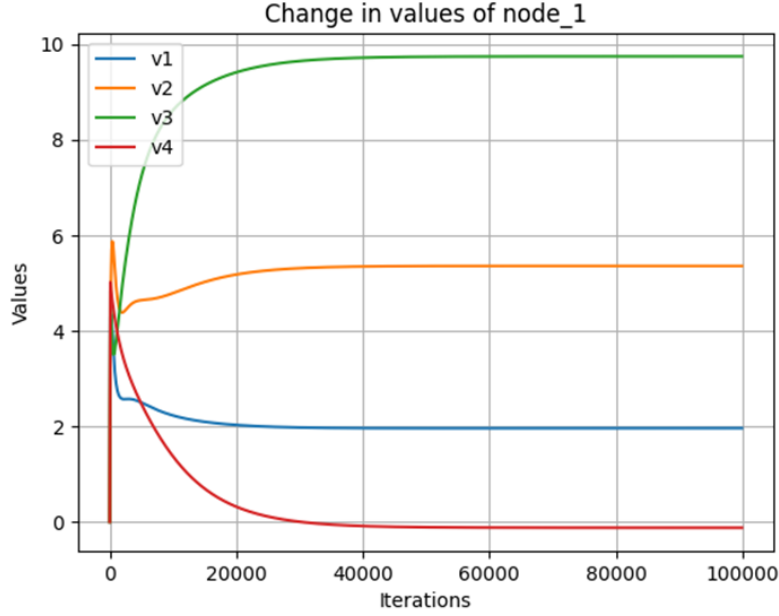


Figure 8.2: Convergence of x in Node 1

Actual solutions in each node for 0.0075s sleep time

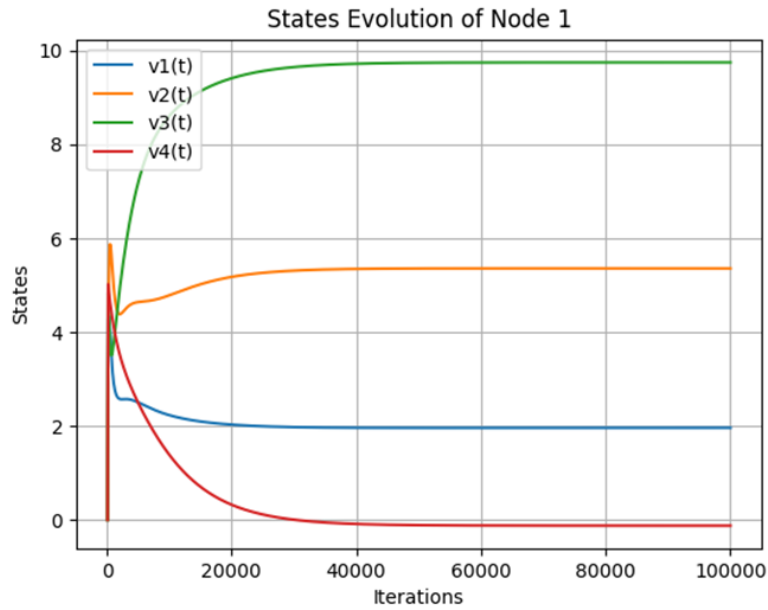


Figure 8.3: Convergence of x in Node 1

Actual solutions in each node for 0.005s sleep time

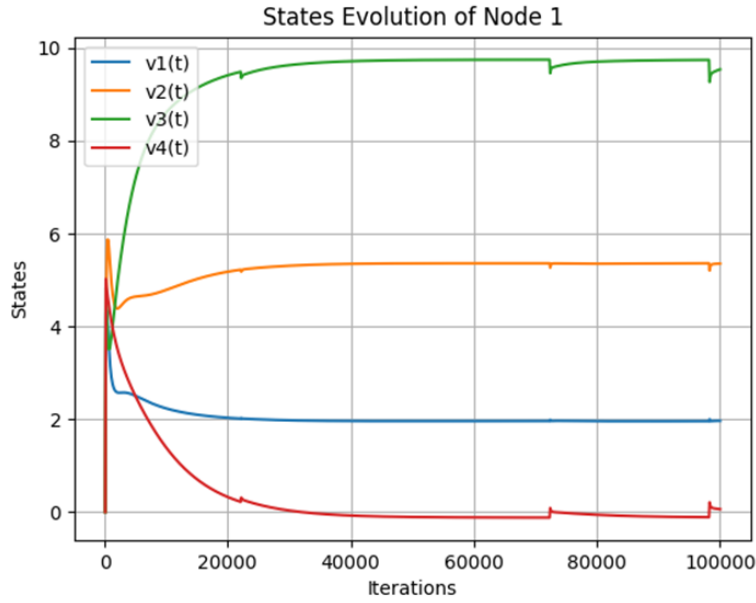


Figure 8.4: Convergence of x in Node 1

We see distortions in the above graph due to low sleep time. Due to low sleep time. Hence its optimal to set sleep time around 0.0075s.

Observation for 7-Node graph

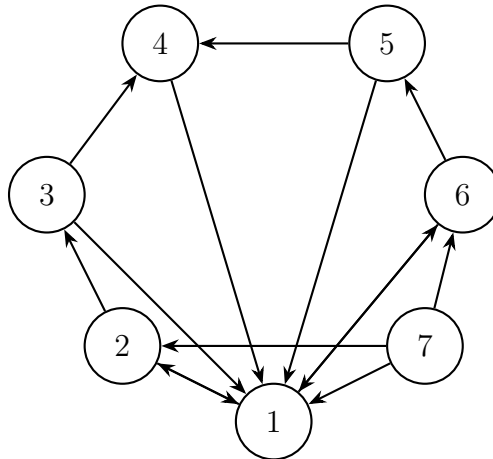


Figure 8.5: Normal 6-Node graph used

The above graph is an example that has been used in the simulation. The A matrix has a dimension of $A \in \mathbb{R}^{9 \times 11}$ and there are 7 Nodes. So for simplicity only the sum of all A matrices and b vectors has been written here. The algorithm works for any kind of split of the sum matrix to 7 matrices and sum vector to any 7 vector.

$$\text{Sum}(A_i) = \begin{bmatrix} 7 & 4 & 8 & 5 & 3 & 1 & 2 & 1 & 6 \\ 7 & 3 & 7 & 8 & 1 & 4 & 4 & 2 & 8 \\ 5 & 4 & 8 & 8 & 3 & 7 & 9 & 11 & 1 \\ 3 & 6 & 5 & 2 & 2 & 2 & 3 & 3 & 4 \\ 8 & 6 & 2 & 5 & 1 & 4 & 9 & 3 & 5 \\ 1 & 9 & 6 & 8 & 8 & 9 & 7 & 1 & 4 \\ 5 & 4 & 4 & 4 & 7 & 8 & 3 & 2 & 1 \\ 2 & 3 & 5 & 2 & 6 & 9 & 1 & 3 & 5 \\ 11 & 2 & 3 & 1 & 2 & 1 & 5 & 11 & 10 \\ 1 & 0 & 4 & 8 & 2 & 5 & 1 & 0 & 0 \\ 2 & 4 & 6 & 7 & 12 & 13 & 1 & 10 & 8 \end{bmatrix} \quad \text{Sum}(b_i) = \begin{bmatrix} 113 \\ 96 \\ 109 \\ 86 \\ 67 \\ 154 \\ 71 \\ 186 \\ 100 \\ 36 \\ 47 \end{bmatrix}$$

The solution of the above sum are expected to be:

$$\text{Solution vector} = \begin{bmatrix} -6.48130661 & -19.6370442 & 28.61634296 & -23.70508843 & 4.9507962 \\ 8.2730601 & 28.12637275 & -13.38024296 & 11.14381635 & \end{bmatrix}$$

Actual Solution with sleep time of 0.004s

For the ease of reading we have added only 3 nodes to show the evolution of solution along with all the finals values.

$$\begin{aligned} \text{Node 1 has values:} & \begin{bmatrix} -6.61562645 & -20.03746178 & 28.85492757 & -23.93407298 & 5.13774792 \\ 8.24298059 & 28.53737171 & -13.53803074 & 11.26458837 & \end{bmatrix} \\ \text{Node 2 has values:} & \begin{bmatrix} -6.61562617 & -20.0374618 & 28.85492808 & -23.93407348 & 5.13774819 \\ 8.24298061 & 28.53737197 & -13.53803145 & 11.26458867 & \end{bmatrix} \\ \text{Node 3 has values:} & \begin{bmatrix} -6.6156262 & -20.03746224 & 28.85492811 & -23.9340739 & 5.1377487 \\ 8.28885525 & 28.53737225 & -13.53803166 & 11.26458864 & \end{bmatrix} \\ \text{Node 4 has values:} & \begin{bmatrix} -6.61562682 & -20.0374636 & 28.85492824 & -23.93407378 & 5.13774878 \\ 8.2429806 & 28.53737279 & -13.53803172 & 11.26458799 & \end{bmatrix} \\ \text{Node 5 has values:} & \begin{bmatrix} -6.61562721 & -20.03746264 & 28.85492751 & -23.93407334 & 5.13774827 \\ 8.24298054 & 28.53737196 & -13.53803063 & 11.26458802 & \end{bmatrix} \\ \text{Node 6 has values:} & \begin{bmatrix} -6.61562687 & -20.03746186 & 28.85492728 & -23.93407307 & 5.13774781 \\ 8.24298077 & 28.53737135 & -13.53803043 & 11.26458807 & \end{bmatrix} \\ \text{Node 7 has values:} & \begin{bmatrix} -6.61562649 & -20.03746175 & 28.85492761 & -23.93407309 & 5.13774793 \\ 8.24298062 & 28.53737168 & -13.53803079 & 11.26458838 & \end{bmatrix} \end{aligned}$$

Evolution of Solutions in each Node

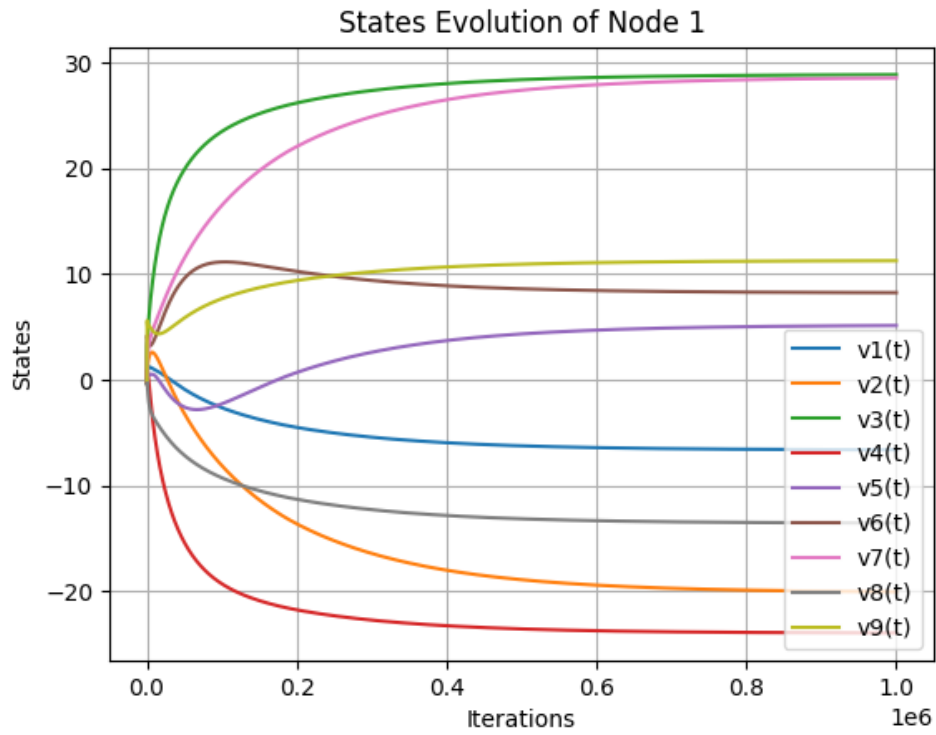


Figure 8.6: Convergence of x in Node 1

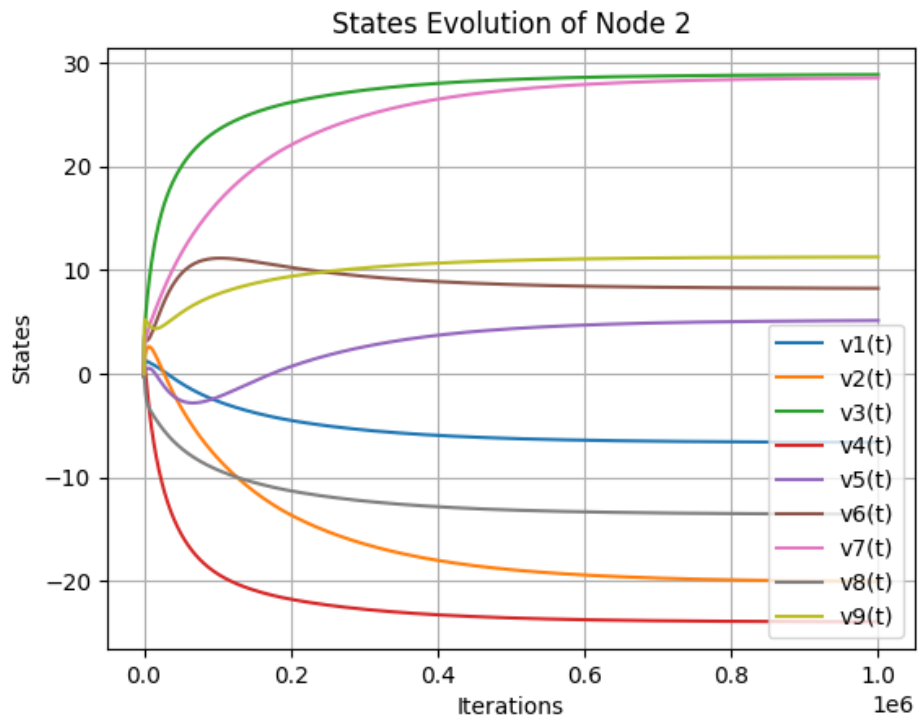


Figure 8.7: Convergence of x in Node 2

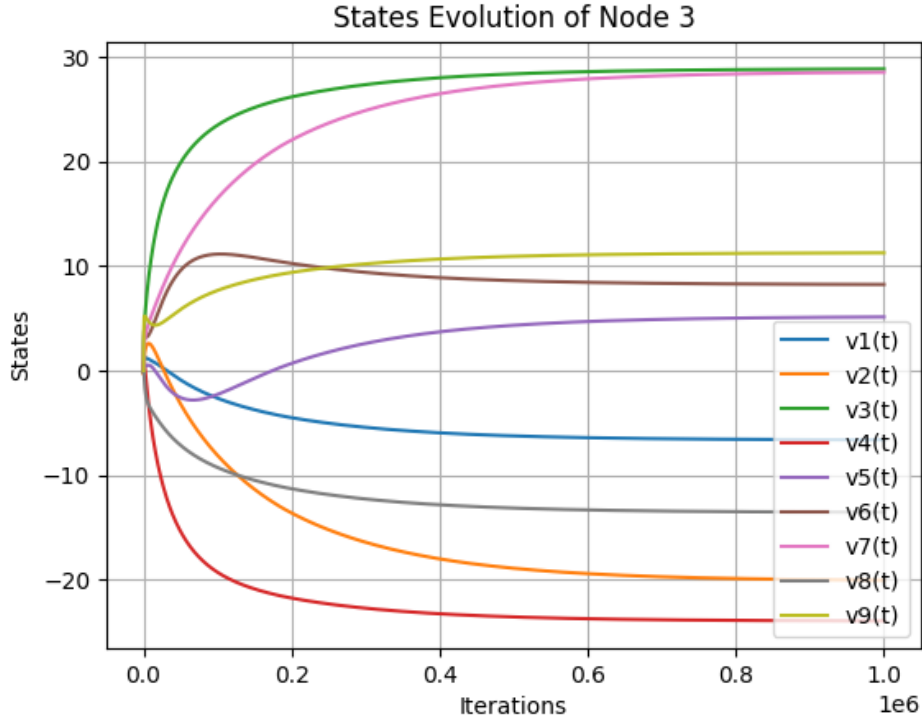


Figure 8.8: Convergence of x in Node 3

The above took 4000 seconds for convergence. We also see that the above code had some considerable error from the required observation. This is because of the low sleep time used and hence we will test it with higher sleeptime now.

Actual Solution with sleep time of 0.006s

For the ease of reading we have added only 3 nodes to show the evolution of solution along with all the finals values.

Node 1 has values:	$\begin{bmatrix} -6.56589965 & -19.88299527 & 28.77336504 & -23.86214152 & 5.03543345 \\ 8.28314495 & 28.37738967 & -13.4789417 & 11.22274024 \end{bmatrix}$
Node 2 has values:	$\begin{bmatrix} -6.56589946 & -19.88299536 & 28.77336546 & -23.86214193 & 5.0354337 \\ 8.28314494 & 28.37738994 & -13.47894226 & 11.22274049 \end{bmatrix}$
Node 3 has values:	$\begin{bmatrix} -6.5658994 & -19.88299544 & 28.77336537 & -23.86214217 & 5.03543391 \\ 8.28314487 & 28.3773899 & -13.47894234 & 11.22274039 \end{bmatrix}$
Node 4 has values:	$\begin{bmatrix} -6.56589984 & -19.88299638 & 28.77336543 & -23.86214204 & 5.0354339 \\ 8.28314505 & 28.37739021 & -13.47894234 & 11.22273987 \end{bmatrix}$
Node 5 has values:	$\begin{bmatrix} -6.56590022 & -19.88299589 & 28.77336498 & -23.86214179 & 5.03543369 \\ 8.28314493 & 28.37738983 & -13.4789416 & 11.22273996 \end{bmatrix}$
Node 6 has values:	$\begin{bmatrix} -6.56590014 & -19.88299584 & 28.77336503 & -23.86214176 & 5.03543371 \\ 8.28314493 & 28.37738991 & -13.47894164 & 11.22274016 \end{bmatrix}$
Node 7 has values:	$\begin{bmatrix} -6.5658997 & -19.88299532 & 28.7733651 & -23.86214163 & 5.0354335 \\ 8.28314495 & 28.37738972 & -13.47894176 & 11.22274027 \end{bmatrix}$

Evolution of Solutions in each Node

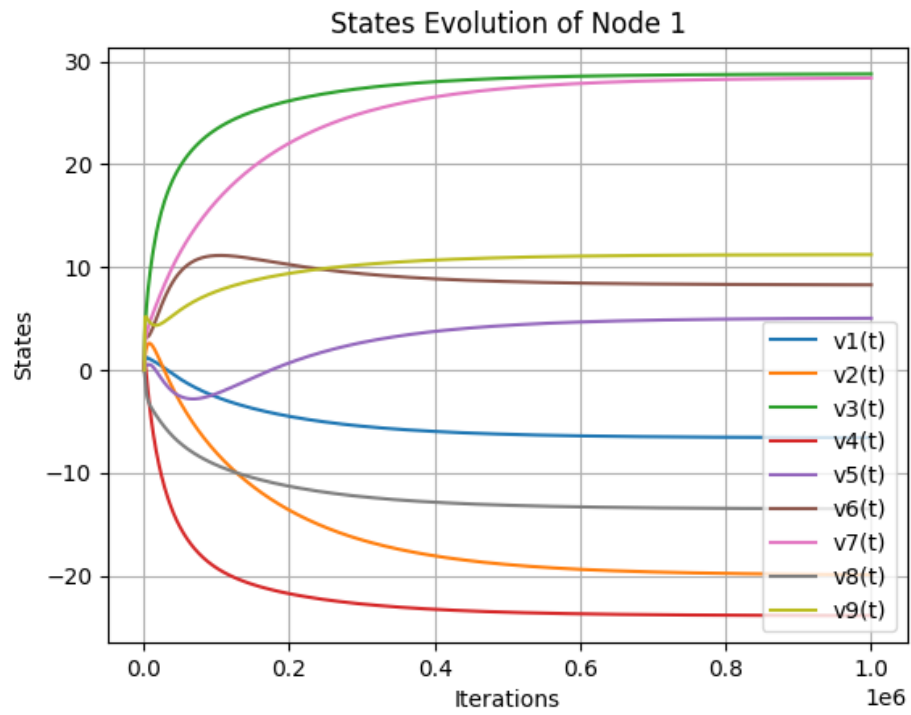


Figure 8.9: Convergence of x in Node 1

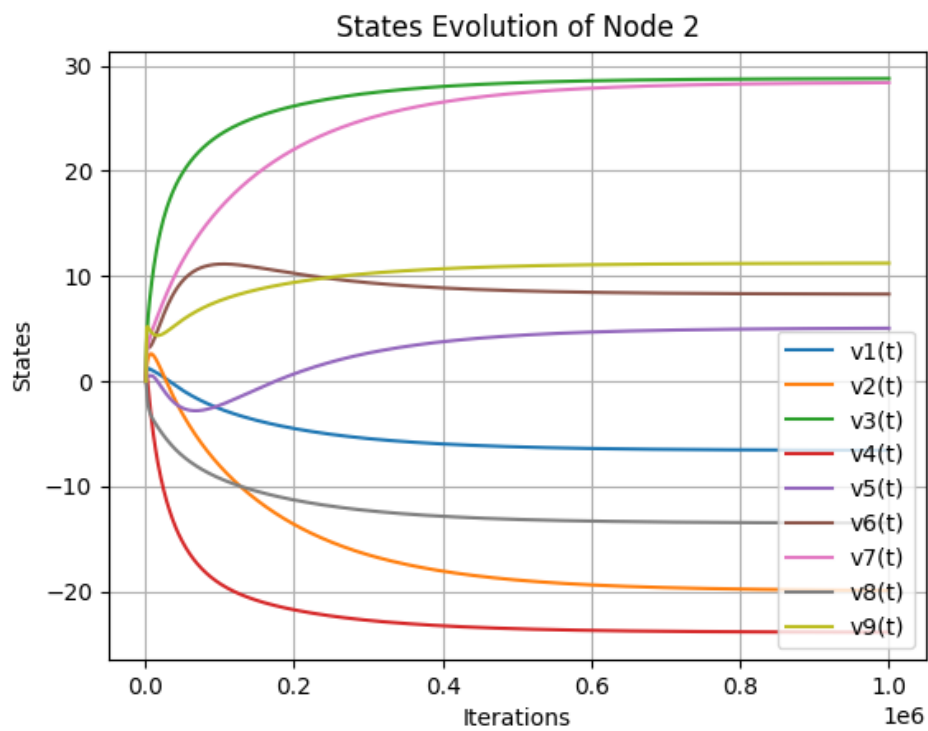


Figure 8.10: Convergence of x in Node 2

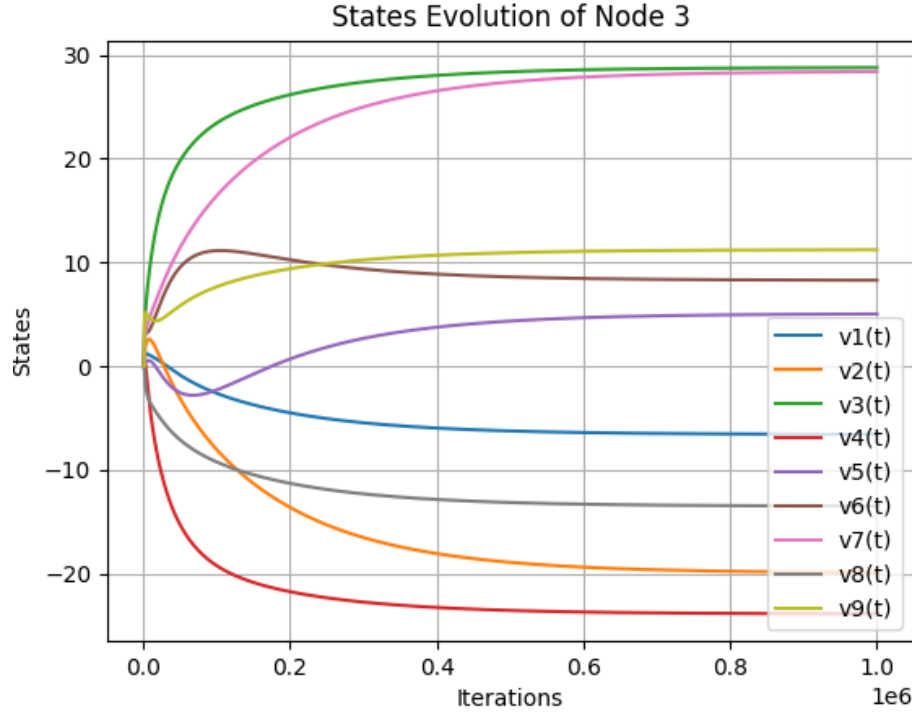


Figure 8.11: Convergence of x in Node 3

The above took 6000 seconds for convergence. We also see that the above code had less error than the previous. Hence we can clearly see that increasing sleeptime will give us better results but slower results.

8.2 Raspberry Pi Implementation in LAN (Using Ethernet)

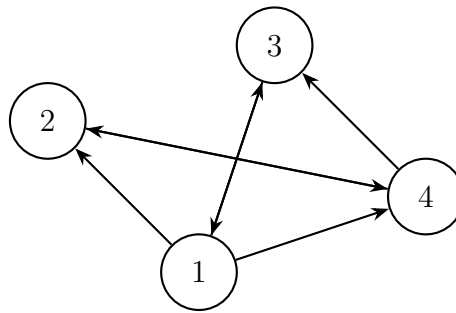


Figure 8.12: Normal 4 Node graph used

$$\text{Sum}(A_i) = \begin{bmatrix} 7 & 4 & 8 & 5 \\ 7 & 3 & 7 & 8 \\ 5 & 4 & 8 & 8 \\ 3 & 6 & 5 & 2 \\ 8 & 6 & 2 & 5 \end{bmatrix} \quad \text{Sum}(b_i) = \begin{bmatrix} 113 \\ 96 \\ 109 \\ 86 \\ 67 \end{bmatrix}$$

The solution of the above sum is expected to be:

Solution vector = $[1.96203376 \quad 5.35884552 \quad 9.74942138 \quad -0.12373404]$

Here again we only show the evolution of single node and terminating values of all nodes.

Setup



Figure 8.13: Picture of the setup with Raspberry Pi

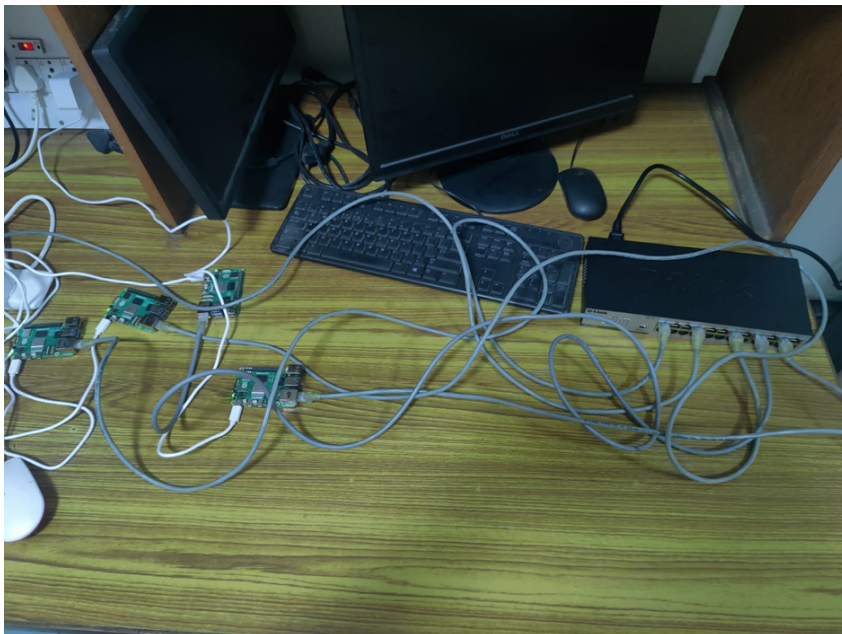


Figure 8.14: Picture with LAN connections



Figure 8.15: Picture of the whole setup

Expected results

From single file:

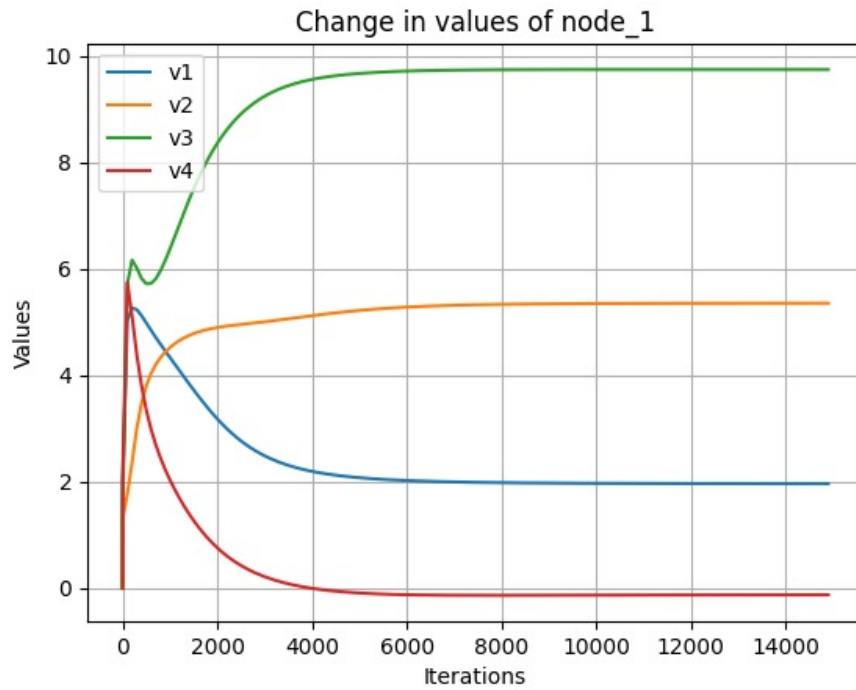


Figure 8.16: Convergence of x in Node 1

Node 1 has values: [1.96360352 5.35766242 9.74991726 -0.12525577]

Node 2 has values: [1.96363829 5.3578499 9.75016796 -0.125040221]

Node 3 has values: [1.9636064 5.35762623 9.7498723 -0.125288521]

Node 4 has values: [1.96357594 5.35768856 9.74993353 -0.1252484]

Actual Values from Raspberry Pi through LAN for 0.005s sleep time

Node 1 has values: [1.96359349 5.35769329 9.7499551 -0.1253137]

Node 2 has values: [1.96362804 5.35782808 9.75200948 -0.12508959]

Node 3 has values: [1.96359654 5.35765674 9.74991107 -0.12534635]

Node 4 has values: [1.9635652 5.35772062 9.7499732 -0.12539651]

Time taken for the above to converge is 75 seconds.

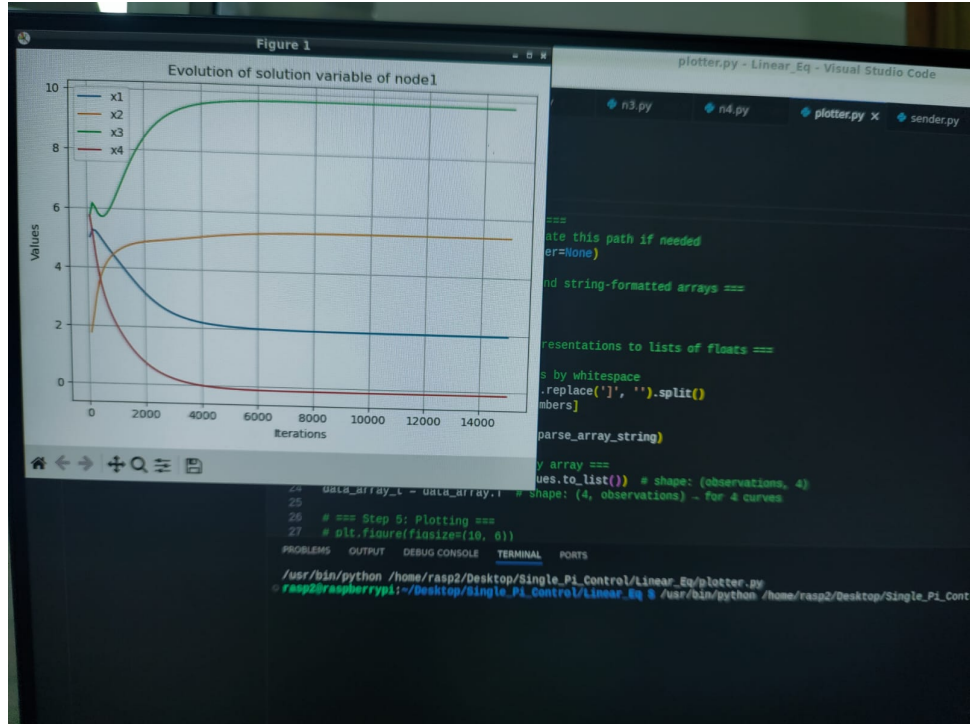


Figure 8.17: Convergence of x in Node 1

With a different example:

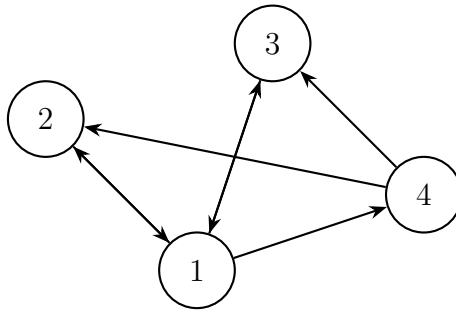


Figure 8.18: Different 4 Node graph used

The above graph is an example that has been used in the simulation. The values of matrix A and b that has been used is given below. In this case we gave single row of a set of five equations to 3 nodes and the other node had 2 rows. We then observed the evolution of solution.

$$\begin{aligned}
A_1 &= \begin{bmatrix} 1 & 1 & 1 & 1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} & A_2 &= \begin{bmatrix} 0 & 0 & 0 & 0 \\ 2 & -1 & 3 & -1 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} & A_3 &= \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 1 & 2 & -1 & 2 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} & A_4 &= \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 3 & -1 & 1 & 2 \\ 2 & 1 & 2 & -3 \end{bmatrix} \\
b_1 &= \begin{bmatrix} 10 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} & b_2 &= \begin{bmatrix} 0 \\ 12 \\ 0 \\ 0 \\ 0 \end{bmatrix} & b_3 &= \begin{bmatrix} 0 \\ 0 \\ 5 \\ 0 \\ 0 \end{bmatrix} & b_4 &= \begin{bmatrix} 0 \\ 0 \\ 0 \\ 15 \\ 7 \end{bmatrix} \\
\text{Sum}(A_i) &= \begin{bmatrix} 1 & 1 & 1 & 1 \\ 2 & -1 & 3 & -1 \\ 1 & 2 & -1 & 2 \\ 3 & -1 & 1 & 2 \\ 2 & 1 & 2 & -3 \end{bmatrix} & \text{Sum}(b_i) &= \begin{bmatrix} 10 \\ 12 \\ 5 \\ 15 \\ 7 \end{bmatrix}
\end{aligned}$$

The solution of the above sum is expected to be:

$$\text{Solution vector} = [2.62343159 \quad 1.0521542 \quad 3.53575208 \quad 2.19002268]$$

Expected Solutions and plots in each node from multi file:

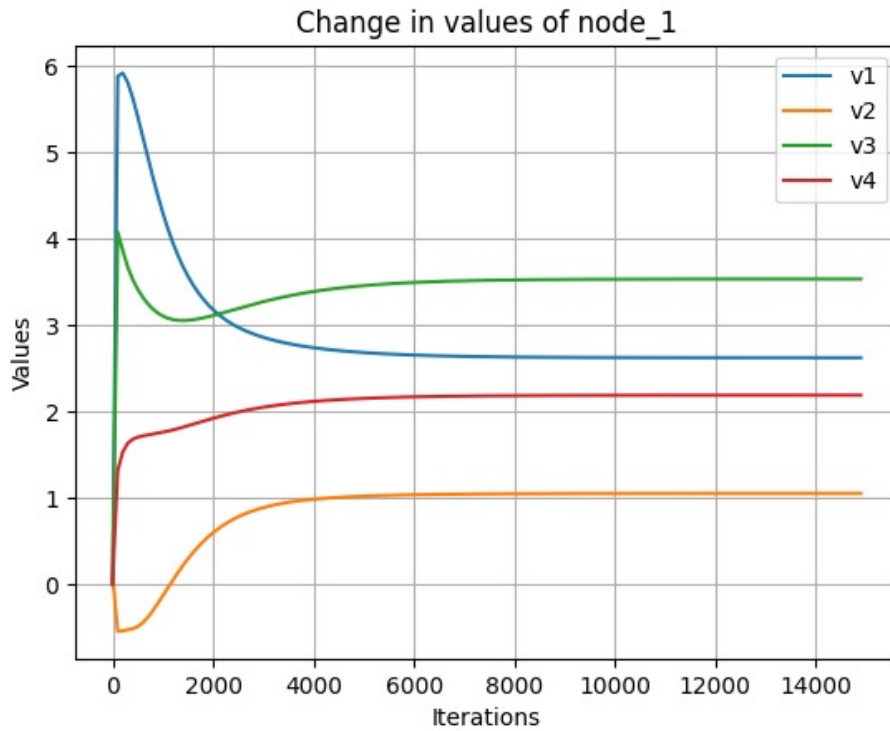


Figure 8.19: Convergence of x in Node 1

Node 1 has values: [2.62347298 1.05212885 3.53571908 2.19000136]
Node 2 has values: [2.62347143 1.05215565 3.53569189 2.19002865]
Node 3 has values: [2.62347268 1.05212468 3.53572298 2.18999711]
Node 4 has values: [2.62347434 1.05212904 3.53571995 2.19000152]

Actual values from RaspberryPi for Sleeptime of 0.005s

Node 1 has values: [2.62358092 1.05205799 3.53557795 2.18993041]
Node 2 has values: [2.62357902 1.0521204 3.53555127 2.18996209]
Node 3 has values: [2.62358097 1.05207178 3.53558162 2.18993177]
Node 4 has values: [2.6235862 1.05207611 3.53557861 2.18993615]

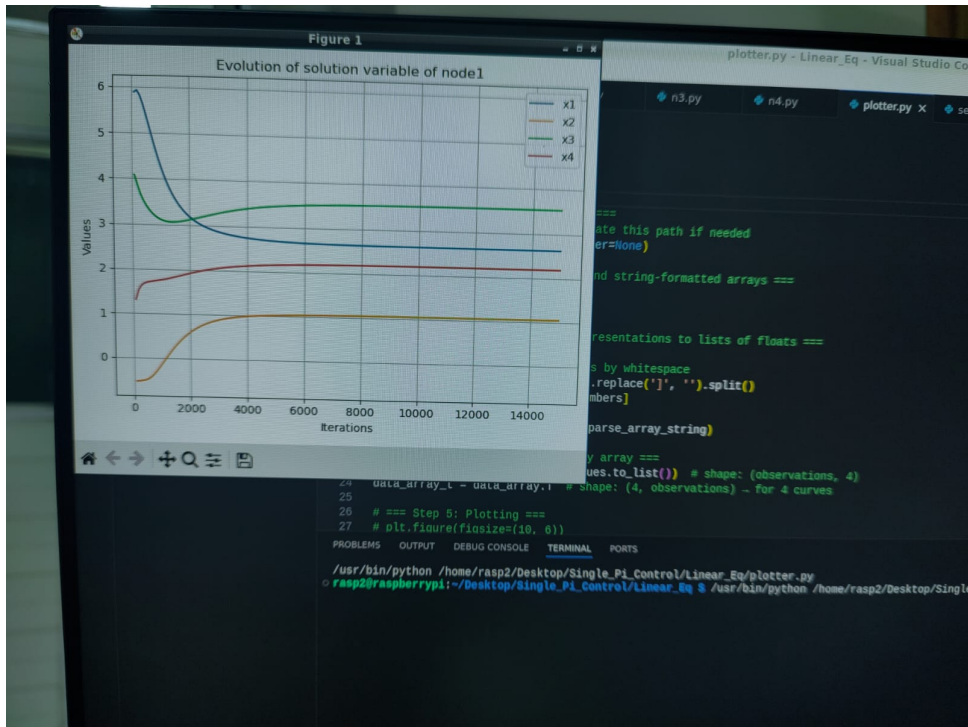


Figure 8.20: Convergence of x in Node 1

We see similar observations for other nodes. In the above Raspberry Pi Implementation only the successful and precise results have been included for clarity. The cases with errors were not included. But it should be understood that when we decrease sleep time the error also increases in this case.

8.3 Raspberry Pi Implementation in WiFi (Using Internet)

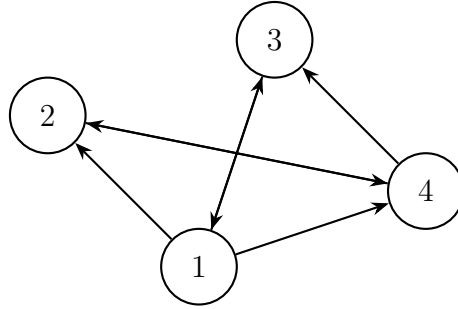


Figure 8.21: Normal 4 Node graph used

$$\text{Sum}(A_i) = \begin{bmatrix} 7 & 4 & 8 & 5 \\ 7 & 3 & 7 & 8 \\ 5 & 4 & 8 & 8 \\ 3 & 6 & 5 & 2 \\ 8 & 6 & 2 & 5 \end{bmatrix} \quad \text{Sum}(b_i) = \begin{bmatrix} 113 \\ 96 \\ 109 \\ 86 \\ 67 \end{bmatrix}$$

The solution of the above sum is expected to be:

$$\text{Solution vector} = [1.96203376 \quad 5.35884552 \quad 9.74942138 \quad -0.12373404]$$

Here again we only show the evolution of single node and terminating values of all nodes.

Setup



Figure 8.22: Picture of the setup with Raspberry Pi

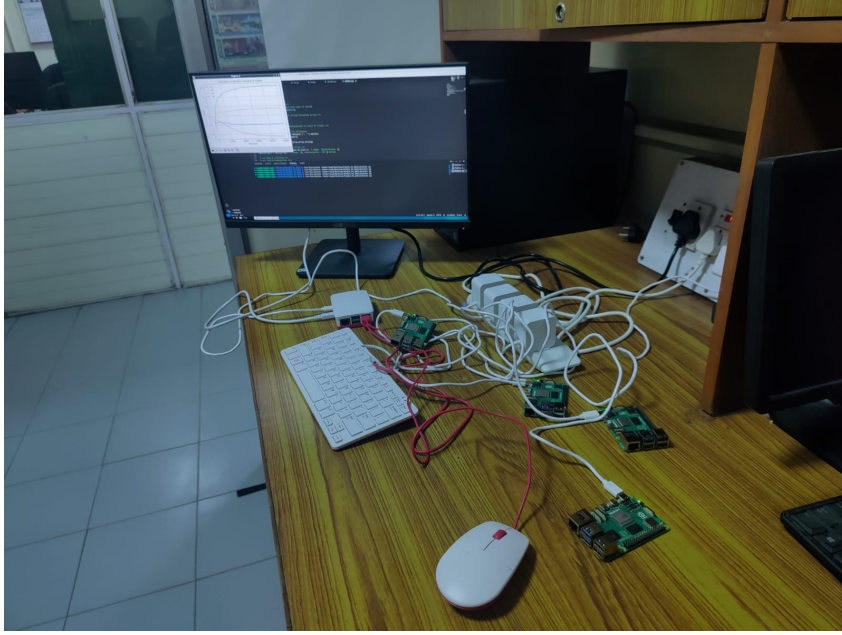


Figure 8.23: Picture of the whole setup

Expected results

Node 1 has values: [[2.2275922 4.79009895 8.62238256 1.37070877]]
Node 2 has values: [[2.22562786 4.78719627 8.61654582 1.36302062]]
Node 3 has values: [[2.20213765 4.77629655 8.60596668 1.34349993]]
Node 4 has values: [[2.25007796 4.81344602 8.65171299 1.41499227]]

Actual Values from Raspberry Pi through WiFi for 0.1s sleep time

Node 1 has values: [[2.18122629 4.85718809 8.46331526 1.54852924]]
Node 2 has values: [[2.17971796 4.84872869 8.45506004 1.54227579]]
Node 3 has values: [[2.15566744 4.83737332 8.44482554 1.52336476]]
Node 4 has values: [[2.20396098 4.87275894 8.49580690 1.59220044]]

Time taken for the above to converge is 1000 seconds.

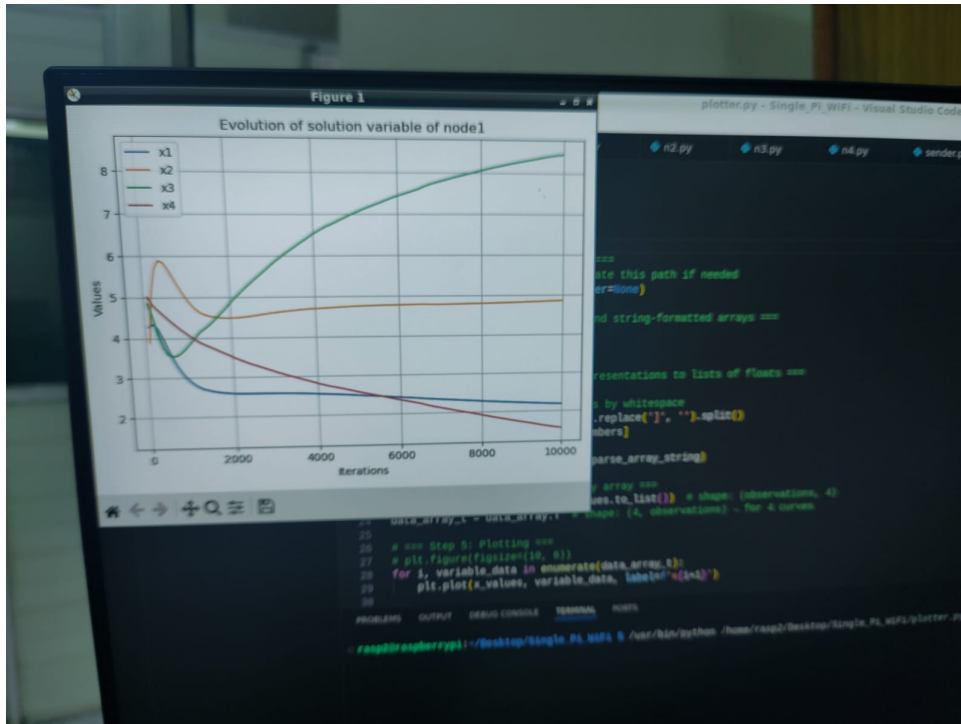


Figure 8.24: Convergence of x in Node 1

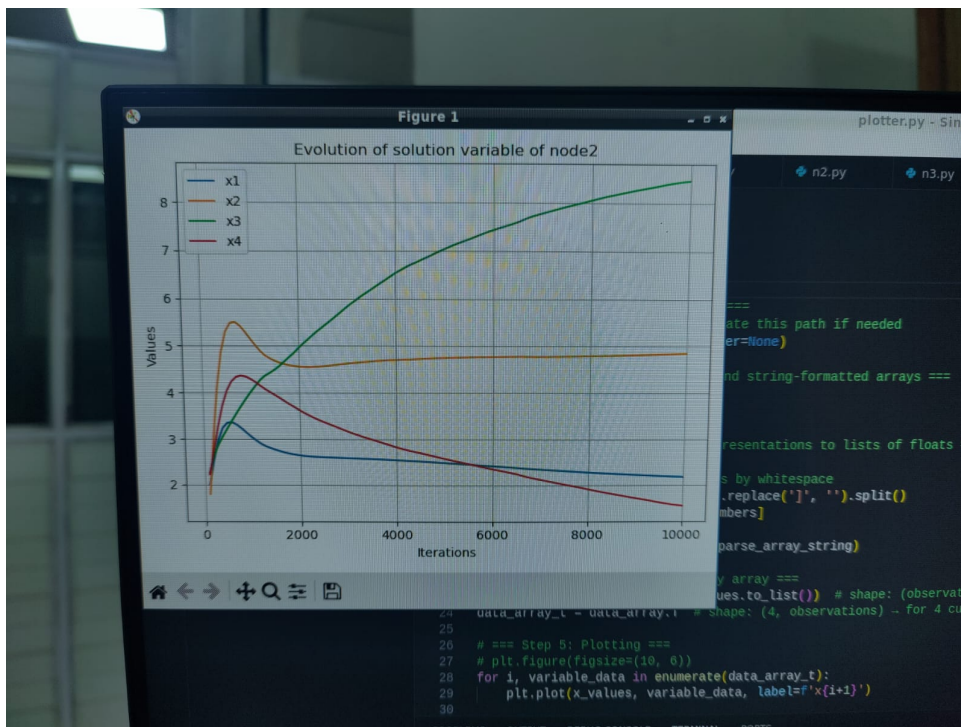


Figure 8.25: Convergence of x in Node 2

In the above observation of values we see that there has been considerable error in the values after running for 10000 iterations, this is a result of slow data transfer between nodes in WiFi scenario. Hence, we can increase sleep time even further to get better results. But in our case, our Raspberry Pi had the overheating issues and hence we could

not run iterations for more than few minutes. Hence the above observation is the only proper evolution that was observed despite the problem.

Inferences

It was observed that the multi-file implementation and Raspberry Pi implementation has similar results as the LAN connectivity works similar to the same device connection.

Chapter 9

Observations of Plots for Time varying A , b and Graph structure

Till now we have been only been discussing about cases where we have fixed A matrices and b vectors. But in this chapter we will be dealing about cases where we have varying A matrices and b vectors and its observations. The varying case tells us a very realistic scenario of practical applications as in sensor fusion, the sensor values tend to change continuously.

Observation of results of Algorithm in single file

The results in this and the upcoming sections will mostly be discussed based on the plots that will be observed. No data about the graph structure or the parameters will be mentioned as the evolution of solution due to the change in parameters will be depicted in the same graph. The changes that we bring in A and b are simple. At every iterations we just do the following changes, $A_{new} = A + \eta A$ where η is a fraction of change that is brought in each iterations step. More is η faster will the solutions change.

By the above definition of change in A and b , we bring in two types of changes, Continuous change and discrete change. Continuous changes occur every iteration uniformly giving smooth evolution of solutions. Whereas in discrete changes the solutions suddenly jump from one value to other in steps.

Observations for Continuous changes in A and b

Behaviour of Algorithm without any change

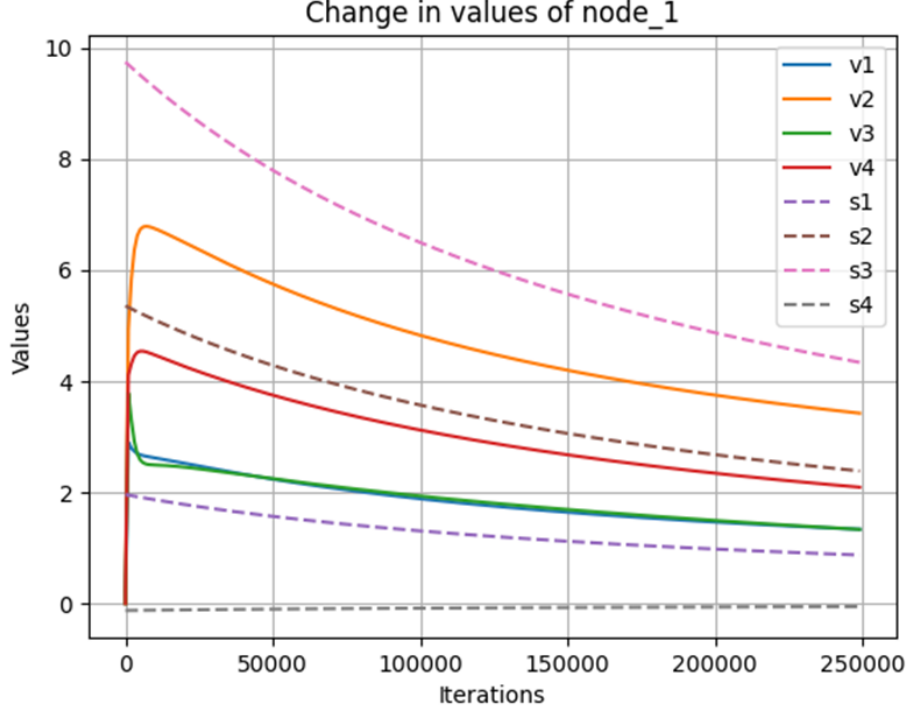


Figure 9.1: Convergence of x in Node 3

In Fig 9.1 the dotted lines shows how the solutions are evolving for the changes in A and b, whereas the thick lines gives us the results of our algorithm. We can clearly see that without any change in the algorithm and using the changing A and b in every step our algorithm approaches wrong values. Now what bringing in some thoughts on the initiation values of parameters, it is visible that the values of $\mathbf{x}$, λ , $\mathbf{v}$ and $\mathbf{w}$ is not responsible for the wrong evolution of the solutions. Hence it is clearly understandable that the variable $\mathbf{y}$ is responsible for the incorrectness. Hence, for a change we do the update $\mathbf{y} = \mathbf{Ax} - \mathbf{b}$ every few iterations to makes sure our y captures the new error.

Behaviour of Algorithm with change $y=Ax-b$

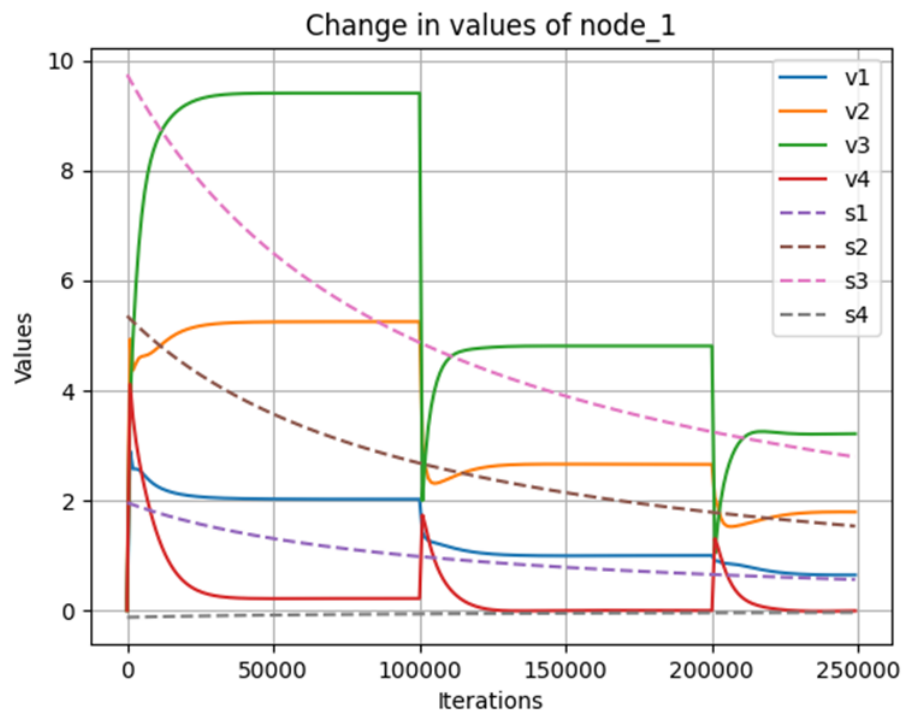


Figure 9.2: Change every 100000 iteration

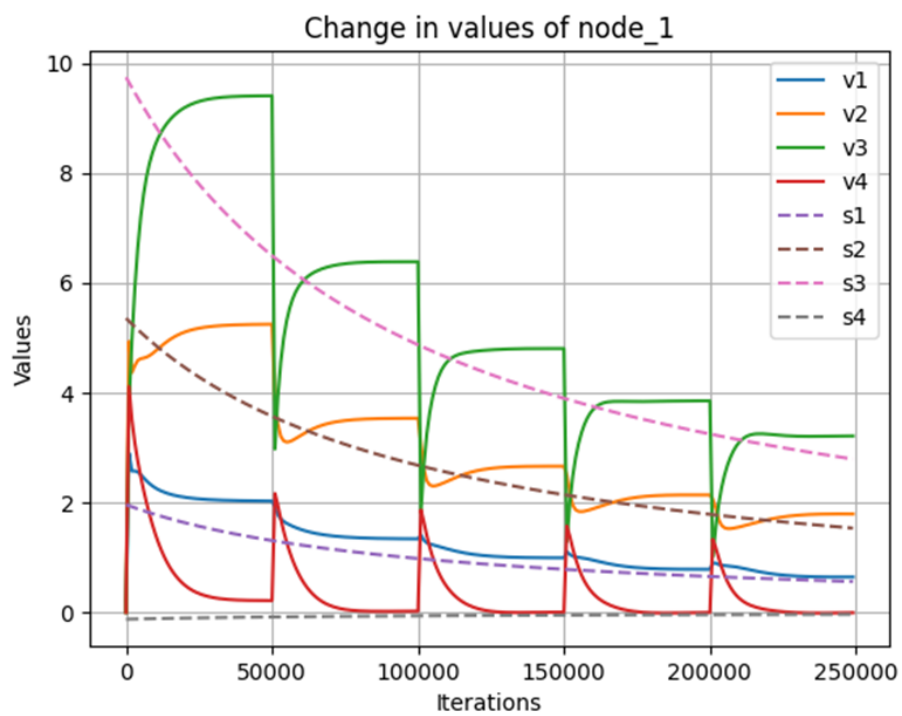


Figure 9.3: Change every 50000 iteration

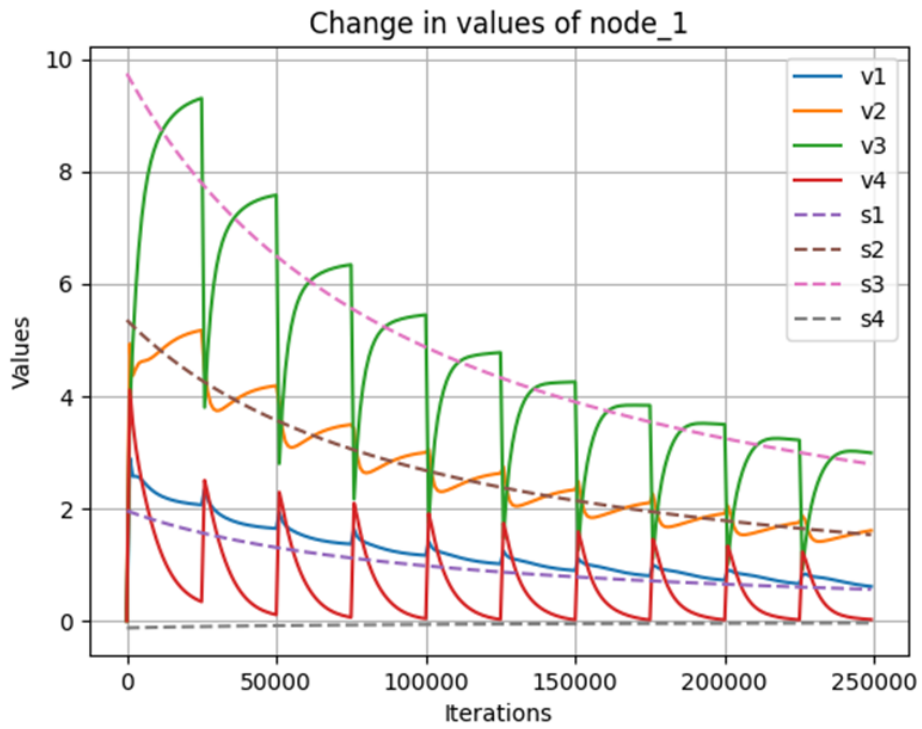


Figure 9.4: Change every 25000 iteration

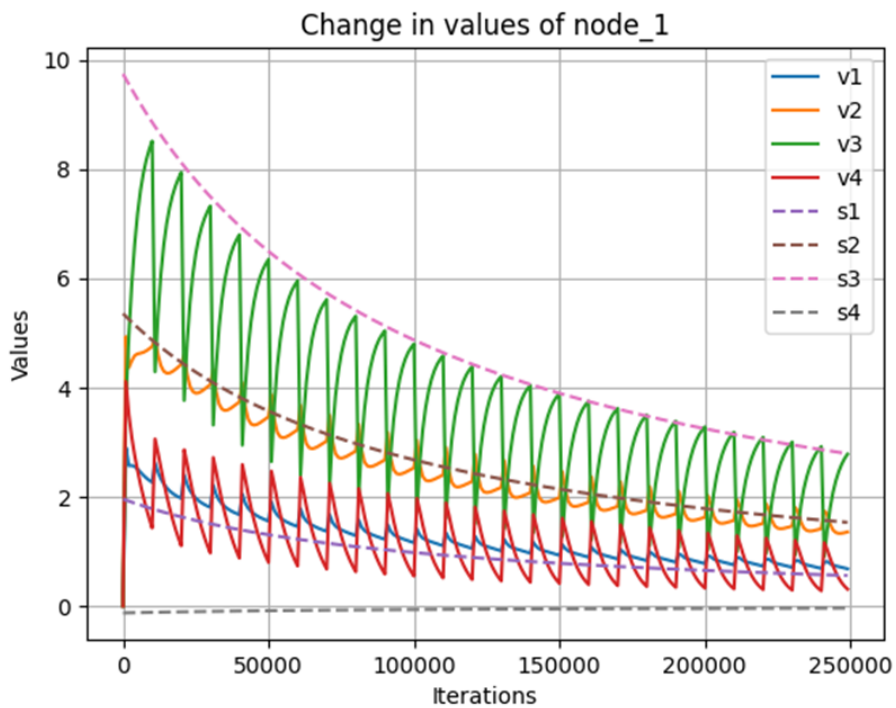


Figure 9.5: Change every 10000 iteration

In the above evolutions now we are able to observe that when we enforce the change $y = Ax - b$, our algorithm recognizes the change and resets and moves to the new x values. But

since our values are evolving faster and since it takes time for our values to converge after the change, our algorithm is not able to keep up to the change and hence just gives us oscillating results. In Fig 9.2 we can see that after the change at 100000th iteration our values converge to the solution at 100000 perfectly at around 150000 iterations. But by this time our solutions moved even further. When increase the frequency of the changes subsequently we see that in Fig 9.5 our solutions just oscillate without settling as we are not giving it enough time to converge after every change.

Now we try to make the changes discrete and try to observe convergence. By discrete we make the changes only after a certain iterations and observe the change in results.

Observations for Discrete changes in A and b

In the below cases we enforce $y = Ax - b$ whenever we detect a change.

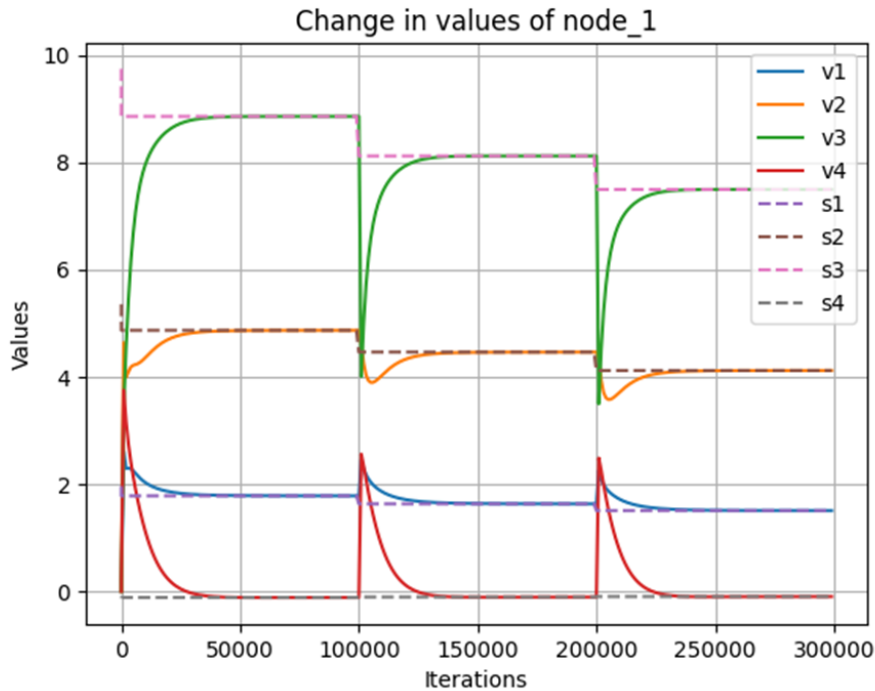


Figure 9.6: Change every 100000 iteration for high η

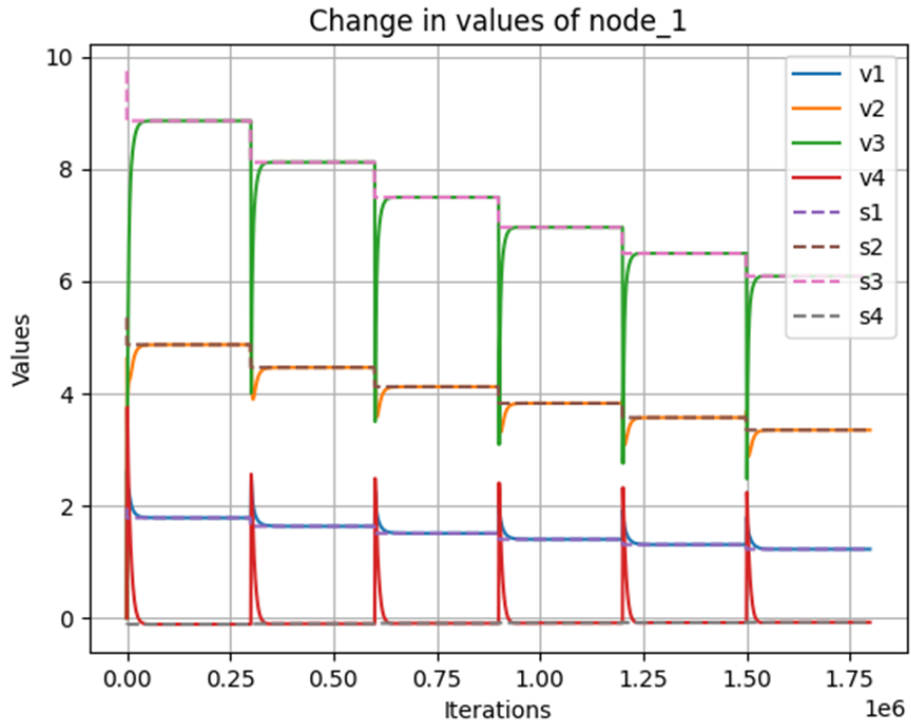


Figure 9.7: Change every 300000 iteration for low η

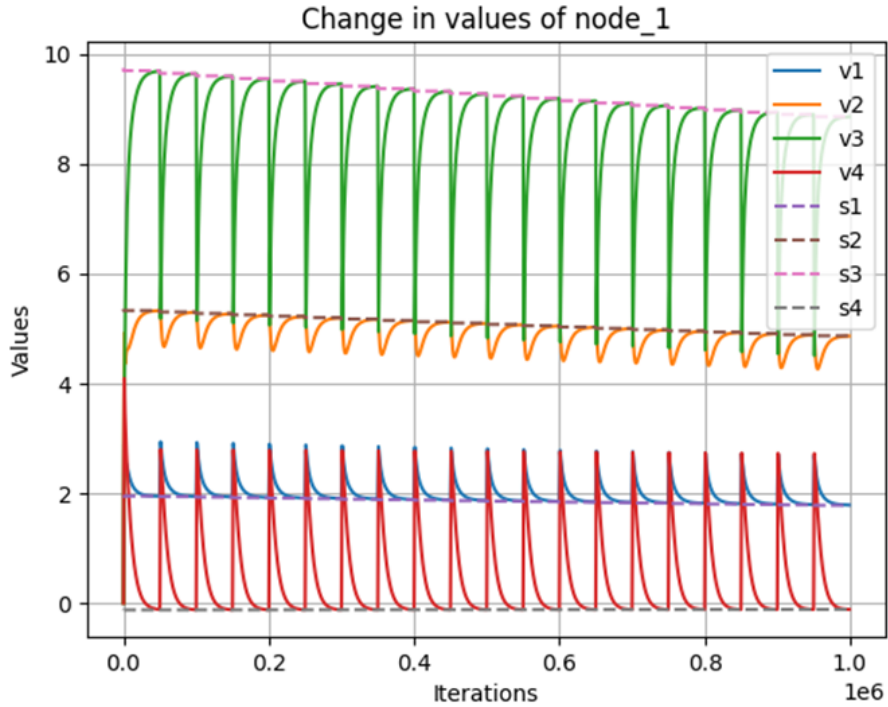


Figure 9.8: Change every 50000 iteration for low η

We now again see similar observation where our solutions converge when we give it enough time. Increasing the frequency of the change again gives us the same observation

of oscillating curves. Hence it is very clear that our algorithm does not work well when there are changes detected in system.

From the algorithm it is observed that the variable $\mathbf{y}$ (term responsible to sum all the errors of the node) is responsible for the convergence issues. Hence, we now try to remove the variable $\mathbf{y}$ and try to run the algorithm with the undistributed equations and observe the results.

Results after removing the variable $\mathbf{y}$

$$\dot{\mathbf{x}} = -2\beta\mathbf{A}^T\mathbb{1}\mathbb{1}^T(\mathbf{Ax} - \mathbf{b}) - \alpha\mathbf{VLx} - \mathbf{VL}\lambda \quad (9.1a)$$

$$\dot{\lambda} = -\mathbf{VLx} \quad (9.1b)$$

$$\dot{\mathbf{v}} = -\mathbf{Lv} \quad (9.1c)$$

$$(9.1d)$$

The above equations were used for testing. The variables $\mathbf{w}$ and $\mathbf{y}$ were removed due to the issues for convergence. The variable $\mathbf{w}$ was solely introduced to deal with the dimensional difference between $\mathbf{y}$ and $\mathbf{x}$. Since the variable $\mathbf{y}$ has been removed $\mathbf{w}$ also has been removed.

Observations

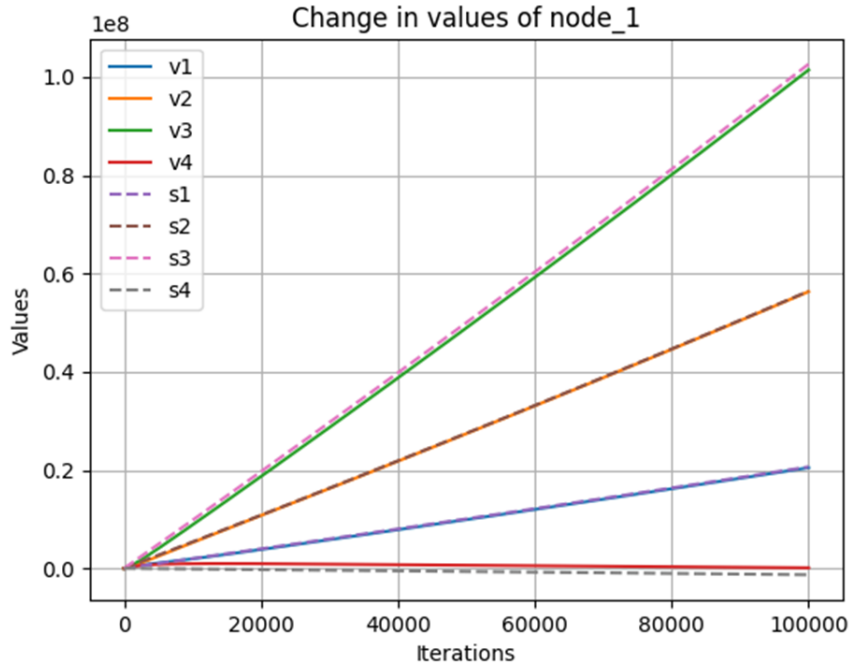


Figure 9.9: Plot for changes without $\mathbf{y}$

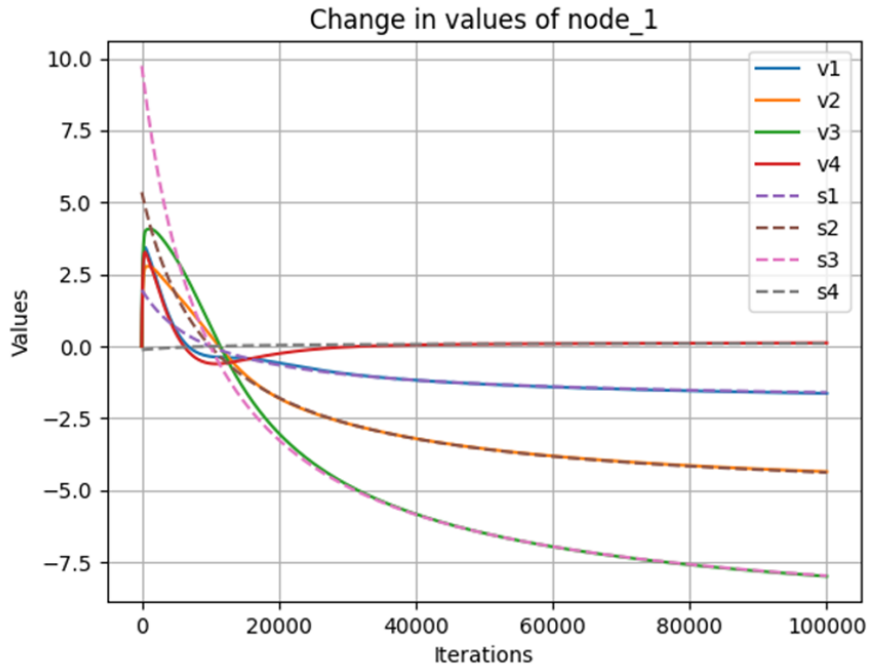


Figure 9.10: Plot for changes without y

We see that our algorithm clearly follows the solution after removing y. This is because now we do not have any error variable that needs to be tracked and share the information. Since the dynamic average consensus tracking does not occur the convergence is much faster. Hence but we can see that the algorithm is clearly not distributed. Hence one has to come up with new methods to make it distributed instead of the dynamic average consensus for measuring errors.

Results for dynamic graphs (with y)

In this section we will only see cases where there is a sudden change in graph network. We let all other parameters remain static.

The changes in graph structure induced are:

Old graph:

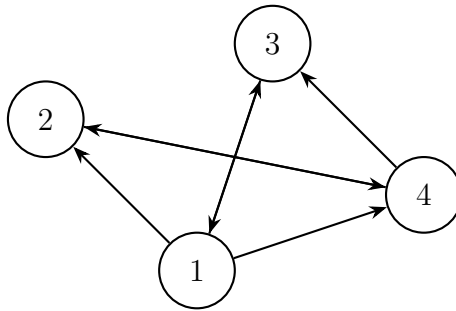


Figure 9.11: Old 4 Node graph used

New graph:

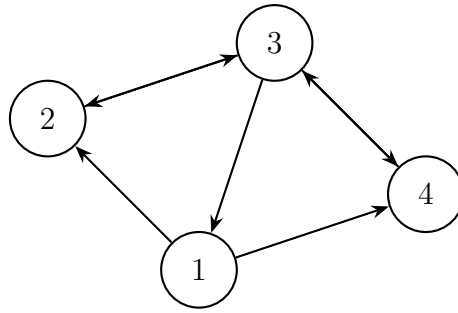


Figure 9.12: New 4 Node graph used

We then bring changes and observe the plots.

Changes induced at 30000th iteration

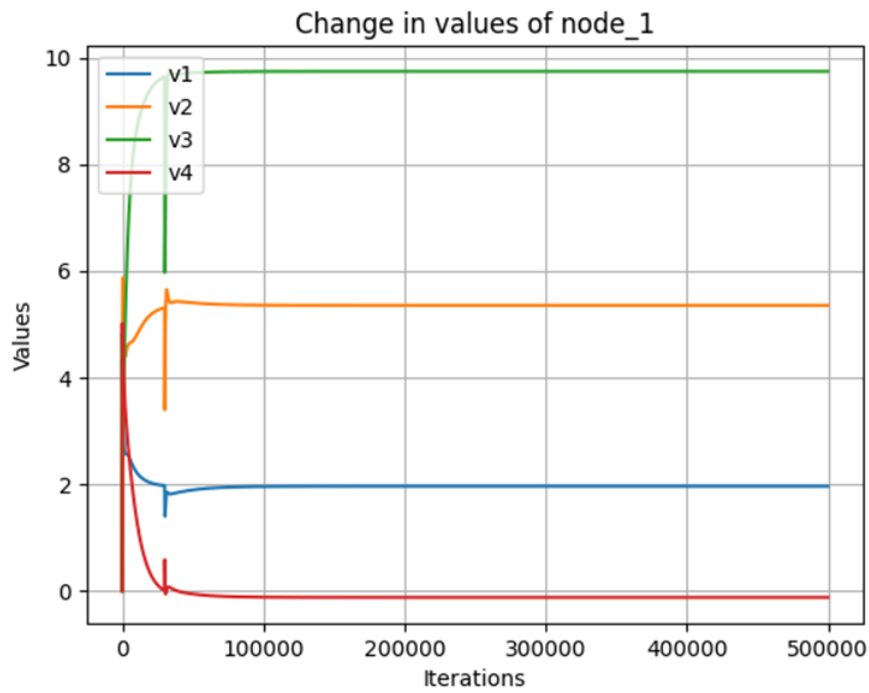


Figure 9.13: Plot for changes without y

Changes induced at 300000th iteration

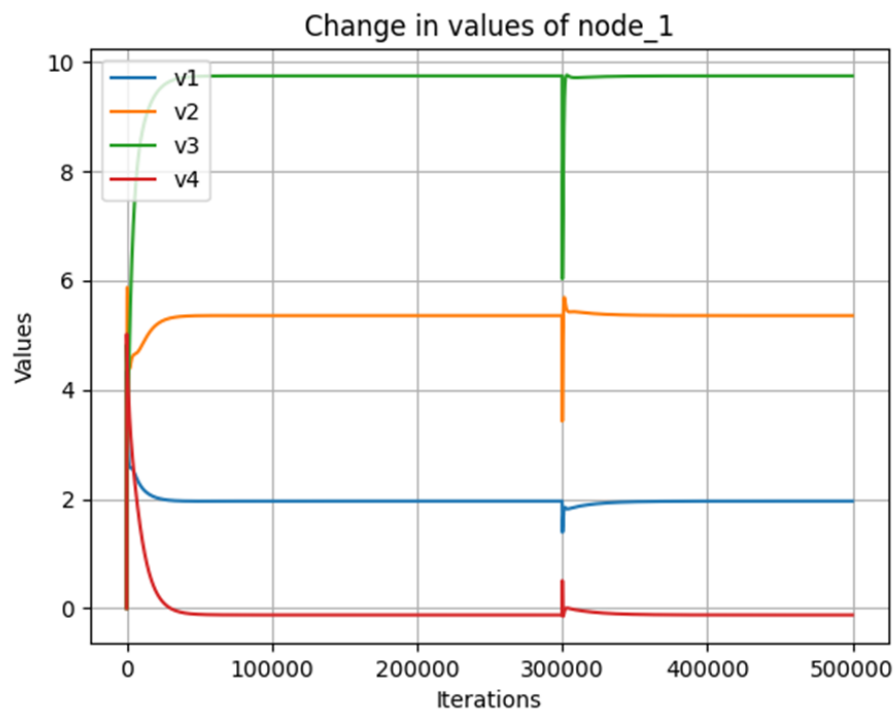


Figure 9.14: Plot for changes without y

Changes induced at 3000th iteration

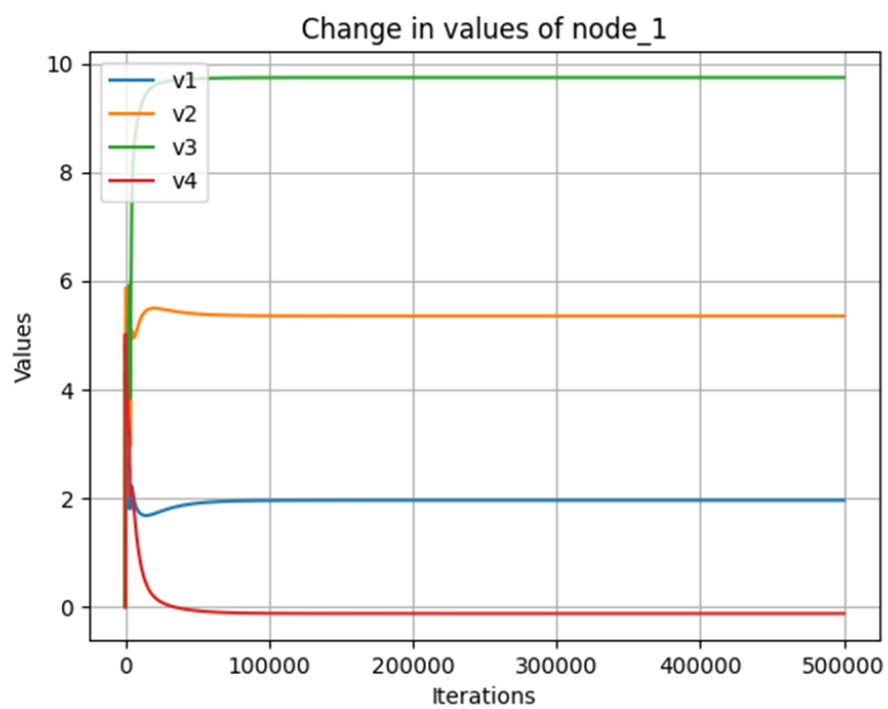


Figure 9.15: Plot for changes without y

We see from the above observations that our results are converging perfectly without fault in some thousands of iterations which is very quick. Hence, our algorithm is very compatible with dynamic graphs due to the presence of the two variables $\mathbf{v}$ and $\mathbf{w}$. The presence of the variables is the reason why our algorithm is able to work on an unbalanced distributed network by directed weight. So, the same variables in turn adjust themselves in case of changes in our old graph network making our algorithm work very well for dynamic graph network.

Another Example with 7 Nodes

Normal case without any change in graph

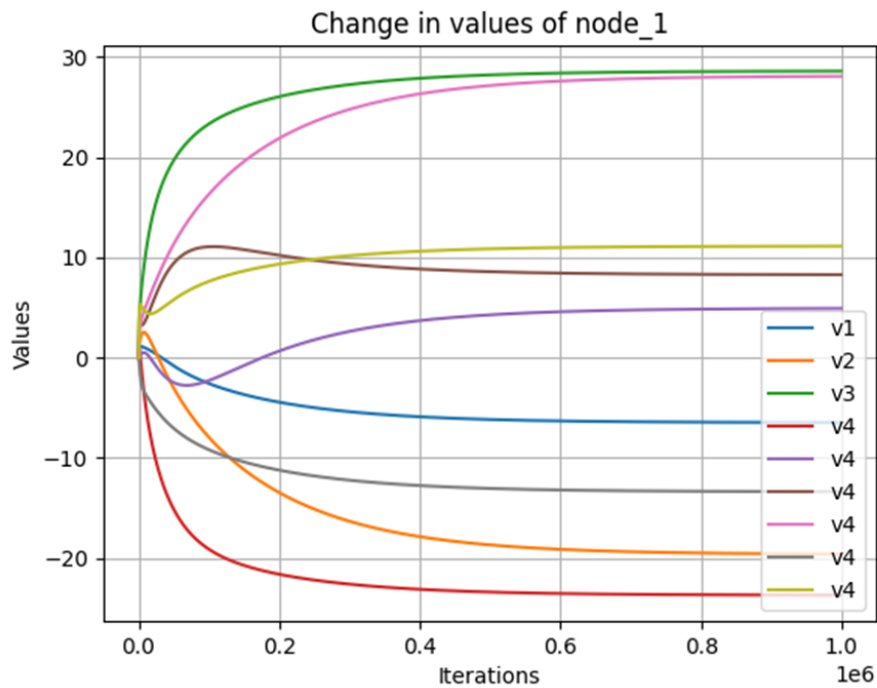


Figure 9.16: Plot for normal convergence without any change in the 7 Node system

Case with the change in graph at 1000000th iteration

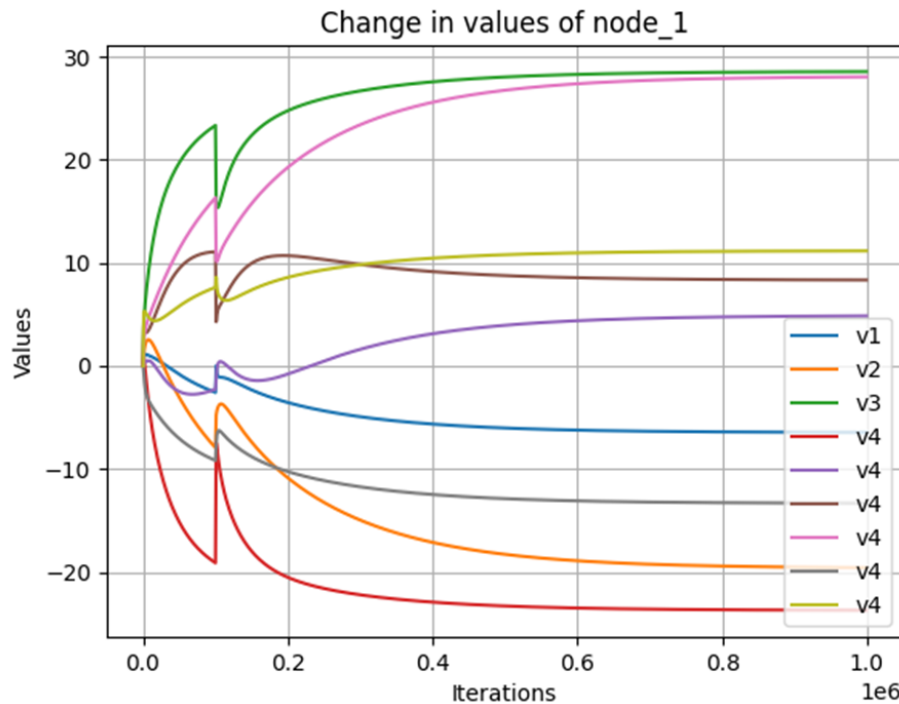


Figure 9.17: Plot for change in graph network in a 7 Node system

We can clearly see that our algorithm can accommodate any change in graph network and converge to the required solution at almost the same speed as in the normal case. The change in the graph network only costs a few iterations to recover, unlike the change in A and b parameters.

Observations from this chapter

- Our algorithm struggles and takes too much time to accommodate changes in the A matrix and b vector
- Detecting the changes and applying $y = Ax - b$ every time is a tedious process. This process also makes our system oscillate rigorously if the frequency of changes is huge.
- Our algorithm is very adaptive towards change in graph network. It adjusts the values of itself quickly to accommodate to the new changes and recovers really quickly.

Conclusions

The problem of solving linear equations arises in numerous engineering and scientific domains, such as control systems, optimization, data analysis, and distributed computing. With the increasing need for scalability, resilience, and privacy-preserving computation, traditional centralized solvers are often inadequate—especially when data is inherently distributed across multiple nodes or agents. This report addressed this challenge by presenting a comprehensive study and implementation of distributed algorithms to obtain least square solutions for systems of linear equations in various types of network topologies.

We began by exploring the mathematical basis for the least squares problem and outlined how it can be reformulated to suit distributed settings. We then developed consensus-based iterative algorithms that allow each node in a network to collaboratively compute the solution using only local communication with neighboring nodes. The algorithms were carefully extended across three progressively complex communication graph structures: undirected, directed weight-balanced, and directed weight-unbalanced graphs. In each case, convergence behavior, computational stability, and scalability were taken into account to ensure robust performance.

One of the significant contributions of this work is the formulation of a general framework that uses primal-dual gradient methods in combination with consensus protocols. The introduction of auxiliary variables such as dual multipliers and scaling factors allowed the algorithm to handle inconsistencies in the equation set and enabled it to operate efficiently under unbalanced communication constraints. Furthermore, the algorithms were implemented and tested on multi-node systems simulating both ideal and real-world networking conditions, including delay, packet loss, and asynchronous updates.

Simulation results across various graph topologies demonstrated consistent convergence to the least square solution, confirming the correctness and effectiveness of the proposed algorithms. The real-time implementation using Python and socket programming allowed for message passing and state synchronization, while visual outputs showcased the evolution of states at each node. These results validate the algorithm’s practical applicability in distributed systems where centralized coordination is infeasible or undesirable.

Moreover, this work underlines the importance of designing algorithms that do not require full knowledge of the global system, which is a critical requirement in privacy-sensitive applications such as distributed sensing, federated learning, and decentralized control. By relying on local computations and neighborhood communication, the solution paradigm aligns well with the architectural demands of modern decentralized systems.

Future Works

While this project has laid down a solid foundation, there are several directions for future research and enhancement:

- Theoretically proving the consistency, equilibrium and convergence of the above algorithm.
- Extending the algorithms to support time varying A matrix and b vector.
- Introducing fault-tolerant mechanisms to handle node failures and communication delays more robustly.
- Applying the distributed framework to real-world datasets and applications such as energy grid optimization, collaborative filtering, and multi-robot coordination.
- Exploring accelerated gradient methods and preconditioning techniques to improve convergence speed.
- Integrating privacy-preserving techniques such as differential privacy into the consensus algorithms.

In conclusion, this work demonstrates the feasibility and value of distributed least square solvers in networked environments. By building scalable and communication-efficient algorithms, we pave the way for advanced distributed computation frameworks that are well-suited to tackle future challenges in large-scale, data-driven systems.

Appendix

Single-File implementation Code:

Code Snippet for Single File Implementation in Undirected Graphs

```
1 import numpy as np
2 from scipy.linalg import block_diag
3 import matplotlib.pyplot as plt
4
5
6 # Adjacency matrix of the graph network
7 Adj = np.array([
8     [0, 1, 0, 0],
9     [0, 0, 1, 0],
10    [0, 0, 0, 1],
11    [1, 0, 0, 0]
12 ])
13 n = len(Adj)
14
15 # Indegree matrix of the graph network
16 Din = np.array([
17     [1, 0, 0, 0],
18     [0, 1, 0, 0],
19     [0, 0, 1, 0],
20     [0, 0, 0, 1]
21 ])
22
23 # Identity matrices
24 I1 = [
25     [1, 0, 0, 0],
26     [0, 1, 0, 0],
27     [0, 0, 1, 0],
28     [0, 0, 0, 1]
29 ]
30
31 I2 = [
32     [1, 0, 0, 0, 0],
33     [0, 1, 0, 0, 0],
34     [0, 0, 1, 0, 0],
35     [0, 0, 0, 1, 0],
36     [0, 0, 0, 0, 1]
37 ]
38
39 # Matrix A and vector b for each node
40 A1 = np.array([[5, 0, 2, 3], [5, 2, 2, 6], [3, 4, 5, 8], [0, 5, 4, 2],
41               [8, 2, 2, 0]])
```

```

41 A2 = np.array([[6, 1, 6, 4], [4, 0, 4, 2], [1, 0, 3, 6], [0, 0, 0, 1],
42               [4, 1, 2, 3]])
43 A3 = np.array([[2, 1, 0, 3], [6, 2, 3, 5], [1, 4, 6, 0], [2, 3, 2, 2],
44               [0, 3, 1, 0]])
45 A4 = np.array([[ -6, 2, 0, -5], [-8, -1, -2, -5], [0, -4, -6, -6], [1,
46               -2, -1, -3], [-4, 0, -3, 2]])
47
48 b1 = np.array([[61], [51], [1], [55], [1]])
49 b2 = np.array([[112], [11], [2], [80], [1]])
50 b3 = np.array([[57], [38], [100], [58], [53]])
51 b4 = np.array([[ -117], [-4], [6], [-107], [12]])
52
53 # Compute Asum and bsum
54 Asum = A1 + A2 + A3 + A4
55 bsum = b1 + b2 + b3 + b4
56 print("Sum(Ai):")
57 print(Asum)
58 print("Sum(bi):")
59 print(bsum)
60
61 # Laplacian computation
62 L = Din - Adj
63 LL = np.kron(L, I1)
64 LL1 = np.kron(L, I2)
65
66 # Initialize solution vectors
67 x = np.array([[0]] * (n * len(A1[0])))
68 ld = x
69
70 # Block diagonal matrix A and concatenated vector b
71 A = block_diag(A1, A2, A3, A4)
72 b = np.concatenate([b1, b2, b3, b4])
73 y = -1 * b
74
75 # Step sizes
76 alpha, beta, gamma = 0.02, 0.02, 0.1
77 delta1, delta2, delta3 = 0.05, 0.05, 0.05
78 iterations = 100000
79 time=[0]
80 # Iterative updates
81 xvals = [x.tolist()]
82 for i in range(iterations):
83     x_change = (-1 * alpha) * np.dot(LL, ld) - (n * beta) *
84                 np.dot(A.T, y)
85     y_change = (-1 * alpha) * np.dot(A, np.dot(LL, ld)) - (n * beta) *
86                 np.dot(np.dot(A, A.T), y) - (gamma) * (np.dot(LL1, y))
87     l_change = np.dot(LL.T, x)
88     x = x + delta1 * x_change
89     y = y + delta2 * y_change
90     ld = ld + delta3 * l_change
91     time += [i+1]
92     xvals += [x.tolist()]
93
94 x1 = []
95 x2 = []
96 x3 = []
97 x4 = []
98 j=0

```

```

94
95 for i in range(0, iterations+1):
96     x1+=xvals[i][j]
97     x2+=xvals[i][j+1]
98     x3+=xvals[i][j+2]
99     x4+=xvals[i][j+3]
100
101 print("-----")
102
103 #print(time, x1)
104
105 plt.plot(time, x1, label=f'v1')
106 plt.plot(time, x2, label=f'v2')
107 plt.plot(time, x3, label=f'v3')
108 plt.plot(time, x4, label=f'v4')
109
110 plt.xlabel('Time')
111 plt.ylabel('Values')
112 plt.title(f'Change in values of node_{int((j+4)/4)}')
113 plt.legend()
114 plt.grid(True)
115 plt.show()
116
117 print(x.T, "X \n")
118 print(y.T, "Y \n")
119 print(ld.T, "Ld \n")

```

Code Snippets for the Single File Implementation in Directed Weight-Balanced Graphs

```

1 import numpy as np
2 from scipy.linalg import block_diag
3 import matplotlib.pyplot as plt
4 import time
5
6 st = time.time()
7
8 # Adjacency matrix of the graph network
9 Adj = np.array([
10     [0, 0, 0, 1],
11     [0, 0, 1, 0],
12     [1, 1, 0, 0],
13     [0, 0, 1, 0]
14 ])
15 Adj = Adj.T
16
17 n = len(Adj)
18
19 # Indegree matrix of the graph network
20 Din = np.array([
21     [1, 0, 0, 0],
22     [0, 1, 0, 0],
23     [0, 0, 2, 0],
24     [0, 0, 0, 1]
25 ])
26

```

```

27 # Identity matrices
28 I1 = [
29     [1, 0, 0, 0],
30     [0, 1, 0, 0],
31     [0, 0, 1, 0],
32     [0, 0, 0, 1]
33 ]
34 I2 = [
35     [1, 0, 0, 0, 0],
36     [0, 1, 0, 0, 0],
37     [0, 0, 1, 0, 0],
38     [0, 0, 0, 1, 0],
39     [0, 0, 0, 0, 1]
40 ]
41
42 # Matrix A and vector b for each node
43 A1 = np.array([[5, 0, 2, 3], [5, 2, 2, 6], [3, 4, 5, 8], [0, 5, 4, 2],
44               [8, 2, 2, 0]])
45 A2 = np.array([[6, 1, 6, 4], [4, 0, 4, 2], [1, 0, 3, 6], [0, 0, 0, 1],
46               [4, 1, 2, 3]])
47 A3 = np.array([[2, 1, 0, 3], [6, 2, 3, 5], [1, 4, 6, 0], [2, 3, 2, 2],
48               [0, 3, 1, 0]])
49 A4 = np.array([[-6, 2, 0, -5], [-8, -1, -2, -5], [0, -4, -6, -6], [1,
50               -2, -1, -3], [-4, 0, -3, 2]])
51
52 b1 = np.array([[61], [51], [1], [55], [1]])
53 b2 = np.array([[112], [11], [2], [80], [1]])
54 b3 = np.array([[57], [38], [100], [58], [53]])
55 b4 = np.array([[-117], [-4], [6], [-107], [12]])
56
57 # Compute Asum and bsum
58 Asum = A1 + A2 + A3 + A4
59 bsum = b1 + b2 + b3 + b4
60 p = len(A1)
61 q = len(A1[0])
62 print("Sum(Ai):")
63 print(Asum)
64 print("Sum(bi):")
65 print(bsum)
66
67 # Laplacian computation
68 L = Din - Adj
69 print(L)
70 LL = np.kron(L, I1)
71 LL1 = np.kron(L, I2)
72
73 # Initialize solution vectors
74 x = np.array([[0]] * (n * len(A1[0])))
75 ld = x
76
77 # Block diagonal matrix A and concatenated vector b
78 A = block_diag(A1, A2, A3, A4)
79 b = np.concatenate([b1, b2, b3, b4])
80 y = -1 * b
81
82 # Step sizes
83 alpha, beta = 25, 5e-2
84 delta = 1e-3

```

```

81 iterations = 300000
82 times = [0]
83
84 oe = np.array([[1]] * n)
85 one = np.kron(oe, I2)
86
87 xvals = [x.tolist()]
88 for i in range(iterations):
89     x_change = -1 * np.dot(LL, ld) - (2 * n * beta) * np.dot(A.T, y) -
90         alpha * np.dot(LL, x)
91     l_change = np.dot(LL, x)
92     y_change = -1 * np.dot(LL1, y) + np.dot(A, x_change)
93     x = x + delta * x_change
94     ld = ld + delta * l_change
95     y = y + delta * y_change
96     times += [i + 1]
97     xvals += [x.tolist()]
98
99 x1, x2, x3, x4 = [], [], [], []
100 j = 0
101 for i in range(0, iterations + 1):
102     x1 += xvals[i][j]
103     x2 += xvals[i][j + 1]
104     x3 += xvals[i][j + 2]
105     x4 += xvals[i][j + 3]
106
107 print("-----")
108 for i in range(0, n):
109     print(f"Node {i + 1} has values:", x[i * q:i * q + q].T)
110
111 end_t = time.time()
112 print("Time taken:", end_t - st)
113
114 plt.plot(times, x1, label=f'v1')
115 plt.plot(times, x2, label=f'v2')
116 plt.plot(times, x3, label=f'v3')
117 plt.plot(times, x4, label=f'v4')
118 plt.xlabel('Iterations')
119 plt.ylabel('Values')
120 plt.title(f'Change in values of node_{int((j+4)/4)}')
121 plt.legend()
122 plt.grid(True)
123 plt.show()

```

Code snippets for Single File Implementation in Directed Weight-Unbalanced case

```

1 import numpy as np
2 from scipy.linalg import block_diag
3 import matplotlib.pyplot as plt
4 import time
5
6 st = time.time()
7
8 # Unbalanced case
9 Adj = np.array([

```

```

10     [0, 1, 1, 1],
11     [1, 0, 0, 0],
12     [0, 1, 0, 0],
13     [0, 1, 0, 0]
14 ])
15
16 Din = np.array([
17     [1, 0, 0, 0],
18     [0, 3, 0, 0],
19     [0, 0, 1, 0],
20     [0, 0, 0, 1]
21 ])
22
23 n = len(Adj)
24
25 # Identity matrices
26 I1 = [
27     [1, 0, 0, 0],
28     [0, 1, 0, 0],
29     [0, 0, 1, 0],
30     [0, 0, 0, 1]
31 ]
32
33 I2 = [
34     [1, 0, 0, 0, 0],
35     [0, 1, 0, 0, 0],
36     [0, 0, 1, 0, 0],
37     [0, 0, 0, 1, 0],
38     [0, 0, 0, 0, 1]
39 ]
40
41 # Matrix A and vector b for each node
42 A1 = np.array([[5, 0, 2, 3], [5, 2, 2, 6], [3, 4, 5, 8], [0, 5, 4, 2],
43               [8, 2, 2, 0]])
44 A2 = np.array([[6, 1, 6, 4], [4, 0, 4, 2], [1, 0, 3, 6], [0, 0, 0, 1],
45               [4, 1, 2, 3]])
46 A3 = np.array([[2, 1, 0, 3], [6, 2, 3, 5], [1, 4, 6, 0], [2, 3, 2, 2],
47               [0, 3, 1, 0]])
48 A4 = np.array([[-6, 2, 0, -5], [-8, -1, -2, -5], [0, -4, -6, -6], [1,
49               -2, -1, -3], [-4, 0, -3, 2]])
50
51 b1 = np.array([[61], [51], [1], [55], [1]])
52 b2 = np.array([[112], [11], [2], [80], [1]])
53 b3 = np.array([[57], [38], [100], [58], [53]])
54 b4 = np.array([[-117], [-4], [6], [-107], [12]])
55
56 p = len(A1)
57 q = len(A1[0])
58
59 # Compute Asum and bsum
60 Asum = A1 + A2 + A3 + A4
61 bsum = b1 + b2 + b3 + b4
62 print("Sum(Ai):")
63 print(Asum)
64 print("Sum(bi):")
65 print(bsum)
66
67 # Laplacian computation

```

```

64 L = Din - Adj
65 print(L)
66 LL = np.kron(L, I1)
67 LL1 = np.kron(L, I2)
68
69 # Initialize solution vectors
70 x = np.array([[0]] * (n * q))
71 ld = x
72 v = np.array([[5]] * (n * q))
73 v_dash = np.array([[5]] * (n * p))
74
75 # Block diagonal matrix A and concatenated vector b
76 A = block_diag(A1, A2, A3, A4)
77 b = np.concatenate([b1, b2, b3, b4])
78 y = -1 * b
79
80 # Step sizes
81 alpha, beta = 25, 5e-2
82 delta = 1e-3
83 iterations = 100000
84 times = [0]
85 cap = 100
86
87 oe = np.array([[1]] * q)
88 ol = np.array([[1]] * p)
89 one = np.kron(oe, I2)
90
91 xvals = [x.tolist()]
92 for i in range(iterations):
93     V = np.diag(v.flatten())
94     V_dash = np.diag(v_dash.flatten())
95
96     x_change = -1 * np.dot(np.dot(LL, V), ld) - (2 * n * beta) *
97         np.dot(A.T, y) - alpha * np.dot(np.dot(LL, V), x)
98     l_change = np.dot(np.dot(LL, V), x)
99     y_change = -1 * np.dot(np.dot(LL1, V_dash), y) + np.dot(A,
100         x_change)
101     v_change = -np.dot(LL, v)
102     v_dash_change = -np.dot(LL1, v_dash)
103
104     x = x + delta * x_change
105     ld = ld + delta * l_change
106     y = y + delta * y_change
107     v = v + delta * v_change
108     v_dash = v_dash + delta * v_dash_change
109
110     if i % cap == 0:
111         times += [i + 1]
112         xvals += [x.tolist()]
113
114 x1, x2, x3, x4 = [], [], [], []
115 j = 0
116 for i in range(0, iterations // cap + 1):
117     x1 += xvals[i][j]
118     x2 += xvals[i][j + 1]
119     x3 += xvals[i][j + 2]
120     x4 += xvals[i][j + 3]

```

```

120 print("-----")
121 for i in range(n):
122     print(f"Node {i + 1} has values:", x[i * q:i * q + q].T)
123
124 end_t = time.time()
125 print("Time taken:", end_t - st)
126
127 plt.plot(times, x1, label='v1')
128 plt.plot(times, x2, label='v2')
129 plt.plot(times, x3, label='v3')
130 plt.plot(times, x4, label='v4')
131 plt.xlabel('Iterations')
132 plt.ylabel('Values')
133 plt.title(f'Change in values of node_{int((j+4)/4)}')
134 plt.legend()
135 plt.grid(True)
136 plt.show()

```

Codes and Implementation for Multinode Systems

In implementation of the algorithm which we found in multiple nodes. We implement the following:

- Multi-File Implementation
- Raspberry Pi Implementation over LAN (Wired Network)
- Raspberry Pi Implementation over WiFi (Wireless Network)

The code for all the above implementation remains the same except for the usage. For all the communication purposes between different nodes we use UDP protocol. We initially bind every node to its respective port in the device and we make them listen for communication and manage data transfer. We will be trying to sync all the nodes by the use of sleep functionality of python by forcing the nodes to sleep for the necessary duration. In order to have a synced start we send a start message from a coordinator node which is responsible to send all the data regarding sleep time, neighbor data, step parameter data and so on. The code that has been used for coordinator and a node is given below for reference. We will be focusing mainly on the implementation of directed weight-unbalanced case as it's the most general case of a graph.

Code Snippets of the Multi-Node Implementation

Coordinator.py

```

1 import socket
2 import json
3 import threading
4 import time
5 import numpy as np
6
7
8 A = [

```



```

9         [[5, 0, 2, 3], [5, 2, 2, 6], [3, 4, 5, 8], [0, 5, 4, 2], [8,
10            2, 2, 0]],
11         [[6, 1, 6, 4], [4, 0, 4, 2], [1, 0, 3, 6], [0, 0, 0, 1], [4,
12            1, 2, 3]],
13         [[2, 1, 0, 3], [6, 2, 3, 5], [1, 4, 6, 0], [2, 3, 2, 2], [0,
14            3, 1, 0]],
15         [[-6, 2, 0, -5], [-8, -1, -2, -5], [0, -4, -6, -6], [1, -2,
16            -1, -3], [-4, 0, -3, 2]]
17     ]
18
19     b = [
20         [[61], [51], [1], [55], [1]],
21         [[112], [11], [2], [80], [1]],
22         [[57], [38], [100], [58], [53]],
23         [[-117], [-4], [6], [-107], [12]]
24     ]
25
26     def print_time_taken(description, start_time):
27         end_time = time.time()
28         print(f"{description} took {end_time - start_time:.6f} seconds")
29
30     def send_init_message(ip, port, node_id, iteration_number, neighbors,
31                           alpha, iter, nodes, beta, gamma, delta, gaps, initiation, indegree):
32         message = {
33             "init": "INIT",
34             "inum": iteration_number,
35             "alpha": alpha,
36             "iter": iter,
37             "neighbors": neighbors,
38             "nodes": nodes,
39             "num_nodes": num_nodes,
40             "sleep_time": sleep_time,
41             "beta": beta,
42             "gamma": gamma,
43             "delta": delta,
44             "A": A[node_id-1],
45             "b": b[node_id-1],
46             "gaps": gaps,
47             "node_id": node_id,
48             "strt": initiation,
49             "indegree": indegree
50         }
51         with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
52             s.sendto(json.dumps(message).encode('utf-8'), (ip, port))
53
54     def send_messages_to_node(ip, port, node_id, neighbors, nodes,
55                               iteration_number, alpha, iter, beta, gamma, delta, gaps,
56                               initiation, indegree):
57         send_init_message(ip, port, node_id, iteration_number, neighbors,
58                           alpha, iter, nodes, beta, gamma, delta, gaps,
59                           initiation, indegree)
60
61     if __name__ == "__main__":
62         nodes = [("127.0.0.1", 12345, 1), ("127.0.0.1", 12346, 2),
63                  ("127.0.0.1", 12347, 3), ("127.0.0.1", 12348, 4)]
64         iteration_number = 15000
65         sleep_time = 0.005
66         gaps = 100

```

```

57     initiation = 15
58     neighbors = [
59         [],
60         [2],
61         [1,3,4],
62         [1],
63         [1],
64     ]
65     neighbors = [
66         [],
67         [3],
68         [1,4],
69         [1,4],
70         [1,2]
71     ]
72     # Data transfer is an edge. (Opposite to convention)
73     siz = len(neighbors)
74     indegree = [0]*siz
75     for i in range(1,siz):
76         for j in neighbors[i]:
77             indegree[j]+=1
78     print(indegree)
79     num_nodes = 4
80     alpha = 25
81     beta = 4e-1
82     gamma = 5e-3
83     delta = 1.05e-3
84     iter = 1
85     start_time = time.time()
86     threads = []
87     for node in nodes:
88         thread = threading.Thread(target=send_messages_to_node,
89                                   args=(node[0], node[1], node[2], neighbors, nodes,
90                                         iteration_number, alpha, iter, beta, gamma, delta, gaps,
91                                         initiation, indegree))
92         threads.append(thread)
93     time.sleep(1)
94     for thread in threads:
95         thread.start()
96     for thread in threads:
97         thread.join()
98     print("Initialization messages sent to all nodes.")

```

Node1.py

```

1  import socket
2  import pandas as pd
3  import time
4  import json
5  import matplotlib.pyplot as plt
6  import csv
7  import numpy as np
8
9  u_st = 0
10 neighbors=[]

```

```

11 nodes = []
12 node_id = 1
13 num_nodes = 5
14 n = num_nodes
15 x_val_list = []
16 y_val_list = []
17 ld_val_list = []
18 v_val_list = []
19 w_val_list = []
20 beta = 0.01
21 gamma = 10
22 delta = 0.01
23 A = []
24 b = []
25 x = []
26 y = []
27 z = []
28 L = []
29 L_dash = []
30 gaps = 1000
31 strt = 10
32 indegree = []
33 p = 0
34 q = 0
35 found_L = 0
36 x_vals = np.array([])
37 y_vals = np.array([])
38 ld_vals = np.array([])
39 v_vals = np.array([])
40 w_vals = np.array([])
41 neighbor_prev_states=[]
42 sleep_time = 1
43 local_ip = "127.0.0.1"
44 port_num = 12345
45
46 def send_state_to_neighbors(ip, port, node_id, x,y,ld,v,w):
47     with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
48         x_val = x.tolist()
49         y_val = y.tolist()
50         ld_val = ld.tolist()
51         v_val = v.tolist()
52         w_val = w.tolist()
53         message = {
54             "node_id": node_id,
55             "x": x_val,
56             "y": y_val,
57             "ld": ld_val,
58             "v": v_val,
59             "w": w_val
60         }
61         json_message = json.dumps(message)
62         s.sendto(json_message.encode('utf-8'), (ip, port))
63
64 def listen_to_neighbors(s,received_from):
65     global x_vals,y_vals,ld_vals,v_vals,w_vals
66     s.settimeout(sleep_time*0.9)
67     try:
68         data, _ = s.recvfrom(65535)

```

```

69     if data:
70         message = json.loads(data.decode('utf-8'))
71         neighbor_id = int(message["node_id"])
72         x_state = np.array((message["x"]))
73         y_state = np.array((message["y"]))
74         ld_state = np.array((message["ld"]))
75         v_state = np.array((message["v"]))
76         w_state = np.array((message["w"]))
77         x_vals[(neighbor_id-1)*q:(neighbor_id-1)*q+q] = x_state
78         y_vals[(neighbor_id-1)*p:(neighbor_id-1)*p+p] = y_state
79         ld_vals[(neighbor_id-1)*q:(neighbor_id-1)*q+q] = ld_state
80         v_vals[(neighbor_id-1)*q:(neighbor_id-1)*q+q] = v_state
81         w_vals[(neighbor_id-1)*p:(neighbor_id-1)*p+p] = w_state
82         received_from[neighbor_id] = 1
83     except socket.timeout:
84         pass
85
86 def log_state(fp, node_id, iteration, state):
87     with open(fp, 'a', newline='') as file:
88         writer = csv.writer(file)
89         writer.writerow([iteration, state])
90
91 def run_consensus(s, alpha, iter, xt):
92     global ii, found_L, p, q, L, L_dash
93     global x_vals, y_vals, ld_vals, v_vals, w_vals
94     global neighbor_prev_states, sleep_time
95     global x, y, ld, v, w
96
97     received_from = [0] * (num_nodes + 1)
98
99     for _ in range(iter):
100         for neighbor in neighbors[node_id]:
101             ip = nodes[neighbor-1][0]
102             port = nodes[neighbor-1][1]
103             send_state_to_neighbors(ip, port, node_id, x, y, ld, v, w)
104
105         for _ in range(indegree[node_id]):
106             listen_to_neighbors(s, received_from)
107
108         x_update = np.zeros((q, 1))
109         y_update = np.zeros((p, 1))
110         ld_update = np.zeros((q, 1))
111         v_update = np.zeros((q, 1))
112         w_update = np.zeros((p, 1))
113
114         x_update = x_update.astype(np.float64)
115         y_update = y_update.astype(np.float64)
116         ld_update = ld_update.astype(np.float64)
117         v_update = v_update.astype(np.float64)
118         w_update = w_update.astype(np.float64)
119
120         if not found_L:
121             L = np.empty((q, 0))
122             L_dash = np.empty((p, 0))
123             for its in range(1, num_nodes + 1):
124                 if its == node_id:
125                     block_L = len(neighbors[its]) * np.eye(q)
126                     block_Ld = len(neighbors[its]) * np.eye(p)

```

```

127         elif received_from[its] == 0:
128             block_L = np.zeros((q, q))
129             block_Ld = np.zeros((p, p))
130         else:
131             block_L = -1 * np.eye(q)
132             block_Ld = -1 * np.eye(p)
133
134         L = np.hstack((L, block_L))
135         L_dash = np.hstack((L_dash, block_Ld))
136         found_L = 1
137
138         x_vals[(node_id-1)*q:(node_id-1)*q+q] = x
139         y_vals[(node_id-1)*p:(node_id-1)*p+p] = y
140         ld_vals[(node_id-1)*q:(node_id-1)*q+q] = ld
141         v_vals[(node_id-1)*q:(node_id-1)*q+q] = v
142         w_vals[(node_id-1)*p:(node_id-1)*p+p] = w
143
144         res = A @ (A.T @ y)
145         result = A.T @ y
146
147         V = np.diag(v_vals.flatten())
148         W = np.diag(w_vals.flatten())
149
150         x_update += (-2 * num_nodes * beta * result -
151                     alpha * (L @ (V @ x_vals)) -
152                     (L @ (V @ ld_vals)))
153
154         y_update += (-L_dash @ (W @ y_vals) -
155                     2 * num_nodes * beta * res -
156                     alpha * A @ (L @ (V @ x_vals)) -
157                     A @ (L @ (V @ ld_vals)))
158
159         ld_update += L @ (V @ x_vals)
160         v_update += -L @ v_vals
161         w_update += -L_dash @ w_vals
162
163         x += delta * x_update
164         y += delta * y_update
165         ld += delta * ld_update
166         v += delta * v_update
167         w += delta * w_update
168
169         sle = max(0, sleep_time - (time.time() - u_st - sleep_time *
170                                   (xt - 1)))
171         time.sleep(sle)
172
173         if ii % gaps == 0:
174             x_val_list.append(x.tolist())
175             y_val_list.append(y.tolist())
176             ld_val_list.append(ld.tolist())
177             v_val_list.append(v.tolist())
178             w_val_list.append(w.tolist())
179
180             log_state(f"x_{node_id}.csv", node_id, ii, x)
181             ii += 1
182
183 def clear_csv(file_path):
184     with open(file_path, 'w') as file:

```

```

184         pass
185
186 if __name__ == "__main__":
187     q = 0
188     ii = 1
189     iterations = 50
190     alpha = 0.1
191     iter = 1
192
193     clear_csv(f"x_{node_id}.csv")
194     clear_csv(f"y_{node_id}.csv")
195     clear_csv(f"ld_{node_id}.csv")
196
197     with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
198         s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
199         s.bind((local_ip, port_num))
200
201     while True:
202         data, _ = s.recvfrom(65535)
203         if data:
204             message = json.loads(data.decode('utf-8'))
205             if message.get("init") == "INIT":
206                 print("INITIATED")
207
208                 nodes = message["nodes"]
209                 neighbors = message["neighbors"]
210                 iterations = message["inum"]
211                 iter = message["iter"]
212                 alpha = message["alpha"]
213                 num_nodes = message["num_nodes"]
214                 sleep_time = message["sleep_time"]
215                 beta = message["beta"]
216                 gamma = message["gamma"]
217                 delta = message["delta"]
218                 A = message["A"]
219                 b = message["b"]
220                 gaps = message["gaps"]
221                 node_id = message["node_id"]
222                 strt = message["strt"]
223                 indegree = message["indegree"]
224
225                 num_nodes = len(neighbors) - 1
226
227                 p = len(A)
228                 q = len(A[0])
229                 A = np.array(A)
230                 b = np.array(b)
231
232                 time_list = []
233                 x = np.zeros((q, 1))
234                 y = -1 * np.array(b).reshape((p, 1))
235                 ld = np.zeros((q, 1))
236                 v = np.full((q, 1), strt)
237                 w = np.full((p, 1), strt)
238
239                 x_vals = np.zeros((num_nodes * q, 1))
240                 y_vals = np.zeros((num_nodes * p, 1))
241                 ld_vals = np.zeros((num_nodes * q, 1))

```

```

242     v_vals = np.zeros((num_nodes * q, 1))
243     w_vals = np.zeros((num_nodes * p, 1))
244
245     x_val_list.append(x)
246     y_val_list.append(y)
247     ld_val_list.append(ld)
248     v_val_list.append(v)
249     w_val_list.append(w)
250
251     x = x.astype(np.float64)
252     y = y.astype(np.float64)
253     ld = ld.astype(np.float64)
254     v = v.astype(np.float64)
255     w = w.astype(np.float64)
256
257     x_vals = x_vals.astype(np.float64)
258     y_vals = y_vals.astype(np.float64)
259     ld_vals = ld_vals.astype(np.float64)
260     v_vals = v_vals.astype(np.float64)
261     w_vals = w_vals.astype(np.float64)
262
263     A = A.astype(np.float64)
264     b = b.astype(np.float64)
265
266     time.sleep(2)
267     u_st = time.time()
268
269     for i in range(1, iterations + 1):
270         run_consensus(s, alpha, iter, i)
271         if i % gaps == 0:
272             time_list.append(i)
273             print(i)
274
275     x_array = []
276     for i in range(q):
277         xy = []
278         for j in range(0, iterations // gaps):
279             xy.append(x_val_list[j][i])
280             x_array.append(xy)
281
282     end_t = time.time()
283     print(end_t - u_st, "seconds")
284
285     for i in range(q):
286         plt.plot(time_list, x_array[i], label=f'v{i +
287             1}(t)')
288
289     plt.xlabel('Iterations')
290     plt.ylabel('States')
291     plt.title(f'States Evolution of Node {node_id}')
292     plt.legend()
293     plt.grid(True)
294     plt.show()
295     break

```

The above code is one used for node 1. Every other nodes will have similar code except that they will have different local IP address and port numbers based on the place and application. The same above code is used in Raspberry Pi and also in multi-file

implementations.

Multi-File Implementation

For multi-file implementation, we run the node codes in the same device with same local ip address = "127.0.0.0" and with different port number for communication. We then run the coordinator to initiate the waiting nodes. The nodes then exchange information with each other to give results.

Raspberry Pi LAN Implementation

Raspberry Pi LAN Implementation is very similar to the multi-file implementation, except that one has to do a lot of set ups in Raspberry Pi which includes finding the IP address setting up the environment and so on. Once the Raspberry Pi set ups are over everything else are pretty much the same.

Raspberry Pi WiFi Implementation

Again here in the case of WiFi implementation we face the main problem of bringing all node agents in the same network. In LAN we connect all the nodes to a switch. This makes the nodes in LAN case come under same local network. Whereas for the case of WiFi implementation, we came with the idea of VPN where all the nodes need to be added in the same VPN where that virtual network becomes a local network for UDP communication. Here, in our case we used ZeroTier VPN services to bring all nodes in the same local network. After this set up it is the same as LAN implementation. We observe vast difference in results for LAN and WiFi cases due to the speed of the data transfer. WiFi has very slow speed whereas LAN communication is very fast

Bibliography

- [1] D. P. Bertsekas and J. N. Tsitsiklis, *Parallel and Distributed Computation: Numerical Methods*. Belmont, MA, USA: Athena Scientific, 1997.
- [2] B. D. O. Anderson, S. Mou, A. S. Morse, and U. Helmke, “Decentralized gradient algorithm for solution of a linear equation,” *Numerical Algebra, Control and Optimization*, vol. 6, no. 3, pp. 319–328, 2016.
- [3] P. Srivastava and J. Cortés, “Solving linear equations with separable problem data over directed networks,” *IEEE Control Systems Letters*, vol. 6, pp. 596–601, 2021.
- [4] P. Srivastava and J. Cortés, “Network optimization via smooth exact penalty functions enabled by distributed gradient computation,” *IEEE Transactions on Control of Network Systems*, vol. 8, pp. 1430–1441, September 2021.
- [5] X. Wang, J. Zhou, S. Mou, and M. J. Corless, “A distributed linear equation solver for least square solutions,” in *2017 IEEE 56th Annual Conference on Decision and Control (CDC)*, (Melbourne, VIC, Australia), pp. 5955–5960, IEEE, 2017.
- [6] S. Mou, J. Liu, and A. S. Morse, “A distributed algorithm for solving a linear algebraic equation,” *IEEE Transactions on Automatic Control*, vol. 60, pp. 2863–2878, Nov. 2015.
- [7] A. Rai, S. Mou, and B. D. O. Anderson, “A distributed algorithm for solving linear equations in clustered multi-agent systems,” *IEEE Control Systems Letters*, vol. 7, pp. 1909–1914, 2023.
- [8] A. Cherukuri and J. Cortés, “Distributed algorithms for convex network optimization under non-sparse equality constraints,” in *2016 54th Annual Allerton Conference on Communication, Control, and Computing (Allerton)*, (Monticello, IL, USA), pp. 452–459, 2016.
- [9] K. Arrow, L. Hurwitz, and H. Uzawa, *Studies in Linear and Non-Linear Programming*. Stanford, CA, USA: Stanford University Press, 1958.
- [10] A. Cherukuri and J. Cortés, “Asymptotic stability of saddle points under the saddle-point dynamics,” in *2015 American Control Conference (ACC)*, (Chicago, IL, USA), pp. –, IEEE, 2015.
- [11] D. P. Spanos, R. Olfati-Saber, and R. M. Murray, “Dynamic consensus for mobile networks,” in *Proceedings of the 16th IFAC World Congress*, (Prague, Czech Republic), IFAC, 2005. Preprint.

- [12] S. S. Kia, B. V. Scoy, J. Cortes, R. A. Freeman, K. M. Lynch, and S. Martinez, “Tutorial on dynamic average consensus: The problem, its applications, and the algorithms,” *IEEE Control Systems Magazine*, vol. 39, pp. 40–72, June 2019.
- [13] L. Xiao, S. Boyd, and S. Lall, “A scheme for robust distributed sensor fusion based on average consensus,” in *IPSN 2005. Fourth International Symposium on Information Processing in Sensor Networks*, (Boise, ID, USA), pp. 63–70, 2005.
- [14] D. Feijer and F. Paganini, “Stability of primal–dual gradient dynamics and applications to network optimization,” *Automatica*, vol. 46, no. 12, pp. 1974–1981, 2010.
- [15] S. Boyd and L. Vandenberghe, *Convex Optimization*. Cambridge: Cambridge University Press, 2004.
- [16] B. Gharesifard and J. Cortés, “Distributed continuous-time convex optimization on weight-balanced digraphs,” *IEEE Transactions on Automatic Control*, vol. 59, no. 3, pp. 781–786, 2014.